Table of Contents

pc_keepalive.8.4

pc_keepalive.8.5

pc_mtd.9.01

pc_mtd.9.02

pc_mtd.9.03

pc_mtd.9.04

pc_mtd.9.05

pc_mtd.9.06

pc_mtd.9.07

pc_mtd.9.08

pc_mtd.9.09

pc_mtd.9.10

pc_mtd.9.11

pc_mtd.9.12

pc_mtd.9.13

pc_mtd.9.14

pc_mtd.9.15

pc_mtd.9.16

pc_mtd.9.17

pc_mtd.9.18

pc_mtd.9.19

pc_mtd.9.20

pc_mtd.9.21

pc_psets.5.1

pc_psets.5.2

pc_psets.5.3

pc_psets.5.4

pc_psets.5.5

pc_psets.5.6

pc_psets.5.7

pc_shutdowns.6.1

pc_shutdowns.6.2

pc_shutdowns.6.3

pc_shutdowns.6.4

pc_shutdowns.6.5

pc_shutdowns.6.6

pc_shutdowns.6.7

pc_testerconfig.7.1

pc_testerconfig.7.2

pc_testerconfig.7.3

pc_testerconfig.7.4

pc_testerconfig.7.5

pc_testerconfig.7.6

ProgrammingConceptsDefTopic

< >

About IG-XL Programming Concepts

About IG-XL Programming Concepts

This information covers general IG-XL programming concepts that affect multiple instruments or tools. The following concepts are included:

- Alarms
- DIB Access
- Hot Switching

- Parameter Sets (PSets)
- Shutdowns
- Tester Configuration
- Keepalive
- Multiple Time Domains

For more information, see Introduction to VBT in the VBT Syntax Reference.

---

pc_alarms.2.01

<	>

Alarms

Alarms

An instrument alarm indicates that an instrument state was not at its programmed value or exceeded programmed limits when a measurement was made, thus invalidating the measurement. Most instruments can detect one or more different alarm conditions. An alarm condition can be caused by a bad device, an incorrectly programmed instrument, or an instrument that has not completely settled before the measurement. When instrument alarms are detected, IG-XL can respond in a number of ways according to your specification.

This section covers general alarm programming. Specific alarm detection capabilities for each instrument can be found in the instrument's documentation. The following topics are included:

- Key Terms
- Alarm Window
- Programming Instrument Alarm Behavior
- Datalogging Alarms
- Alarms in a Pattern
- Setting Alarms Using Debug Displays
- Debugging Alarms

- Alarms Debug Display

For information on instruments that support alarms, click Related Topics.

---

pc_alarms.2.02

< >

Alarms : Key Terms

Key Terms

The following terms are important to understanding alarms and how alarms work:

alarm behavior	The action an instrument takes when an alarm condition occurs. You can program IG-XL to respond to alarms in several ways.
alarm window	A period of time when any alarm that occurs is reported. An alarm window is open during pattern execution and when a measurement is being made. Alarm windows can be nested. Measurements that cause the alarm window to open include: • DC meter strobes • AC waveform captures • Functional patterns
latched alarm	Any alarm that occurs is latched by the hardware. A latched alarm is only reported if it occurs during the time the alarm window is open.
real time alarm status	The current status of an alarm. If an alarm is currently active, it is called a “real time alarm.” Alarms can be activated at any point during a test program. If latching is enabled, a “real time alarm” immediately becomes a latched alarm. The alarm display shows real time alarms. Knowing the particular place in your test program flow when an alarm becomes active

	is useful in debugging alarm conditions. Clearing an alarm's latched bit while it is currently active has no effect.
reported alarm	A latched alarm that was present during the period of time the alarm window was open. The alarm is captured, stored, and reported after a measurement, capture, DSP procedure completes, or a pattern stops.
transient alarm	A condition that causes an alarm on an instrument but is typically not reported because no alarm window was open. Setting up an instrument may cause an alarm to occur. For example, programming a power supply may cause the overcurrent alarm to occur. Since this was due to a programming sequence and not the device, an alarm is not reported.

---

pc_alarms.2.03

<	>

Alarms : Alarm Window

Alarm Window

An alarm is always latched whether it happened during instrument setup or when a measurement was occurring. Some alarms are of concern, such as those that occur during a measurement, and others are not, such as those that occur during instrument setup. Therefore, IG-XL only reports latched alarms when the alarm window is open.

An alarm window is a period of time that begins at the start of a measurement, a capture, or a pattern start. When the alarm window opens, all alarms are cleared, and it remains open for the time required to finish the measurement that forced the alarm window to open. When the window closes, IG-XL checks for alarms and reports any it finds.

The period of time the window is open varies depending on the type of measurement. For a meter reading, the window remains open only long enough for the measurement to be made. For a capture, the window remains open until all data is captured.

When multiple measurements occur near or at the same time throughout the test system, the alarm window uses nested levels to be certain alarms are not incorrectly cleared. See Alarm Window Nesting for details.

The steps that occur when an alarm window opens are as follows:

1.	An event occurs to initiate opening an alarm window, such as a meter strobe.
2.	A global alarm clear is executed, clearing all instrument alarms.
3.	If additional events occur, the alarm window becomes nested, but alarms are not cleared.

4.	An alarm check reports any alarms found each time an event ends, and the alarm window is decremented by one.

5.	An alarm check performed at an alarm window at level 1 closes the alarm window. The next time an alarm window is opened a global clear is executed.

Alarm Window Nesting

When multiple measurements are made at the same time in a test execution, the alarm window has nesting levels to track alarms for the different measurements. Each event that would require the alarm window to be open increases the nesting level of the alarm window if the window is currently open.

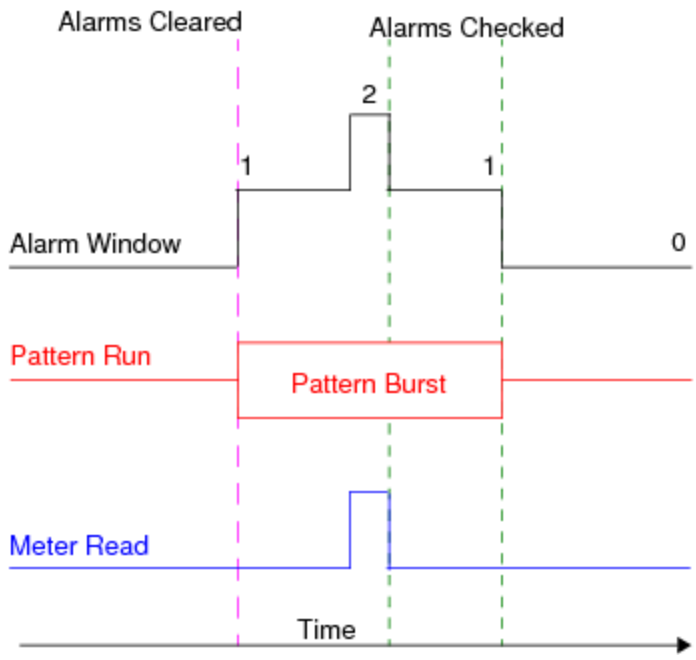
The following figure shows how the nested alarm window works.

1.	The alarm window opens at the start of a pattern run and is considered to be at nested level one.

2.	While the pattern is still in execution a meter read is initiated, making the alarm window increment by one to become a nested level two.

3.	The alarm window returns to level one after the read meter is done.

4.	The alarm window closes once the pattern ends.



Alarm Window Nesting Levels

You can read back the alarm window nesting level using the following VBT code:

```
Dim NestingLevel As Integer
NestingLevel = TheHdw.Alarms.WindowNesting
```

An integer returned as a 0 means the alarm window is closed. An integer of 1 or greater indicates the nesting level of the window.

< >

2015 Teradyne, Inc. All rights reserved.

pc_alarms.2.04

<	>

Alarms : Alarm Window : Alarm Checking

Alarm Checking

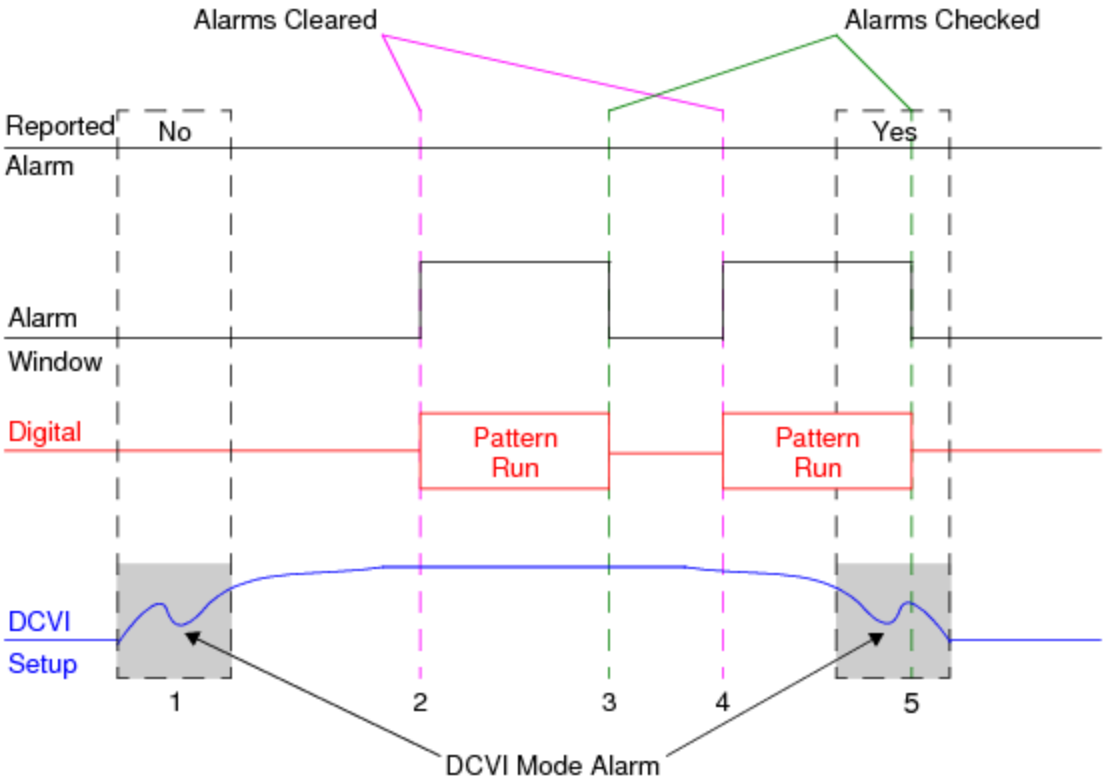
By default, alarms are enabled and checked at the end of measurements and patterns and when capture results are retrieved. Each alarm occurs when a hardware component reaches a limit that it should not. For example, a DCVI may reach its current clamps when forcing voltage or its voltage clamps when forcing current. An alarm can be caused by a bad device or an incorrectly programmed instrument.

Latched alarms are typically checked during:

- A capture
- A measurement
- A pattern burst

Since alarms can occur during the setup of an instrument, all alarms are cleared right before a measurement period begins. The exception is pattern alarms which are cleared at pattern start. Any alarm that occurs after the clear is latched by the instrument and reported after the measurement completes.

The following figure shows the occurrence of several alarms and which of them would be reported:



Alarm Latching Example

The following sequence occurs:

1.	A transient alarm is encountered. Since no measurements are being made, the alarm window is not open so the alarm is not reported.
2.	The alarm window opens, and alarms are cleared before the pattern run starts.
3.	The pattern ends, the alarm window is closed, and alarms are checked. Since the DCVI is settled, there is no alarm at this point.
4.	The alarm window opens, and alarms are cleared before the pattern run starts.
5.	During the pattern run, the DCVI is reprogrammed and encounters a mode alarm before it settles to the new programmed values. This mode alarm is reported since the alarm occurs while the window is open. The alarm is reported in the IG-XL output window when the pattern stops.

Alarms and Captures

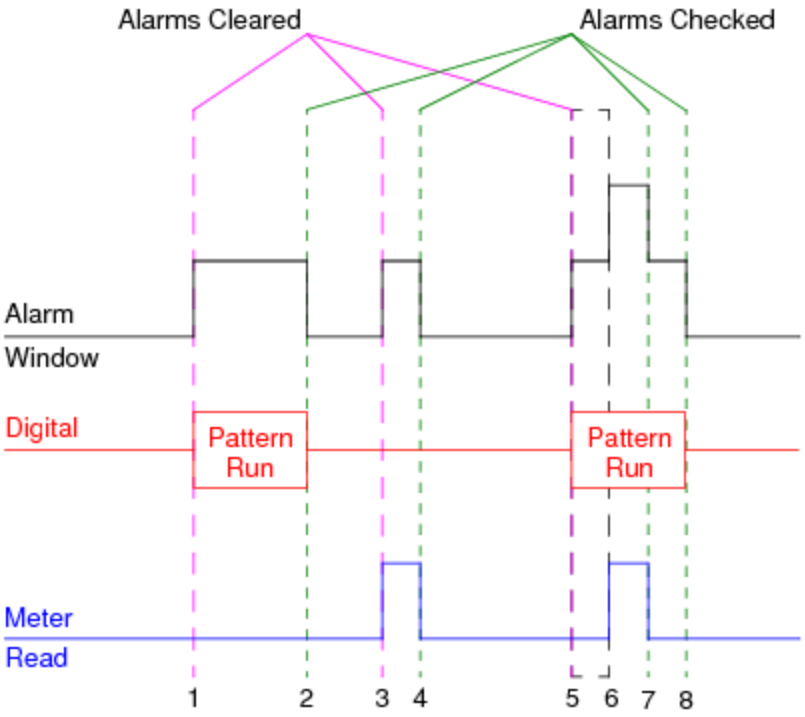
Since captures are performed in hardware, IG-XL cannot know that a capture has concluded until the instrument has moved the data to the host, at which point IG-XL checks alarms. Setup changes can cause transient alarm conditions. Therefore, changing the setup of any instrument after a

capture is initiated, but before its data is moved, may result in reported alarms. To handle these alarms see Transient Alarms in a Pattern.

Alarms and Measurements

IG-XL counts the initiation of measurements and bursts, so that it can continue to check for alarms until all measurements or bursts have concluded. IG-XL does not clear alarms before initiating a measurement or burst while another is already in progress.

The following figure shows how IG-XL handles clearing and nesting the alarm window in a test program when the alarm window is already open.



Alarm Windows with Multiple Measurements at One Time

The following sequence occurs:

1.	Alarms are cleared before the first pattern starts.
2.	Alarm checking is performed.
3.	Alarms are cleared before the first meter read starts as no other alarm window is open.
4.	Alarm checking is performed.
5.	Alarms are cleared before the second pattern as no other alarm window is open.

6.	Alarms are not cleared at the start of the second meter read as an alarm window is open and a pattern is running. A nested level is added to the alarm window.

7.	Alarm checking is performed for the nested level.

8.	Alarm checking is performed.

The TheExec.Flow.TestLimit VBT method, however, checks the count of measurements and bursts. If a measurement or burst is still in progress when it executes, it performs an alarm check and resets the counter.

Controlling the Alarm Window

You can also force IG-XL to close an event-driven alarm window before the end of the pattern, capture, or measurement. To do this, use the following VBT method:

TheHdw.Alarms.CloseAlarmWindow

This can be useful when, for example, you want to use deferred limits. In this case, the capture completes, but you do not want to transfer and process the captured data immediately. IG-XL closes the alarm window only when you transfer the captured data. You can force the window to close to ensure proper reporting of any alarms that occur between the end of the capture and the transfer of your data.

Starting Alarm Monitoring Manually

IG-XL opens an alarm window when you are perform a measurement, burst a pattern, or capture data. If you need to monitor alarms during events during which IG-XL would not normally do so, you can start your own alarm monitoring. For example, you may want to check for improper responses when powering up the device or switching device modes. IG-XL provides VBT language to help you manage the alarm window.

You can also use these methods as a debugging aid:

•	To open an alarm window, use the following VBT method:

TheHdw.Alarms.StartMonitoringAlarms

The alarm window you open with this method cannot coincide with one that IG-XL opens.

•	To close an alarm window that you opened, use the following VBT method:

TheHdw.Alarms.StopMonitoringAlarms

•	To check the status of an alarm window you've opened, read the following VBT property:

TheHdw.Alarms.MonitoringStatus

It is good practice to do this before beginning an event to make sure that your alarm window does not overlap an IG-XL alarm window.

For more information on VBT alarm syntax, see Alarms in the VBT Syntax Reference.

---

pc_alarms.2.05

< >

Alarms : Programming Instrument Alarm Behavior

Programming Instrument Alarm Behavior

You can program how IG-XL responds to alarms individually or by instrument using VBT language. You can set behavior to one of the following:

Force Bin	If an alarm of the specified type occurs, the test fails and the part is binned to the error bin.
Off	Disables the specified alarm in hardware, thus removing the time overhead normally required to process alarms.
Force Fail	If an alarm of the specified type occurs, the test fails.
Default	Programs the alarm to the default action, which is force fail for most instruments. See the alarms information for any default settings.
Continue	If an alarm of the specified type occurs, the alarm is both recorded and reported. However the alarm has no effect on the pass/fail state of the current test or binning. Unlike the Off behavior, alarms are processed and the required processing time is required.

Programming Alarm Behavior Using VBT

You can program alarm behavior using VBT code using an instrument's .Alarm property. (Some instruments do not support this property.) For example:

- Specify that all alarms for the DCTime channel connected to the pin are programmed to the alarm behavior of force bin:

```
TheHdw.DCTime.Pins("Pin").Alarm(tlDCTimeAlarmAll) = tlAlarmForceBin
```

- Specify that the DCTime capture alarm specifically is programmed to the alarm behavior of force bin:

TheHdw.DCTime.Pins("Pin").Alarm(tlDCTimeAlarmCapture) = tlAlarmForceBin

For information on syntax for a specific instrument, see the VBT Syntax Reference.

VBT includes syntax for managing alarms. These properties and methods control alarm behavior throughout the test system. For more information, see TheHdw.Alarms.

You can also manage alarms for many instrument types. For more information, see help for TheHdw.<inst>.Pins.AlarmLatching and TheHdw.<inst>.Pins.AlarmClear, where <inst> is the instrument for which you want to manage alarms.

Programming Alarm Behavior Using a Debug Display

You can program alarm behavior using a debug display. If an instrument supports alarms, its debug display includes alarm controls similar to those shown in the figure. The Behavior control contains the selections found in Programming Instrument Alarm Behavior.

Alarm Programming Using a Debug Display

Alarms differ from instrument to instrument, but alarm behavior is the same for all instruments. Alarm behavior programmed using a debug display is programmed for the current site displayed by the debug display, unless you select the Apply to All Sites TDE button ( ).

You can also use the Alarms Debug Display to manage all test system alarms.

Handling Alarms Using an Exec Interpose Function

IG-XL includes an interpose function, OnAlarmOccurred, that is called immediately before reporting alarms. This function appears in the Exec_IP_Module code module. You can use this function to include your own program control in the alarm handling procedure.

IG-XL passes the function a tab-delimited list of the error messages. You can parse the message string to display a list of the alarms, or you can search for a specific alarm. The template for new test programs, Empty.xlt, contains a commented default function that you can use as the basis for your own implementation.

For information on writing exec interpose functions, see Exec Interpose Functions in the DataTool section.

< >

2015 Teradyne, Inc. All rights reserved.

pc_alarms.2.06

< >

Alarms : Datalogging Alarms

Datalogging Alarms

When the TheExec.Flow.TestLimit method is executed, IG-XL closes the alarm window and datalogs the test results. If an alarm has occurred, the indicator "(A)" is written to the datalog. Ideally, the alarm should be logged with the test that caused the alarm. But, if multiple results are captured before the limits check is executed, the alarm can be logged only after the limits check. As a result, the "(A)" indicator might not be associated in the datalog with the test that raised the alarm.

If you want ideal matching of datalog and alarm for a test, you should capture data and then call TestLimit immediately. If you capture multiple results and then call TestLimit later, there is the possibility of a mismatch.

The .TestLimit method closes the alarm window immediately, regardless of whether a test is deferred or nondeferred, and does not wait for any measurement or capture currently in progress to complete. This method also does not wait for results to arrive from background DSP. For tests that include AC or DC capture acquisitions, take care to ensure that the alarm window is not closed before capture is complete. Otherwise, alarms may not be reported.

To ensure that capture is complete and that all alarms are reported, use the .WaitForCaptureDone property for the AC capture instrument before executing .TestLimit.

< >

pc_alarms.2.07

< >

Alarms : Alarms in a Pattern

Alarms in a Pattern

Alarm behavior in a pattern is at the same settings programmed before the pattern started. Once a pattern starts, individual alarm behavior cannot be changed although all alarms can be disabled and reenabled per instrument.

☑ **Note:**

If an alarm is disabled before a pattern starts, that alarm cannot be enabled inside the pattern.

If the following syntax precedes the start of a pattern:

TheHdw.DCVI.Pins("PinName").Alarm(tlDCVIAlarmMode) = tlAlarmOff

then during the pattern run any mode alarms on the specified pin are not reported at the alarm reporting event, even if the Enable_Alarms microcode is used in the pattern.

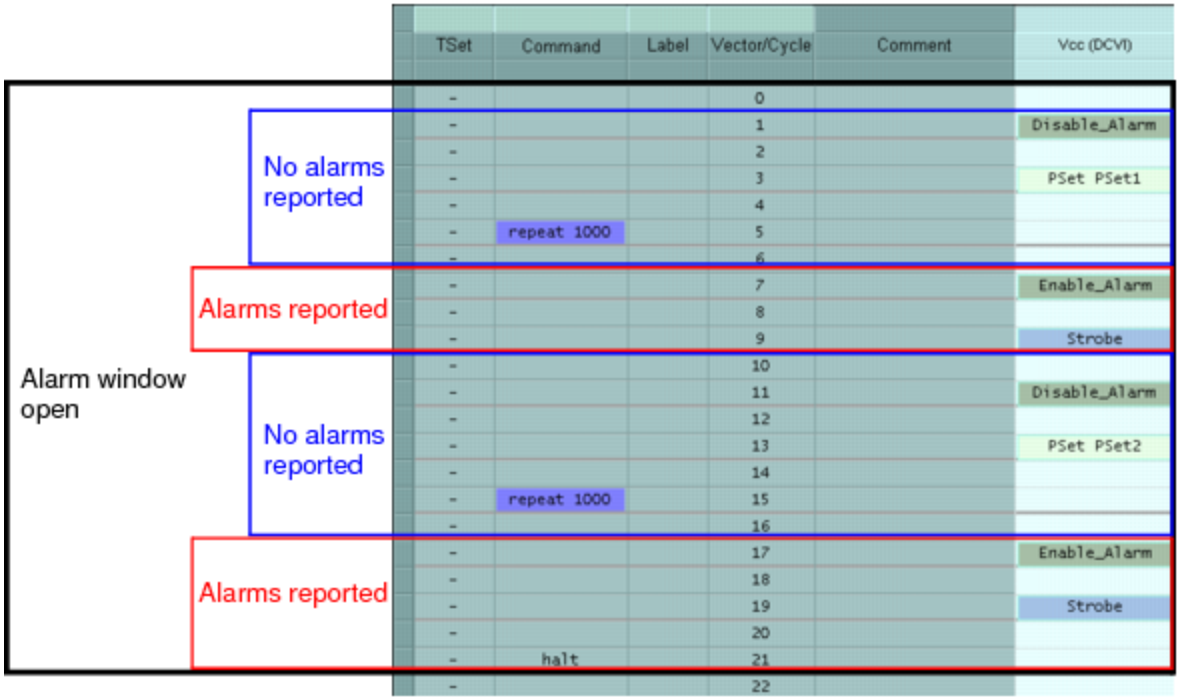
The alarm window is opened when a pattern is started and remains open until the pattern is halted or put in keepalive mode. While the alarm window is open, any alarm that is enabled and occurs is captured and stored until an alarm reporting event.

The following figure shows a pattern that is started after the mode alarm for the DCVI pin was disabled in VBT. The alarm window is open for the entire pattern, but any mode alarms for that specific pin are ignored, due to the alarm being disabled in VBT before the pattern began.

```
TheHdw.DCVI.Pins("PinName").Alarm(tlDCVIAlarmMode) = tlAlarmOff
```

Pattern Following Disabled Mode Alarm in VBT

The following figure shows a pattern using the Enable_Alarm and Disable_Alarm microcode to avoid latched alarms when PSets are being applied. The alarm window is open for the entire pattern, but alarms are only latched when the alarms are enabled.



Pattern Using enable_alarm and disable_alarm Microcodes

Once a pattern ends, alarm behaviors are programmed to the state that was set before the pattern began.

☑ **Note:**

Never disable alarms unless you are certain they are of no value or will not affect the measurement or test.

Rules for Alarms in a Pattern

- Alarm settings before a pattern starts apply to the pattern.
- Alarms can only be enabled or disabled in a pattern.
- Disable_alarms disables all alarms for the specified pin. No individual control of alarms is provided.
- Enable_alarms only enables alarms that were enabled before the pattern start. An alarm disabled in VBT before a pattern remains disabled even when the enable_alarm microcode is set.
- At pattern end all alarms are programmed to the state they were in before the pattern began.

pc_alarms.2.08

< >

Alarms : Alarms in a Pattern : Transient Alarms in a Pattern

Transient Alarms in a Pattern

Since an instrument setup can be changed during a pattern (using PSet programming), an alarm that occurs during the pattern may be a transient alarm and have no affect on the test results. The following DCVI example highlights this issue:

1. A pattern starts, and the alarm window opens.
2. The current of a DCVI is reprogrammed using a PSet.
3. The DCVI reaches the voltage clamp before settling to the new programmed current.
4. The mode alarm is latched and stored.
5. An alarm check is made followed by a measurement and no alarm occurs.
6. The latched alarm from step 4 is reported.
7. The test fails due to a transient alarm.

Determining Transient Alarms in a Pattern

If alarms are occurring in a pattern and you are changing an instrument's setup in the pattern, you may want to determine if the alarms are transient alarms. One way to do this is to rerun the metering portion of your test or use enable_alarm and disable_alarm in a pattern.

If the alarm reported from the pattern is not reported now, then the alarm that occurred was probably a transient alarm.

Programming to Avoid Transient Alarms

To avoid transient alarms you can add a large wait time between the instrument setup and the instrument measurement.

You can also disable the occurring alarm for the instrument you are setting up. Program the instrument as required and then re-enable all alarms before making the measurement.

Use enable/disable microcodes in a pattern to do this when making the measurement or capture inside the pattern. Use VBT language to do this when making a measurement outside the pattern.

< >

2015 Teradyne, Inc. All rights reserved.

pc_alarms.2.09

< >

Alarms : Setting Alarms Using Debug Displays

Setting Alarms Using Debug Displays

Instrument debug displays include several controls that allow you to program alarm behaviors. These controls generally look as follows:

Example Debug Display Alarm Controls

You can only set all alarms to a certain behavior, or a single alarm to a certain behavior at a time. However, you can set the behavior of more than one alarm as the instrument remembers each alarm's state even if you change the controls to display a different alarm.

To program more than one alarm to a different behavior, do the following:

1.	Using the Alarm control, select the alarm you want to set a behavior for.
2.	Select the behavior for that alarm from the Behavior control.
3.	Select the next alarm you want to set a behavior for from the Alarm control.
4.	Select the behavior for that alarm from the Behavior control.
5.	If you select the first alarm you programmed a behavior for, it still has the behavior you assigned to it. You can do this for each alarm available to the instrument.

You can also use the Alarms Debug Display to manage all test system alarms.

pc_alarms.2.10

<	>

Alarms : Debugging Alarms

Debugging Alarms

The first evidence that an alarm has occurred is the presence of an alarm message in the system output window and an “A” appended to the measured value in the datalog file. (For information on how datalog indicates an alarm, see Notes on Window and Text Output.)

Use the following general steps to identify an alarm:

1.	Put a breakpoint in the Flow on the test that has the alarm.
2.	Run to the test.
3.	Bring up the appropriate debug displays.
4.	Step through the test noting the point at which the alarm message appears in the output window.
5.	Activate the alarming instrument’s debug display and perform a Read/Trigger or otherwise force an alarm detection. Watch the system output window when doing this. If a new alarm message is written, the alarm condition is present.
6.	If the alarm is present, experiment with changes to the instrument setup followed by a Read/Trigger.
7.	If the alarm is not present, the alarm condition is transient. Problems of this kind are frequently caused by attempting a measurement before the instruments have fully settled.

	Experiment with Wait time before performing the measurement.

If you use the Alarms Debug Display, you can also follow the procedure in Debugging Alarms Using the Alarm Debug Display.

<	>

pc_alarms.2.11

< >

Alarms : Alarms Debug Display

Alarms Debug Display

An alarm alerts you to an unexpected condition in your test program. In a large test program with many instruments and tests, you may see multiple alarms. You can use the Alarm Debug Display to help determine the problem that caused each alarm, which instrument and channel raised the alarm, and where in your test program it occurred. Individual instrument debug displays alert you when an alarm occurs on the instrument and pin you are viewing in the display; the Alarm Debug Display gathers information on all alarms in the test system.

Because the display requires information from a device and actual hardware, you can use the Alarm Debug Display on a test system only. The display does not work properly on the simulator.

This section covers the following topics:

- Starting the Alarms Debug Display
- Using the Alarms Debug Display
- Debugging Alarms Using the Alarm Debug Display

Starting the Alarms Debug Display

To start the Alarms Debug Display:

1. Load and validate your test program.
2. Set at least one site active.

3.	Start TDE.

4.	Select TDE>Tools>Alarms.

The Alarms Debug Display opens.

< >

2015 Teradyne, Inc. All rights reserved.

pc_alarms.2.12

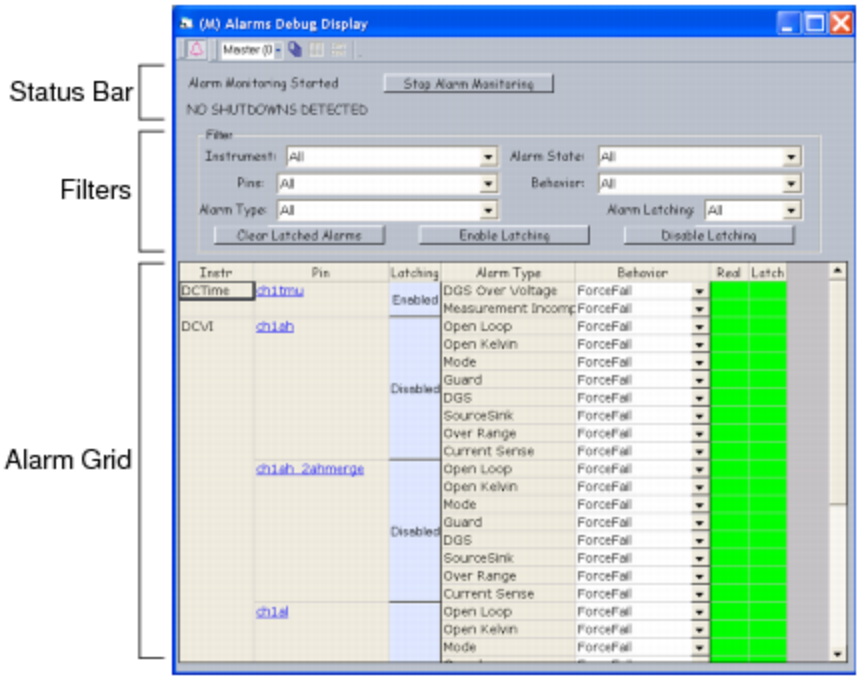
< >

Alarms : Alarms Debug Display : Using the Alarms Debug Display

Using the Alarms Debug Display

The Alarms Debug Display has the following sections:

- Status Bar
- Filters
- Alarm Grid



Alarms Debug Display

Status Bar

The Alarm Debug Display's status bar shows the state of alarm and shutdown monitoring.

The alarm label in the upper left of the status bar shows whether alarms are monitored and displays either Alarm Monitoring Started or Alarm Monitoring Stopped.

To start monitoring alarms in a test program, click Start Alarm Monitoring. This opens an alarm window when IG-XL would not normally do so and is equivalent to the VBT method TheHdw.Alarms.StartMonitoringAlarms.

To cancel monitoring for alarms in a test program, click Stop Alarm Monitoring. This closes any open alarm windows, whether you opened them explicitly or IG-XL opened them automatically. This is the equivalent of the TheHdw.Alarms.StopMonitoringAlarms and TheHdw.Alarms.CloseAlarmWindow VBT methods.

The shutdown label in the lower left of status bar shows the shutdown status and reads NO SHUTDOWN DETECTED. If IG-XL detects a shutdown, the label turns red and reads SHUTDOWN DETECTED. For more information, see Shutdowns.

Filters

You can use the following filters to help you select the pins that appear in the alarm grid:

Instrument	Selects only channels connected to the instrument type specified.
Pins	Selects only the pin specified.
Alarm Type	Selects only channels that can report alarms of the type specified.
Alarm State	Selects alarms in one of the following states: • All: Shows all channels that match other criteria. • Latched: Shows only channels that match other criteria and have latched alarms. • RealTime: Shows only channels that match other criteria and are generating alarms. • Latched or RealTime: Shows only those channels that match other criteria and have latched or current alarms.
Behavior	Shows only channels that can act on alarms in the manner specified.
Alarm Latching	Shows only channels with alarm latching in the state specified. Options are All, Enabled, and Disabled.

The following buttons operate on all channels and alarms visible in the alarm grid:

Clear Latched Alarms	Clears alarms. Note that, if an instrument is alarming and latching is enabled, the alarm relatches immediately.
Enable Latching	Enables alarm latching.
Disable Latching	Disables alarm latching.

Making a selection in one list does not change the contents of other lists. For example, selecting DCVI from the Instrument list does not limit the Pins list to DCVI pins. Take care not to select filter options that produce an empty list.

You can select one item in each list.

Operations in the alarm grid, such as enabling alarm latching, apply only to the pins that appear in the grid.

Alarm Grid

The alarm grid contains information in the following columns:

Instr	Lists, in alphabetic order, all instruments that your test program uses. This column is read-only.
Pin	Lists, in alphabetic order, all pins in the test program. Arranged by instrument. To open the debug display for the channel, click the pin name.
Latching	Shows whether IG-XL reports alarms on this pin. If latching is enabled, IG-XL reports alarms. When latching is disabled, IG-XL ignores alarms. If you disable latching after an alarm occurs, that alarm latches but subsequent alarms do not. To change a pin's latching state, click the pin's cell in the Latching column.
Alarm Type	Lists possible alarms for the channel type. This column is read-only.
Behavior	Lists the current behavior for each alarm type. The contents of this list depend on the alarm type. To set another behavior for an alarm type, select the new behavior from the list box to the right of the type.
Real	Shows whether the instrument behind the pin is in an alarm state. The color of this cell shows the alarm state:

	• Red indicates that the instrument is causing an alarm. • Green indicates that the instrument is operating normally. This cell returns to green when you resolve the cause of the alarm. This column is read-only.
Latch	Shows whether the instrument behind the pin has any unreported alarms. The color of this cell shows the alarm state: • Red indicates that an alarm has occurred but has not been reported and cleared. • Green indicates that no outstanding alarms exist for the instrument. This cell remains red after you resolve the cause of the alarm until you clear all alarms for the channel by clicking Clear Latched Alarms or until IG-XL closes the alarm window and performs the alarm check. This column is read-only.

Changes you make to these settings apply to the currently selected site. To apply your changes to all active sites, enable Apply All Sites ( ).

< >

2015 Teradyne, Inc. All rights reserved.

pc_alarms.2.13

<	>

Alarms : Alarms Debug Display : Debugging Alarms Using the Alarm Debug Display

Debugging Alarms Using the Alarm Debug Display

Use the following general steps to identify an alarm:

1.	Put a breakpoint in the Flow on the test that raised the alarm.
2.	Debug Run to the breakpoint.
3.	Start the Alarm Debug Displays (and any other debug displays you know you will want).

You can start other displays from the Alarm Debug Display.

4.	Use filters in the Alarm Debug Display to display only those pins you want to debug.
5.	Step through the test and note the colors of the Real and Latched columns.

When a cell in the Real column turns red to indicate an alarm, start or switch to the alarming instrument's debug display and adjust values until the cell in the Alarm Debug Display's Real column turns green.

If the alarm is not present, the alarm condition is transient. Problems of this kind are frequently caused by attempting a measurement before the instruments have fully settled. Experiment with Wait time before performing the measurement.

6.	Debug other alarms or continue your test program run.

pc_dibaccess.3.1

<	>

DIB Access

DIB Access

DIB access is one way to connect multiple instruments to a single DUT pin without using additional relays on a DIB. In describing a DIB access configuration involving two or more kinds of instruments, the two classes of instruments are:

Primary instrument	The instrument that covers most test requirements as well as the most demanding test requirements.
Secondary instruments	Additional instruments needed for some tests.

This distinction is made because the performance of secondary instruments may be degraded.

☑ **Note:**

Configure the most important instrument as the primary instrument.

This section includes the following topics:

- DIB Access Physical Connections
- DIB Design of DIB Access Lines
- Programming a DIB Access Connection

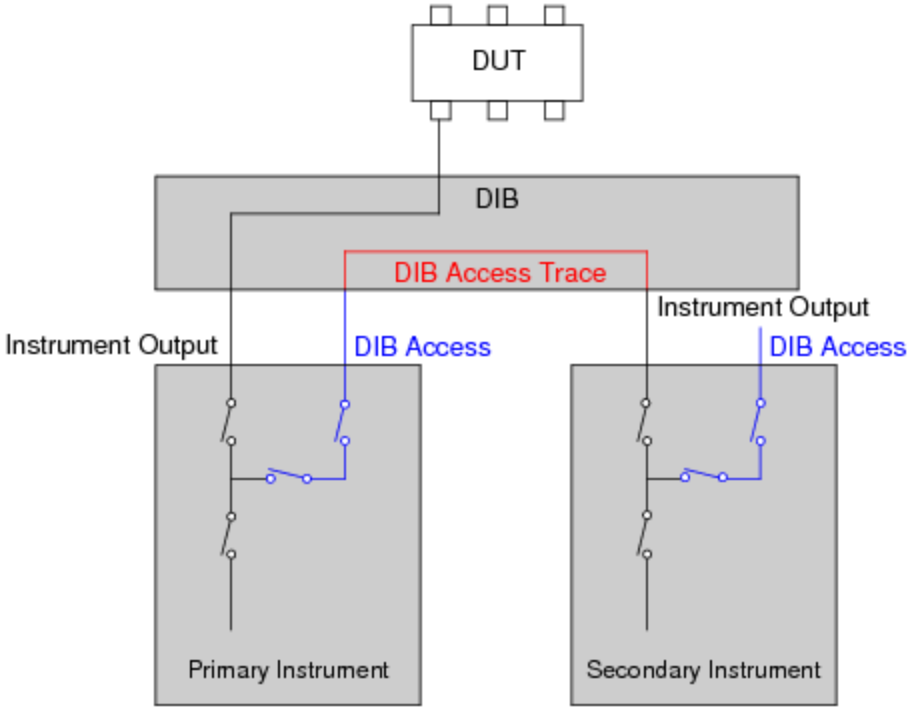
pc_dibaccess.3.2

< >

DIB Access : DIB Access Physical Connections

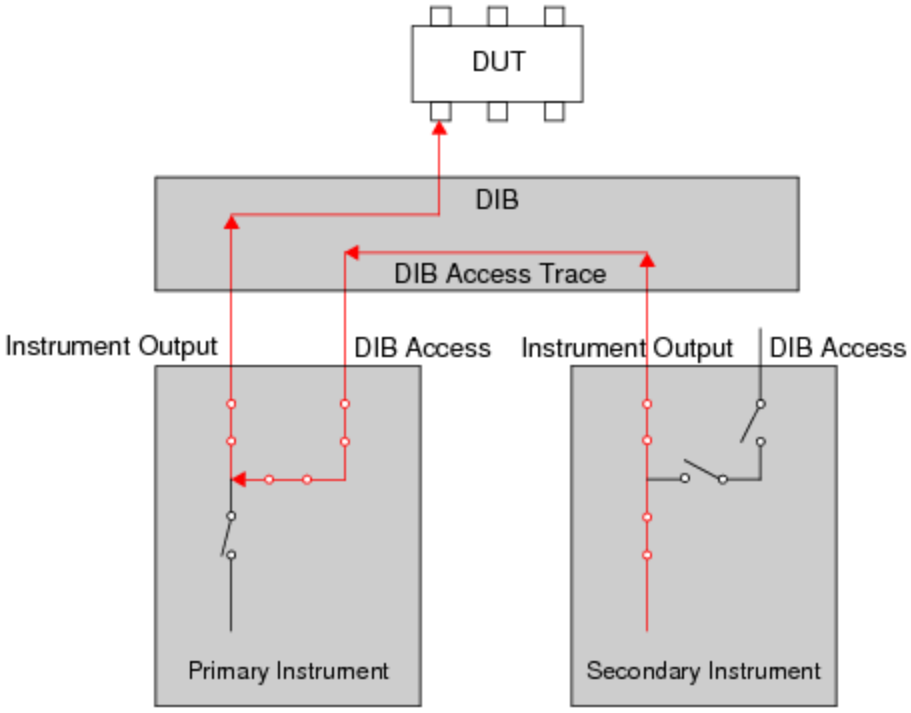
DIB Access Physical Connections

Some UltraFLEX instruments use DIB access lines to connect a DUT pin to multiple tester resources. DIB access lines are connections between a DUT pin and an instrument using another instrument's signal path. To use instrument DIB access pins, traces must be incorporated in a DIB to provide the physical connection between the instruments and the DUT pins. The figure shows the connection of a primary instrument to a DUT pin and to a secondary instrument using the primary instrument's DIB access to connect to the DUT pin.



DIB Access Connection

DIB access connections allow a signal from a secondary instrument to travel to the DIB access pin of the primary instrument. This pin connects to a DUT pin. The following figure shows the signal path of a secondary instrument's main output traveling into the DIB access connection of the primary instrument to reach the DUT pin. The arrows show the direction of the signal path.



Signal Path of a DIB Access Connection

☑ **Note:**

The primary instrument cannot source a signal through DIB access to a secondary instrument.

---

pc_dibaccess.3.3

< >

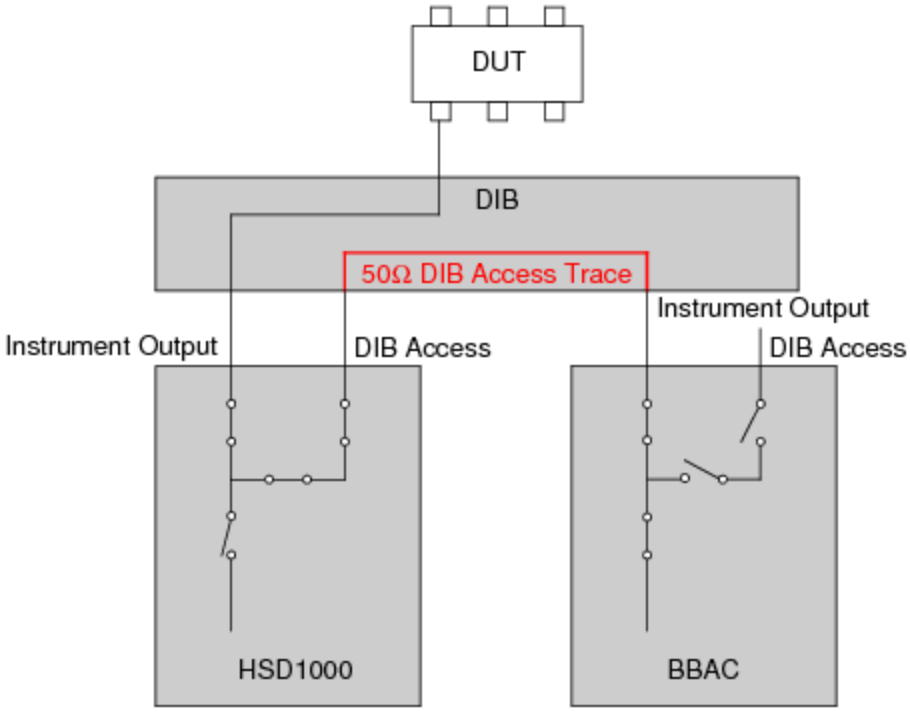
DIB Access : DIB Design of DIB Access Lines

DIB Design of DIB Access Lines

The number of access lines available varies by instrument. Consult each instrument's specification for details about incorporating DIB access lines in a DIB layout.

The bandwidth of a DIB trace is limited by the path of the primary instrument, not the secondary instrument, since the signal of the secondary instrument travels into the DIB access of the primary instrument.

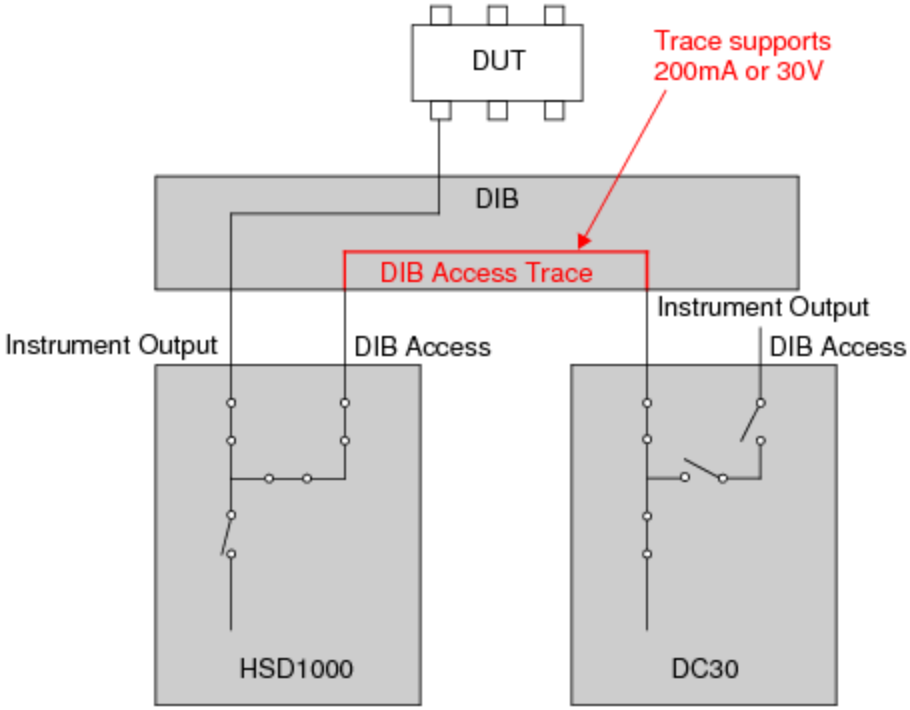
Use 50ohm traces to connect an instrument to the DIB access of another instrument if the trace is intended to carry an AC signal. The figure shows a 50ohm trace connecting the BBAC to the DIB access of an HSD1000.



DIB Trace Requirements for an AC Instrument

To use a DIB access trace as a DC path, design the trace to support the current that it is expected to flow through it. The following figure connects a DC30 to the HSD DIB access requiring the DIB

access line trace to be able to support the 200mA or 30V sourced by the DC30.



DIB Trace Requirements for DC Instrument

When designing a DIB that requires DIB access, be sure that any instrument's DIB access can handle the voltage/current of the instrument that is using its DIB access. For example, the HSD1000 has a maximum voltage of 7V while the HexVS can source up to 90V.

< >

2015 Teradyne, Inc. All rights reserved.

pc_dibaccess.3.4

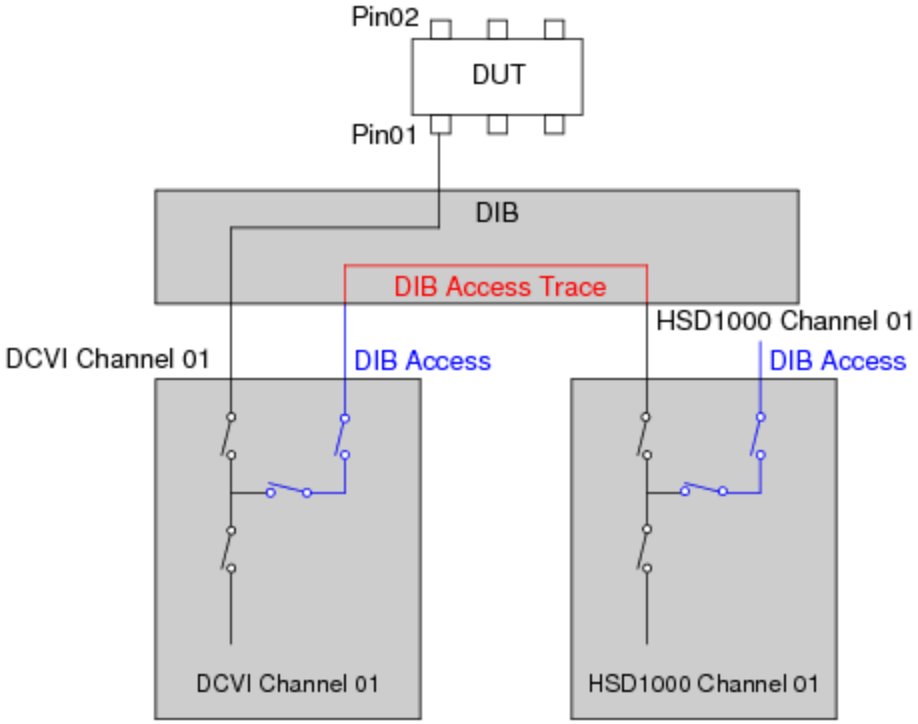
< >

DIB Access : Programming a DIB Access Connection

Programming a DIB Access Connection

If a DIB has the proper wiring, you can program instruments using DIB access. The first requirement is determining which instrument is the primary instrument and which are the secondary instruments. The figure shows a DCVI connected to a DUT pin as the primary instrument with an HSD1000 pin using the DIB access of the DC board to connect to the same pin.

The following topics are included:



Example DIB Access Setup

---

pc_dibaccess.3.5

< >

DIB Access : Programming a DIB Access Connection : Pin Map

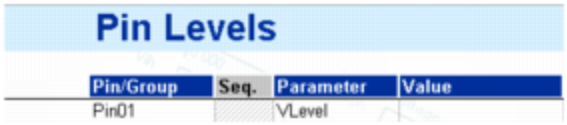
Pin Map

You create a single pin map entry for each DUT pin even when multiple pins are connected to the DUT pin using DIB access. The Type entered in the pin map directly affects the parameters listed in the Pin Levels sheet when the pin name is added.

Only one instrument resource in the DIB access connection can have Pin Levels associated with it, and that is determined by the pin Type.

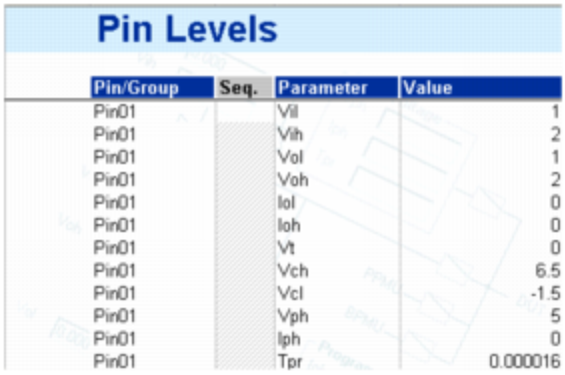
If DUT Pin01 is listed in a pin map as Type Analog and then entered in a Pin Levels sheet, the levels associated with the Analog instrument listed in the channel map are displayed.

The figure shows the pin levels for Pin01 if the pin map type is Analog and the channel map type is DCVI.



Pin Levels for Pin Type Analog

If DUT Pin01 is listed in the pin map as type I/O and then entered in a Pin Levels sheet, the levels associated with the I/O instrument listed in the channel map are displayed. This figure shows the pin levels for Pin01 if the pin map type is I/O and the channel map Type is I/O.



Pin Levels for Pin Type I/O

The Pin type selection does not affect instrument programming with VBT other than that the pin is limited to having pin levels for only one of the instruments connected using DIB access.

---

pc_dibaccess.3.6

< >

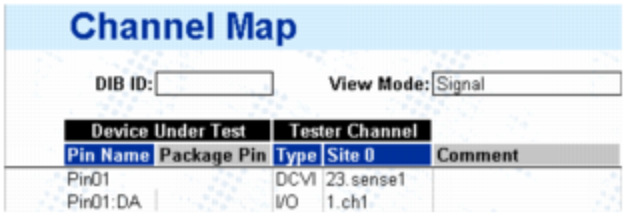
DIB Access : Programming a DIB Access Connection : Channel Map

Channel Map

In a Channel Map sheet, the row that specifies a pin's primary instrument uses the pin name as defined in the pin map. The rows that specify a pin's secondary instruments use this syntax to indicate that the instruments are connected to the pin by DIB access:

<pin-name>:DA

The following figure shows the primary instrument as a DCVI listed first, followed by the secondary instrument listed as I/O using the :DA extension.



Channel Map Setup for DIB Access

Note the following:

- You can use the :DA syntax only on a pin name that has appeared earlier in the channel map.
- The :DA syntax must be used on the row or rows immediately following the row defining the primary instrument.
- You cannot use DIB access between two resources assigned to the same pin name if both resources use the same channel map channel type. For example, a DC30 DCVI cannot use a DC75 DCVI through DIB access because both the channels use the DCVI channel type.

- Under some conditions the full functionality of the secondary instrument may not be available when routing through an analog power pin. For example, if the HVD is a secondary instrument, you can use it as an analog HVPMU resource, but not as a digital HVPE resource (which requires levels and timing programming).

< >

2015 Teradyne, Inc. All rights reserved.

pc_dibaccess.3.7

< >

DIB Access : Programming a DIB Access Connection : Connecting Through DIB Access

Connecting Through DIB Access

Once a DIB is wired and DataTool is set up, the instruments can be connected to the DUT pin. Connecting to a pin through DIB access is done in the same manner as connecting any other pin.

☑ **Note:**

You must manage all connections using DIB access. Connect only one instrument to a DUT pin at a time.

Primary Instrument

To connect the primary instrument to a DUT pin, use any standard programming method for the instrument. If the primary channel is a DCVI, for example, you could also use VBT language to make this connection:

TheHdw.DCVI.Pins(“Pin01”).Connect tlDCVIConnectDefault

Secondary Instrument

To connect a secondary instrument to a DUT pin use any standard programming method for the instrument.

If the secondary channel is a HSD1000 for example, you could also use VBT language to make this connection:

TheHdw.Digital.connectPins ("pin01")

< >

pc_dibaccess.3.8

< >

DIB Access : Programming a DIB Access Connection : Programming Instruments Using DIB Access

Programming Instruments Using DIB Access

Programming instruments involved in a DIB access configuration is the same as for instruments not in a DIB access configuration. Use VBT language, for the specific instrument you want to program. For example, to program the BBACSRC you could use BBACSRC VBT syntax.

Programming instruments through DIB access requires a knowledge of both the primary and secondary instrument specifications. You would not want to program high power instruments through DIB access if the primary instrument cannot handle the output power. Make sure that the primary instrument will not be damaged by the secondary instrument.

For information on instrument specific DIB access considerations, click Related Topics.

< >

pc_hotswitching.4.1

< >

Hot Switching

Hot Switching

Changing the state of a relay while current is flowing through the relay, called “hot switching,” shortens the life of the relay and, under some conditions, can damage the device under test. When possible, minimize or eliminate hot switch in test programs that use instruments, such as DC instruments, that can change relay states while current is flowing through the relays. It is good practice to minimize or eliminate hot switching in test programs.

This section describes hot switching, its effects, and how to avoid it. The following topics are included:

< >

pc_hotswitching.4.2

< >

Hot Switching : Understanding Hot Switching

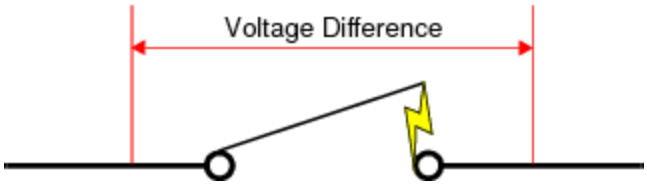
Understanding Hot Switching

Hot switching is closing a relay with a voltage across the relay contacts or opening a relay with current flowing through the contacts. If voltage and current levels are sufficient, power arcs between the relay and the contact. Many instruments, particularly DC instruments, use physical relays in the signal delivery lines. Opening or closing these relays when a voltage potential exists or when current is flowing can damage and shorten the life of the relays.

Hot switching occurs any time you change the state of a relay with differing potentials on either side of the relay and have current sufficient to cause an arc between the two sides. This can happen when you:

- Close a relay with differing potentials on either side
- Open a relay while current is flowing
- Set compliance voltages incorrectly for instruments that support compliance
- Program DCVI voltages incorrectly

The following figure demonstrates a hot switch condition.



Hot Switch Condition

The types of hot switches are:

live voltage	Live voltage hot switching occurs when you: • Connect a relay while a voltage difference exists across the relay • Disconnect the relay while current is flowing The result is an arc between one terminal and the relay.
settle wait	Changing the state of a relay is a physical operation and does not occur instantaneously. During any connect or disconnect operation, relays “bounce” briefly before settling. Sending power across the relay before it settles causes arcing and can damage the relay.

Physical relays have a life expectancy, or approximate number of switches before failure, which varies according to the voltage and current levels sent across the relay. Hot switching shortens a relay’s life expectancy and can cause early failures. This may require repair or replacement of the board containing the failed relay.

For a DCVI channel, hot switching happens when you connect or disconnect the force line while current is flowing. Note that current can flow even if you have a channel gated off. Because sense lines use a different type of switch, you can connect and disconnect them while current is flowing without risking damage.

< >

2015 Teradyne, Inc. All rights reserved.

pc_hotswitching.4.3

<	>

Hot Switching : Finding Hot Switching in a Test Program

Finding Hot Switching in a Test Program

IG-XL includes VBT syntax that allows you to locate and fix unintended hot switching in your test program. Using this syntax, you can enable h detection for live voltage or settle wait hot switching. Performing hot switch detection involves the following steps:

1.	Enable hot switch detection, either live voltage or settle wait.
2.	Program what action to take when a hot switch condition is detected.
3.	Optionally, disable hot switch detection at the end of the code you want to check.
4.	Run the test program.
5.	Check the test program when an error is returned or check the logged results for hot switches.

The following topics are available:

- Enabling Live Voltage Hot Switch Detection
- Enabling Settle Wait Hot Switch Detection
- Setting IG-XL Response to a Hot Switch

- Selecting Code for Hot Switch Detection

- Limitations of Hot Switch Detection

Enabling Live Voltage Hot Switch Detection

A live voltage hot switch condition occurs when you connect or disconnect a DCVI force line while current runs through the relay. Enable detection of live voltage hot switching using the following statement:

TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.Enable = <True | False>

Hot switch detection is off by default. Once you enable hot switch detection, it is active on all DCVI channels in the test system.

Live voltage detection checks relay connects and disconnects.

For connects, IG-XL compares the measured DUT voltage with the current VI voltage. (If Gate is off, 0 is used for the VI voltage.) If the connect threshold, the maximum difference between the DUT and VI voltage, is exceeded, IG-XL reports a hot switch condition. See Setting IG-XL Response to a Hot Switch.

The default threshold is 1V. You can change the connect threshold using:

TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.ConnectThreshold

Note that a hot switch condition can also occur in the simulator if the voltage threshold is exceeded. If hot switching is not required in such cases, you can disable it using:

TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.Enable = False

For disconnects, IG-XL first calculates the power of the DUT using:

(Measured Current) x (Measured Voltage)

IG-XL then compares the calculated power with the disconnect threshold. The default disconnect threshold is:

- 10mW for a single channel

- 20mW for 2-channel merged channels

- 30mW for 3-channel merged channels

- 40mW for 4-channel merged channels

If the power exceeds the threshold, IG-XL reports a hot switch condition.

At the same time, IG-XL also checks that DUT current does not exceed 10mA for a single channel. For merged channels, the threshold is 10mA times the number of merged channels. So, for a 4-

channel merged channel, the DUT current threshold is 40mA.

You can change the disconnect threshold using:

TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.DisconnectThreshold

Do not enable both live voltage and settle wait hot switch detection at the same time. Enabling live voltage detection changes settling times, and settle wait detection then does not work properly.

For more information, see TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.

Enabling Settle Wait Hot Switch Detection

A settle wait hot switch condition occurs when you apply current to a line before the relay completely settles. Enable detection of settle wait hot switching using the following statement:

TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.Enable = <True | False>

Hot switch detection is off by default. Once you enable hot switch detection, it is active on all DCVI channels in the test system.

The settle wait detect sequence is:

1. Enable settle wait hot switch detection in the test program.
2. Run the program.
3. One of the following events occurs:
   - Connect or disconnect the force line
   - Program the V/I
   - Turn the Gate on or off
4. IG-XL turns on three timers for connect/disconnect, V/I, and Gate on/off.
5. The next connect or disconnect occurs in the program.
6. IG-XL checks to see if all three timers have expired.
7. If any timer has not expired, IG-XL reports a settle wait hot switch condition. See Setting IG-XL Response to a Hot Switch.

The default settle wait threshold is 0.5ms. You can change the threshold using: TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.TimeThreshold

Do not enable both types of hot switch detection at the same time. Enabling live voltage detection changes settling times, and settle wait detection then does not work properly.

For more information, see TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.

Setting IG-XL Response to a Hot Switch

You can set the action to take when IG-XL finds a live voltage or settle wait hot switch (if the hot switch detection tool is active). Use the following statements:

TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.Mode

TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.Mode

This property can take the following values:

tlDCVIHotSwitchDetectModeError	Stops the test program with a run-time error at the line that caused the hot switch. This assumes that you do not use the error handler in a test function.
tlDCVIHotSwitchDetectModeErrorAndLogging	Logs the hot switch to a text file and stops the test program with a run-time error at the line that caused the hot switch.
tlDCVIHotSwitchDetectModeLogging	Logs the hot switch to a text file but does not interrupt test program flow. This allows you to note the locations of all hot switches and fix them at your convenience. Default.

⚠ Caution:

If you program IG-XL to halt on error and an error occurs while the DCVI is delivering power to the device, the instrument does not shut down. If continuous power delivery can exceed the limits of your device or the instrument, do not program IG-XL to halt on error.

For more information, see:

- TheHdw.DCVI.HotSwitchDetect.LiveVoltageHotSwitch.Mode
- TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.Mode

IG-XL stores the log files in your temp folder. The default location of this folder is: C:\Users\<userID>\AppData\Local\Temp

where <userID> is the name of your Windows login account.

Live voltage switches are recorded to the file HotSwitchLiveVoltageLog.txt.

Settle wait switches on the DC30 and DC75 are recorded to the file HotSwitchSettleWaitLogDCVI3075.txt.

Selecting Code for Hot Switch Detection

IG-XL performs hot switch detection only after you set the .Enable property to True. This allows you to enable hot switch detection in specific areas of code by surrounding the code with .Enable = True and .Enable = False. For example:

```
TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.Enable = True
TheHdw.DCVI.Pins("MyPin").Connect (tlDCVIConnectDefault)
TheHdw.DCVI.Pins("MyPin").Current = 1
TheHdw.DCVI.HotSwitchDetect.SettleWaitHotSwitch.Enable = False
```

Limitations of Hot Switch Detection

Hot switch detection introduces some overhead processing and can reduce device throughput. IG-XL does not automatically disable hot switch detection in production mode. You must remove these statements manually. The following practices can reduce test time:

- Use this feature only during test development and debug.
- Enable hot switch detection only when connecting and disconnecting DCVI channels, and disable it when the relays do not change state.

Hot switch detection works for DCVI channels only. It does not apply to:

- DCTime relays
- HSD channels
- PPMU relays

PPMUs often operate at lower voltage and current levels than DCVI channels and are therefore less likely to experience hot switching. However, they are susceptible to effects from capacitors and other current and voltage sources.

- DIB Access relays
- User relays on the DIB

In addition, exercise caution under the following conditions:

- A defective device causes a short

- A capacitor requires discharge

Ensure that you allow sufficient time for DIB capacitors to discharge before connecting a DCVI relay to the line. This prevents surges or spikes through the instrument.

---

pc_hotswitching.4.4

<	>

Hot Switching : Preventing Hot Switching for DCVI Channels

Preventing Hot Switching for DCVI Channels

You should make connections when your VI channel is at 0V. This minimizes the current flowing through the relay and reduces the occurrence of hot switching.

Use this procedure to connect a DCVI channel to a device pin at 0V:

1.	Program the DCVI parameters.
2.	Connect the channel.
3.	Program the channel to gate on.

This procedure applies to connecting a channel in a test program using VBT statements and when using the DCVI Debug Display.

Connecting and Disconnecting DC30 and DC75 DCVI Channels

The best way to switch relays is to remove the load before connecting to or disconnecting from the device. If you are connecting, you can reapply the load after you make the connection to the device.

You can switch with low voltage levels without significantly damaging relays. Most damage occurs when you switch at voltages sufficient to cause arcing at the contacts.

Follow these guidelines to avoid hot switching when using DC30 and DC75 DCVI channels:

•	Allow at least 1ms after you make a connection before you gate on.

- For circuits with a large capacitance (such as bulk capacitance for voltage stability), discharge the capacitor and keep it discharged before connecting or disconnecting.

- If possible, make all DCVI connections at the beginning of your test program, then disconnect all channels at the end of the test program.

When disconnecting DCVI force lines, make sure that no current flows from the VI, from the device, or from DIB capacitance. Program the DCVI channel to the slowest bandwidth and lowest current range before you disconnect the instrument.

Related Topics

- Connecting a DCVI

- Connecting a DCVI to a Live DUT Pin

- HotSwitchDetect

< >

pc_keepalive.8.1

< >

Keepalive

Keepalive

Keepalive is a subroutine that keeps a device operating after the completion of a test pattern. The keepalive routine is stored in a protected portion of the subroutine memory (SRM) so the rest of SRM can be reloaded while keepalive is running. Keepalive also allows you to run multiple patterns seamlessly in a burst mode by keeping the device running between patterns. While keepalive is running, test program control is given to the host computer, so you can perform any operation that does not affect the keepalive subroutine, levels, or timing.

This section describes how to create a keepalive subroutine on the HSD1000, UltraPin800, UltraPin1600, and UltraPin4000. It also describes special guidelines for keepalive when using the Digital Signal Source and Capture (DSSC) instrument available on these digital instrument boards.

This section includes these topics:

- Keepalive Concepts
- Keepalive Subroutine Syntax
- Keepalive Programming
- DSSC Keepalive Patterns

For a complete example of keepalive programming, see HSD1000 Keepalive Programming Example.

☑ **Note:**

On the HPM and UltraPin4000, you can change timing edges while in keepalive. You cannot change the timing period while in keepalive.

☑ **Note:**

The keepalive examples in this section are intended to show the correct opcode sequencing and work only in single mode on the HSD1000, UltraPin800, and UltraPin1600 or By1 mode on the UltraPin4000. In dual mode on the HSD1000, UltraPin800, and UltraPin1600, quad mode on the UltraPin1600, and By16 and By16IIO modes on the UltraPin4000, you follow the same opcode sequence, but you must place opcodes on the last vector of each vector group, where a group is a vector pair in dual, four vectors in quad, and 16 vectors in By16.

☑ **Note:**

On the HPM instrument, you can also implement a keepalive subroutine using Logical MicroCode (LMC). For more information, see Using Keepalive in LMC Programming.

< >

2015 Teradyne, Inc. All rights reserved.

pc_keepalive.8.2

<	>

Keepalive : Keepalive Concepts

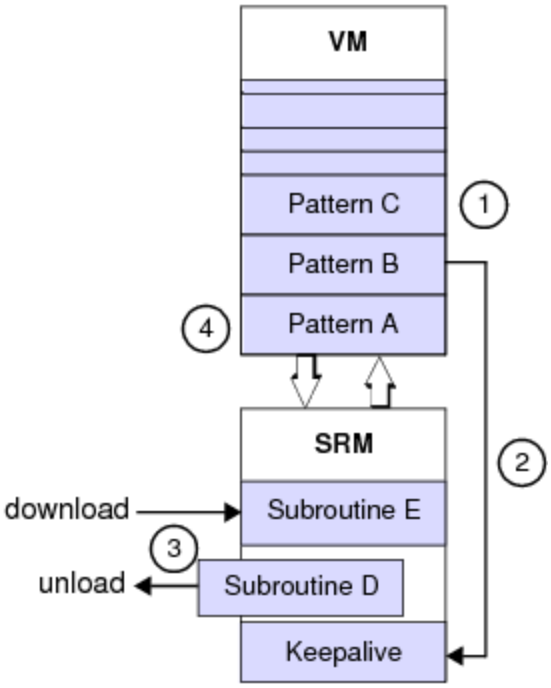
Keepalive Concepts

Keepalive uses IG-XL software to create a “safe” place to initiate a download of subroutine memory (SRM). IG-XL implements keepalive as a user-defined part of the SRM memory that is not overwritten during pattern download. Keepalive subroutines can be up to 1K in size and have the full functionality of any other pattern subroutines.

For example, if you have three patterns (A, B, and C) and patterns A and C both call subroutines, your pattern flow using keepalive would be:

1.	Burst pattern C, which calls subroutine D.
2.	Burst pattern B, then call the keepalive subroutine.
3.	While in keepalive, unload subroutine D and download subroutine E from the VM.
4.	Return to burst pattern A, which calls subroutine E.

All SRM is reloaded except the keepalive subroutine.



Keepalive Example

< >

2015 Teradyne, Inc. All rights reserved.

pc_keepalive.8.3

<	>

Keepalive : Keepalive Subroutine Syntax

Keepalive Subroutine Syntax

Keepalive subroutines allow you to modify test system hardware while the PatGen is running. Keepalive subroutines are different from regular subroutines in that IG-XL knows what resources the keepalive subroutine uses and will not overwrite those resources during the modification of test system memories. The resources that are protected are the locations in the SRM itself which store the keepalive subroutine and the timing memory locations (timesets) used during execution of the keepalive subroutine.

Pattern File Syntax

You create a keepalive subroutine in the same manner as any other SRM subroutine, except that it must use the keepalive keyword to identify its use in keepalive mode. The subroutine must do the following:

- Set the CPU flag that you have selected to communicate with the host computer for keepalive.
- Loop while checking the flag to see if the host computer has reset the keepalive flag.
- When the flag is reset, return.

When a test pattern completes execution while keepalive is enabled, the keepalive subroutine continues to loop until the CPU resets the keepalive CPU flag. While in keepalive, starting another test pattern automatically resets the flag, exits keepalive, and starts the new pattern.

The following example shows how to set up a simple keepalive subroutine that uses the cpuA PatGen flag as the keepalive flag.

```
import tset ts_ka;
srm_vector keepalive_mod( $tset, pin0)
```

```
{
keepalive subr keepalive_ex:
   clr_subr                    > ts_ka   X;
   set_cpu_cond(cpuA_cond)     > ts_ka   X;
KA_Loop:
   branch_expr = (cpuA_cond)   > ts_ka   X;
   if (branch_expr) jump KA_Loop  > ts_ka   X;
   return                      > ts_ka   X;
}
```

☑ **Note:**

Keepalive subroutines must be executable in a single PatGen burst with the patterns that call them. This means that the pins, pin setups, and PatGen opcode mode for the keepalive subroutines and patterns must be the same. IG-XL returns a validation error if any inconsistencies exist.

Each pattern in a job can have its own keepalive subroutine. You make the association between a pattern and a keepalive subroutine by importing the keepalive subroutine label name in the pattern file. If more than one module is contained in the pattern file, the keepalive subroutine is associated with each module in the file. Keepalive subroutine labels are global, which means that a given named label can only be defined in one module.

Another option is to create one keepalive subroutine to be used with multiple patterns. To make the keepalive subroutine available to all patterns, include the name of the pattern file containing the subroutine on the Pattern Subroutine sheet.

☑ **Note:**

On HPM only, you must use the disable_fail_count and enable_fail_count opcodes in your keepalive subroutine for proper operation of keepalive with ActualMode in Waveform Display. Other digital instruments do not support keepalive in ActualMode. HPM does not support the By1_TOF timing mode in ActualMode.

Moreover, when using ActualMode with keepalive patterns, IG-XL automatically disables the Capture All HRAM setup mode. To use Capture All with a keepalive pattern, you must mark all vectors in the pattern with stv, since the Capture STV HRAM mode is compatible with ActualMode.

Example Keepalive Subroutine

The following is a more complex keepalive subroutine that provides more useful features:

```
import tset tska:
// More complicated example keepalive subroutine
srm_vector my_keepalive                  ($tset, pin0)
{
keepalive subr example_keepalive:
        mask disable_fail_count          > tska   0;
```

```
        mask clr_subr                   > tska   0;
        mask set_cpu_cond(cpuB_cond)    > tska   0;
KA_loop:  mask branch_expr = (cpuB_cond)    > tska   0;
        mask if (branch_expr) jump KA_loop  > tska   0;
        mask enable_fail_count          > tska   0;
        mask return                     > tska   0;
}
```

The mask opcode modifier keeps the pattern from failing inside the keepalive subroutine.

The disable_fail_count opcode disables all fail counters:

- Module cycle counter
- Absolute cycle counter
- Time counter

The clr_subr opcode is required to clear the call stack. Entering the keepalive subroutine using the call opcode pushes a return address on to the call stack. Once in keepalive mode, IG-XL pushes the next pattern address on to the stack, returns to that address after exiting keepalive, and pops it off the stack. If you do not use the clr_subr opcode, then the original return address from the keepalive call remains on the stack and is not used again. If you call keepalive multiple times without clearing the call stack, then IG-XL may return a stack overflow error. By including the clr_subr opcode, however, IG-XL removes all “leftover” stack entries and keeps the stack from overflowing. Place the clr_subr opcode before the set_cpu_cond opcode so that IG-XL does not remove the return address that the test system host computer will push on to the stack. If you use the jump opcode to enter the keepalive subroutine, then do not use the clr_subr opcode because jump does not push a return address on to the stack.

Handshaking with the test system host computer is done using the cpuB flag, which has been designated as the keepalive user flag. The keepalive subroutine sets the flag and then waits for the test system host computer reset the flag.

The enable_fail_count enables the fail counters.

The return is used to return either to SRM or VM. The jump_vm opcode is not needed because the test system host computer has pushed the correct SRM or VM address on the stack.

---

pc_keepalive.8.4

<	>

Keepalive : Keepalive Programming

Keepalive Programming

To program a keepalive subroutine, follow these steps:

1.	Create a keepalive subroutine.
2.	Set up test patterns to enter keepalive on completion.
3.	Write a VBT function that enables keepalive.
4.	Write VBT functions to perform desired tasks while keepalive is running.

This can include reloading the SRM, loading DSSC settings, or simply starting another test pattern.

5.	Write a VBT function that disables keepalive.

You should call this function before running patterns that do not need to go into keepalive on completion.

Creating a Keepalive Subroutine

A keepalive subroutine is a CPU flag-controlled looping pattern. It uses pattern syntax to notify the compiler that it is a keepalive routine. In addition, the subroutine must use one of the CPU user flags to loop. You can define multiple keepalive subroutines in a test program, although only one can be active at a time. Each pattern file can have its own keepalive subroutine. If you want to use one keepalive subroutine for the entire test program, however, you can specify the pattern file containing the keepalive subroutine on the Pattern Subroutine sheet.

For more information, see Keepalive Subroutine Syntax.

Setting Up Test Patterns to Enter Keepalive

To enter keepalive after a test pattern completes, the pattern must call the keepalive subroutine. If the pattern is an SRM module, the last vector in the pattern must explicitly call the keepalive subroutine.

```
srm_vector  single_mod( $tset, DRV_PINS RCV_PINS )
{
                > ts_single 0000 XXXX;
call keepalive_sub  > ts_single 0000 XXXX;
}
```

If the pattern is a VM module, you can either call the keepalive subroutine directly or use the soft stop subroutine to enter keepalive. To use a stop subroutine, you write an SRM pattern and use the special label that designates it as a stop subroutine. Note that you cannot mix user-defined and default stop subroutines. Make sure that your VM patterns do not contain a halt opcode. If your pattern contains a halt opcode, it will not enter the stop subroutine after it completes execution. The advantage of using the soft stop subroutine is that you can control whether or not you use keepalive without having to modify your patterns. In the following example, the stop subroutine enters keepalive if the cpuB flag is set. If the flag is not set, the PatGen simply halts.

```
vm_vector  my_vm_stop_subroutine ( $tset, pin0 )
{
stop subr vmStopLabel:
call my_srm_stop_subroutine    > ts0    0 ;
}
import tset ts0:
srm_vector  my_srm_stop_subroutine  ($tset, pin0)
{
stop subr stopsubr:
branch_expr = (cpuB_cond)          > ts0   0;
if (branch_expr) call keepalivesub  > ts0   0;
halt                          > ts0   0;
}
```

☑ **Note:**

The test patterns, keepalive subroutines, and stop subroutine must all be executable in a single PatGen burst. This means that the pins, pin setups, and PatGen opcode mode for all of these patterns must be the same. The exception is that the pins in a keepalive subroutine can be a subset of the pins defined in the rest of the patterns. IG-XL returns a validation error if any of these modules are inconsistent.

For more information, see Stop Subroutines.

Enabling and Disabling Keepalive

You must enable and disable keepalive by creating and executing VBT functions that perform these tasks. Execute this function before starting the first pattern that must enter keepalive. The following example function enables keepalive mode and defines the cpuA flag to control the keepalive loop. The example also uses the cpuB flag as a signal to the stop subroutine to enter keepalive.

```
Public Function KA_setup() As Long
    TheHdw.Digital.Patgen.KeepAlive.Flag = cpuA
    TheHdw.Digital.Patgen.KeepAlive.Enable = True
    TheHdw.Digital.Patgen.Continue cpuB, 0
End Function
```

After keepalive is no longer needed, you must halt the PatGen and disable keepalive. The following example function halts the PatGen, disables keepalive, and clears the cpuB flag so that the stop subroutine no longer calls the keepalive subroutine.

```
Public Function KA_end() As Long
    TheHdw.Digital.Patgen.Halt
    TheHdw.Digital.Patgen.KeepAlive.Enable = False
    TheHdw.Digital.Patgen.Continue 0, cpuB
End Function
```

For more information, see TheHdw.Digital.Patgen.KeepAlive in the VBT Syntax Reference.

Keepalive Execution Sequence

The following test flow sequence describes a typical usage of keepalive.

1.	Run the VBT function that enables keepalive.
2.	Start the test pattern. When the pattern completes, keepalive runs.
3.	While in keepalive, execute the function that performs desired tasks such as SRM reloading or DSSC setup.

Note that if entering keepalive from a longer pattern, make sure that the PatGen is actually in keepalive mode before executing any VBT functions. To do this, use TheHdw.Digital.Patgen.IsKeepAlive (see IsKeepAlive).

4.	Start the next test pattern. Pattern execution flows seamlessly from the keepalive subroutine into this test pattern.
5.	Repeat steps 2-4 until all tests requiring keepalive are completed.

6.	Run the VBT function that stops and disables keepalive.

You can execute tests that require keepalive in several different ways:

- Perform all tests in a single custom VBT function. This function would contain the code to enable keepalive, load and execute patterns, stop pattern execution, and disable keepalive.

- Use VBT test instances, calling the functions for keepalive setup and disable as interpose functions.

- Use VBT test instances, adding custom functions for keepalive setup and disable as steps in the Flow Table sheet.

The third method above is a flexible way to include keepalive in a test program. When calling keepalive from the stop subroutine, this method makes it easy to run with or without keepalive and to change the flow as desired.
In the example below (which shows only two columns from the Flow Table sheet), the SetKeepAlive function enables keepalive, so test instances VMModule1 and VMModule2 run in keepalive mode. The PatGen stops in EndKeepAlive, and VMModule3 runs without keepalive.

Opcode	Parameter
Test	SetKeepAlive
Test	VMModule1
Test	VMModule2
Test	EndKeepAlive
Test	VMModule3

Initial pin states specified in a functional template are not applied in keepalive once the PatGen has been started. If three functional templates are to be run in sequence using keepalive, the initial pin states defined in the first template will be applied before the PatGen is started. Once the PatGen is started, any initial pin states in the second and third template are ignored.
For more information, see Enabling and Disabling Keepalive.

---

pc_keepalive.8.5

<	>

Keepalive : DSSC Keepalive Patterns

DSSC Keepalive Patterns

Keepalive patterns allow you to burst multiple patterns in sequence, or to debug a single pattern, without stopping the pattern generator (PatGen) and reinitializing the device after each pattern or test. You can use keepalive patterns with the DSSC.

DSSC keepalive modules are more flexible than standard digital keepalive patterns. In standard digital patterns, all digital pins and instrument definitions must be identical for all pattern modules in a single burst, including the keepalive module itself. The DSSC instrument definitions in a keepalive pattern module, however, do not need to match the DSSC instrument definitions in other patterns in the PatGen burst. Also, the digital pins in a keepalive module can be a subset of the pins in all other patterns in the burst; you do not need to include all of the digital pins in the keepalive subroutine. Non-DSSC instruments must still have the same instruments statements for all patterns in a pattern set.

Since DSSC instrument definitions can be modified while the PatGen is running in keepalive, you can have different DSSC settings in patterns that occur before and after a keepalive subroutine. To accomplish this, the keepalive subroutine should exclude any digital pins that are associated with any DSSC instruments across the entire burst sequence. The keepalive subroutine should only retain the pins that must be either toggling, or in a specific state to keep the device in the desired condition.

Follow these rules when creating patterns that will modify DSSC instrument settings while the PatGen is running in a keepalive loop:

For Keepalive Pattern Modules

- Create a keepalive subroutine as a separate pattern (.pat) file. This allows a keepalive pattern module to compile even though the digital pin header does not match the other pattern modules.

- Do not include a pin in the keepalive subroutine, if that pin is part of a DSSC instrument in any of the patterns in the burst. (Note that the keepalive subroutine will contain a subset of the original pin list.)

- Pins that appear in the keepalive subroutine must appear in all non-keepalive pattern modules in that burst.

- Include any pins that are not used in the keepalive subroutine in the keepalive pattern module's time set, and set them to the STAY format. This prevents unwanted data transitions in the pattern while in keepalive mode.

- If you use the Local MoveLink to transfer data, include the microcode REPEAT 410 in your keepalive pattern. This allows sufficient time for capture data to transfer over the MoveLink bus.

- Ensure that each pattern module that executes before keepalive captures at least the number of samples specified for the capture segment. If fewer samples are captured, then the segment will not be available while in keepalive, and subsequent segments will not be captured correctly.

- When using auto_trig_enable mode, ensure that each pattern module that executes before keepalive captures no more than the number of samples specified for the capture segment. If more samples are captured, then another capture segment will start before entering keepalive. Segments that you intend to capture after keepalive can contain pre-keepalive data. In auto_trig_disable mode, capturing extra samples is not an issue.

For Non-Keepalive Pattern Modules

- The first non-keepalive pattern module must contain all pins that will be used in that burst. All subsequent non-keepalive pattern modules can contain a subset of that first module's pin list, but they cannot add any new pins. Note that any pins that were excluded from the subsequent modules are still enabled.

IG-XL returns a run-time error if subsequent non-keepalive patterns contain pins that were not in the first pattern. IG-XL only returns this error if extended validation is enabled using TheHdw.Patterns.EnableExtendedValidation(True). Note that in production mode, extended

validation is disabled by default; and in engineering mode, extended validation is enabled by default.

To avoid unexpected behavior, make the digital pin header list identical across all non-keepalive pattern modules during the DSSC keepalive burst.

- Ensure that all non-DSSC instruments header statements are identical across both the keepalive pattern module and the non-keepalive pattern modules that will be called during the burst.

DSSC Keepalive Pattern Example

This example shows how to use DSSC patterns with keepalive. This example uses three patterns to execute a DSSC keepalive test flow:

- DSSC Setup 1 Pattern (patDSSC1.atp): The first DSSC setup pattern calls a keepalive subroutine.
- DSSC Keepalive Pattern (patKA.atp): This keepalive pattern allows changes to the DSSC instrument while the PatGen is running.
- DSSC Setup 2 Pattern (patDSSC2.atp): The second DSSC pattern executes and then halts the PatGen.

The DSSC keepalive test flow is shown below:

1. Enable keepalive using VBT language.
2. Set up the DSSC signals for the first pattern.
3. Start the patDSSC1 pattern.
4. In VBT, wait for patDSSC1 pattern to enter keepalive (TheHdw.Digital.Patgen.HaltWait).
5. Set up the DSSC signals for the second pattern. (PatGen is running in keepalive.)
6. Start the patDSSC2 pattern (from the keepalive loop).

7.	In VBT, wait for the PatDSSC2 pattern to halt.

The following patterns are used in this test flow.

DSSC Setup 1 Pattern (patDSSC1.atp)

```
opcode_mode = dual;
import subr myKA;
import tset ts1;

instruments = {
    (p1,p2,p3,p4):DigSrc 4:msb:parallel;
    (p1,p2,p3,p4):DigCap 4:msb:parallel;
}

srm_vector ($tset,(p1,p2,p3,p4),clk)
{
                                > ts1    0000    1;
((p1,p2,p3,p4):DigSrc = START)              > ts1    0000    0;
                                > ts1    0000    1;
repeat 288                            > ts1    0000    0;

// Burst data.
((p1,p2,p3,p4):DigSrc = SEND)               > ts1    DDDD    1;
repeat 2048 ((p1,p2,p3,p4):DigSrc = SEND)    > ts1    DDDD    0;
                                > ts1    0000    1;
repeat 10000                            > ts1    0000    0;

// Capture data.
((p1,p2,p3,p4):DigCap= STORE)               > ts1    VVVV    1;
repeat 256 ((p1,p2,p3,p4):DigCap = STORE)    > ts1    VVVV    0;
                                > ts1    0000    1;
call myKA                               > ts1    0000    0;
}
```

DSSC Keepalive Pattern (patKA.atp)

```
opcode_mode = dual;
import tset tsKA;

// NOTE: The tsKA time set uses the STAY format for p1, p2, p3, and p4.
```

```
srm_vector ($tset,clk)
{
keepalive subr myKA:
                              > tsKA    1;
   clr_subr                    > tsKA    0;
                              > tsKA    1;
   set_cpu_cond(cpuA_cond)        > tsKA    0;
KA_Loop:
                              > tsKA    1;
   branch_expr = (cpuA_cond)      > tsKA    0;
                              > tsKA    1;
   if(branch_expr) jump KA_Loop    > tsKA    0;
                              > tsKA    1;
   return                      > tsKA    0;
}
DSSC Setup 2 Pattern (patDSSC2.atp)
opcode_mode = dual;
import tset ts1;

instruments = {
   (p1,p2):DigSrc 2:msb:parallel;
   (p3,p4):DigCap 2:msb:parallel;
}

srm_vector ($tset,(p1,p2,p3,p4),clk)
{
                              > ts1    0000    1;
((p1,p2):DigSrc = START)             > ts1    0000    0;
                              > ts1    0000    1;
repeat 288                         > ts1    0000    0;

// Burst data.
((p1,p2):DigSrc = SEND)              > ts1    DD00    1;
repeat 1024 ((p1,p2):DigSrc = SEND)    > ts1    DD00    0;
                              > ts1    0000    1;
repeat 10000                       > ts1    0000    0;

// Capture data.
```

```
((p3,p4):DigCap= STORE)              > ts1    00VV    1;
repeat 1024 ((p3,p4):DigCap = STORE)   > ts1    00VV    0;
                                 > ts1    0000    1;
halt                               > ts1    0000    0;
}
```

< >

2015 Teradyne, Inc. All rights reserved.

pc_mtd.9.01

<	>

Multiple Time Domains

Multiple Time Domains

UltraFLEX allows you to test several different functional parts of a device simultaneously using multiple time domains. A time domain, or logical PatGen, is a set of digital hardware PatGens and any associated analog instruments that operate at the same vector rate (cycle period) during a test. Multiple time domain testing increases efficiency because you can run multiple patterns at multiple data rates in parallel to test multiple functional parts of a device. It also provides enhanced pattern management because you can create and modify each time domain pattern independently. IG-XL supports multiple time domains through Characterization Studio, PatternTool, HRAM capture, the test instance editor, and multisite handling.

This section includes the following topics:

- Key Terms
- UltraFLEX Time Domains
- Multiple Time Domain Testing
- Multiple Time Domain Operation
- Multiple Time Domain Programming
- Multiple Time Domain HRAM Capture

- Multiple Time Domain Programming Examples

Multiple time domains are used when creating concurrent tests. For more information, see: About Concurrent Test.

< >

pc_mtd.9.02

< >

Multiple Time Domains : Key Terms

Key Terms

asynchronous pattern start	Starting a pattern in one or more time domains while patterns are currently running in other time domains.
immediate pattern start	The VBT language concurrent pattern start mode where a pattern is immediately started and concurrency is launched.
synchronous pattern start	A request to start patterns for two or more time domains at the same time. IG-XL sends a command to the corresponding hardware PatGens to begin execution.
system time domain (also called the "All" time domain)	A single time domain that includes all pins defined in a test program. This time domain is created by IG-XL and is present in all test programs. When compiling a pattern file, if pins specified in a pattern file do not match any user-specified time domains, the pattern compiler sets the time domain to the All domain. This can occur during compile and during pattern load. For time domains language (TheHdw.Digital.TimeDomains), if the empty string ("") is specified as the TimeDomains parameter, then the All domain is used. For Timing sheets in DataTool, if the Time Domain field is left blank, then the timing sheet is associated with the All time domain.
time domain	A collection of digital and analog pins and instrument resources sharing synchronized timing and pattern control, used to test one section (or core) of a DUT.

	Every pattern uses a time domain, either the default “All” domain or one you specify in a pin map. Defined time domain pin groups must match, or be a superset of, pins used in the pattern. A pin can be a member of multiple time domain pin group definitions, but neither a pin nor a tester resource can be a member of multiple time domains used simultaneously.
timing and levels context	A combination of timing and levels used in a test instance. This context includes Time Sets and Edge Sets sheets, AC Specs, Pin Levels sheet, and DC Specs.
timing set	A unique combination of timing sheets (Time Sets Basic or Time Sets and Edge Sets) and AC spec Category/Selector. The number of timing sets is limited by the pin slice processor (PSP) on a digital board.

< >

pc_mtd.9.03

<	>

Multiple Time Domains : UltraFLEX Time Domains

UltraFLEX Time Domains

An UltraFLEX time domain (TD) is a set of digital hardware PatGens and any associated analog instruments that operate at the same vector rate (cycle period). Multiple time domain (MTD) testing allows you to burst multiple patterns in parallel to test various parts of a device.

Each TD is independent and can execute its own patterns at the same time as the other domains. Unique patterns (with unique pins) can start simultaneously. This allows you to use a statement such as the following in a concurrent test program:

TheHdw.Patterns(patA & "+" & patB & "+" & patC).Start

Time domain groupings can change throughout a test flow.

Each TD also:

- Supports multisite testing and multiple sites
- Has flexible time scaling (for example, one TD per 128 pins on UltraPin800)
- Has unique master timing, timing sets, CPU hand-shakes, and counters
- Can independently branch, communicate with analog instruments, report test results, and perform other operations

Instrument Support for Time Domains

Time domain support varies by instrument.

For digital options:

- HSD1000, UltraPin800, UltraPin1600, UltraPin4000, SB6G, and UltraSerial10G support multiple time domains. Note that HPM does not support MTD.

- Each digital card can act as its own domain, supporting multiple sites. Multiple cards can be linked to create a logical time domain for higher pin counts

- Each digital domain can have independent timing, including a unique timing sets sheet, time set names, and MOSC timing

- Each digital domain can have its own pattern definitions and unique pins compared to other domains

  - Each time domain supports pattern modules, pattern files, and pattern sets

  - Each domain has its own CPU flags, counters, and branching,

  - Patterns can be started synchronously and can end individually as each pattern completes

All analog instruments support basic analog instrument time domain mapping, which means:

- Instruments on each analog slot can be assigned to only one domain per pattern burst

- Time domain mapping can change on a per pattern burst basis

Some instruments (including UltraWave12G, UltraPAC80, and UltraVI80) can support multiple time domains in one instrument slot.
For more information on MTD instrument support for concurrent test, see Concurrent Support By Instrument in the Concurrent Test section.

< >

pc_mtd.9.04

< >

Multiple Time Domains : Multiple Time Domain Testing

Multiple Time Domain Testing

Testing multiple test device functions historically involved testing each device function serially or combining different functional blocks together and testing them as a single block. The serial method involved creating a separate pattern for each device function and running them one at a time. The combination method involved creating one pattern that contains multiple device functions.

The serial method, however, has limitations:

- Running tests serially increases test time.
- The method does not cover cases where you must test multiple functions simultaneously.

The combination method also has limitations:

- You cannot independently control each pattern using pattern opcodes.
- This method is only as efficient as the slowest functional block being tested. The combined pattern must operate at the rate of the fastest block, and you must fill slower blocks with repeated vectors.
- Creating an edge set, time set, and pattern that combines all functional blocks can be difficult. Sometimes, you cannot create all required data rates using one pattern.

With multiple time domains, you get the improved test time of parallel testing while maintaining independent patterns and timing.

The UltraFLEX can test multiple time domains in two different modes of operation: synchronous and asynchronous.

For more information, see Multiple Time Domain Programming.

Asynchronous Multiple Time Domains

When multiple time domains run asynchronously, the domains execute independently of each other with no restrictions on the relative timing between the domains. Patterns, however, must be started together. One time domain cannot start executing a pattern while another time domain is still running. When time domains are asynchronous, IG-XL does not guarantee that the patterns start with a repeatable phase alignment between the domains, nor does it guarantee that coincident points will occur.

For more information, see Creating MTD Timing.

Synchronous Multiple Time Domains

When multiple time domains are running synchronously, patterns in each domain start with precise time alignment. The time domains retain a fixed relationship with each other throughout the pattern burst based on the relative periods of each domain. Patterns start with a repeatable phase alignment and coincident points will occur based on the frequency relationships between the time domains.

Synchronous operation of multiple time domains allows you to program the domains using the different MOSC periods, and IG-XL handles pattern start repeatability and coincidence points.

For more information, see Creating MTD Timing.

☑ **Note:**

To improve pattern start performance and reduce test time, execute all test instances that test an individual time domain in one block per time domain. Following this guideline reduces switching between time domains that can slow down pattern start. IG-XL only reprograms the instrument sync-link (ISL) at pattern start if the time domain to be started is not a subset of the previous pattern start's time domains.

< >

2015 Teradyne, Inc. All rights reserved.

pc_mtd.9.05

<	>

Multiple Time Domains : Multiple Time Domain Operation

Multiple Time Domain Operation

The UltraFLEX supports multiple time domains where different instrument boards operate at different frequencies to test multiple functions of a device simultaneously. Each time domain executes its own pattern or pattern set at the same time as the other domains, but the time domains are otherwise independent of each other. This means that each time domain can:

- Branch independently
- Handshake with the host independently
- Communicate with analog instruments independently
- Report test results independently
- Perform a test bin as a logical AND function of all time domain results

The UltraFLEX supports up to four PatGen time domains with no restrictions on PatGen time domain configuration. For the HSD1000, UltraPin800, and UltraPin4000 digital instruments, a time domain consists of one or more instrument boards, and each instrument board can belong to only one time domain. On UltraPin1600, each 128-channel half board can be a member of a unique PatGen time domain, but only one half can be grouped with other instrument boards to create a Logical Pattern Generator (LPG) time domain due to synchronous reject (SR) line restrictions on the test head. The other half board can only operate as a local PatGen time domain. All UltraPin1600 Protocol Aware (PA) time domains are independent of these restrictions. On UltraPAC80, UltraVI80,

and UltraWave12G, individual instrument channels on a single board can belong to different time domains.

You can program more than four time domains, but you must follow rules for board slot assignments. Specifically, up to four time domains can consist of boards contained in both the upper and lower slots of the test head. You can create more time domains, but these time domains must reside entirely in the upper or lower half of the test head.

For more information, see Multiple Time Domains in the UltraFLEX DIB Design Guidelines.

☑ **Note:**

You must start a pattern on all time domains at the same time, whether or not they are running synchronously. However, patterns running in different time domains can end at different times. You must wait for the PatGen to stop before starting another pattern on any time domain.

Note the following limitations for multiple time domains:

- Only one time domain can talk to analog instruments in one burst.

- IG-XL performs error checking for conflicting time domains at run time. You can define conflicting time domains, as long as they are not used in the same pattern burst. During validation, however, IG-XL does not check if two domains overlap. So, make sure that your test program does not have overlapping time domains.

< >

pc_mtd.9.06

<	>

Multiple Time Domains : Multiple Time Domain Programming

Multiple Time Domain Programming

The following topics describe how to set up a test program that uses multiple time domains:

- Defining Time Domains
- Creating MTD Timing
- Programming MTD Pin Levels
- Creating MTD Patterns
- Using MTD Pattern Sets
- Programming MTD Test Instances
- Executing VBT Code on Time Domains
- MTD Flag Handshaking
- Datalogging MTD Tests

pc_mtd.9.07

< >

Multiple Time Domains : Multiple Time Domain Programming : Defining Time Domains

Defining Time Domains

Pin Map Sheet Programming

Define time domains on a Pin Map sheet. Assign a name to the time domain in the Group Name column and list all pins that belong to that time domain. Use the TimeDomain keyword in the Type column to define the time domain. Repeat this procedure for all time domains. Only include pins that will be controlled from a pattern in a time domain pin group.

For more information, see Creating Multiple Time Domains in the DataTool sheet reference section.

Channel Map Sheet Programming

You must follow certain rules when assigning channels to time domains. The basic rule is that each instrument board can only belong to one domain. You cannot split channels on one instrument to multiple time domains. This equates to a 64 channel boundary on HSD1000 (one domain per slot), a 128 channel boundary on UltraPin800 (one domain per slot) and a 128 channel boundary on UltraPin1600 (two domains per slot).

HSD1000, UltraPin800, and UltraPin1600 boards can be combined in the same time domain, but UltraPin4000 boards cannot be in the same time domain as the other two instrument types.

For more information on channel selection, see Multiple Time Domains in the UltraFLEX DIB Design Guidelines.

< >

pc_mtd.9.08

< >

Multiple Time Domains : Multiple Time Domain Programming : Creating MTD Timing

Creating MTD Timing

Each time domain must have its own Time Sets and Edge Sets sheets or Time Sets (Basic) sheet. Applying timing associates pins and timesets to a specified domain.

Specify the name of the time domain in the Time Domain field on these sheets. The field contains a drop down list of all time domains specified on the Pin Map sheet. Program timing according to the instrument type contained in the time domain that you are using.

IG-XL returns a validation error if you create a multiple time domain test using a timing sheet that does not specify a time domain.

If you are testing multiple time domains asynchronously, you do not need to do anything else. Each time domain will be programmed as specified in the sheets and no synchronization between time domains will occur.

If you are testing multiple time domains synchronously, you should program a Fractional Bus sheet. This sheet provides flexibility in defining frequency relationships between time domains while also providing a synchronous pattern start between domains. For more information, see Fractional Bus Sheet and Fractional Bus Sheet MTD Example for Synchronous Pattern Start.

Alternatively, you can specify a master time set to force all time domains to use the same MOSC period. Enter the name of the Time Sets sheet that will be used to calculate the MOSC period in the Master Timeset Name field of the other Time Sets sheets. In cases where you are using the UltraPin4000 synchronously with an HSD1000, UltraPin800 or UltraPin1600, you must specify the UltraPin4000 as the master time set. This is because the UltraPin4000 is more restrictive in what MOSC periods it can program. When using this approach, you must also include the following VBT statement in the OnProgramLoaded function:

TheHdw.Digital.Timing.MOSCFullRange = True

This property sets the unified MOSC range from 1.0GHz to 2.0GHz for all digital instruments that ensures proper MOSC calculation. For an example that uses this alternative method, see HSD1000

and UltraPin4000 MTD Loopback Example.

---

pc_mtd.9.09

< >

Multiple Time Domains : Multiple Time Domain Programming : Programming MTD Pin Levels

Programming MTD Pin Levels

Multiple time domain tests do not impose any additional requirements on pin level timing. Simply program all pins on the same Pin Levels sheet; you do not have to keep track of which pins belong to which time domains.

< >

pc_mtd.9.10

< >

Multiple Time Domains : Multiple Time Domain Programming : Creating MTD Patterns

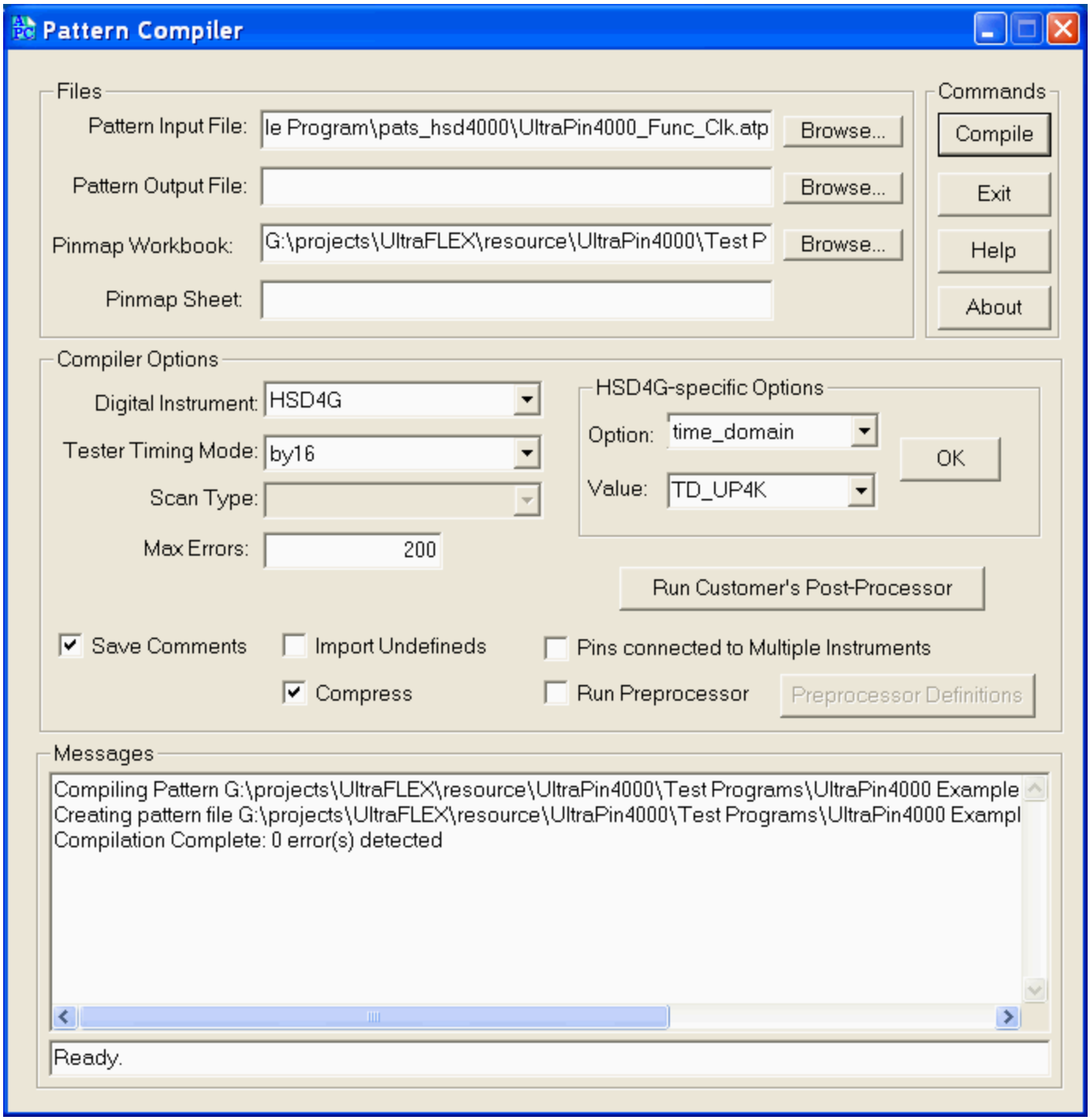
Creating MTD Patterns

For each multiple time domain functional test, every time domain must execute its own pattern or pattern set. A pattern or pattern set cannot be shared across domains.

To set up patterns for multiple time domain testing, first create each pattern as you normally would for the instrument used by each time domain.

You must then compile patterns so that they are associated with a time domain if you intend to use them in multiple time domain tests. You can either specify the time domain by using the time_domain keyword in the pattern header section, or you can specify the time domain name in the pattern compiler instead.

The figure below shows an example of the Pattern Compiler with the time_domain option enabled.



Pattern Compiler Using Time Domain Option

If you do not specify a time domain for a pattern, IG-XL attempts to assign a time domain by comparing the pins in use to the time domain definitions on the Pin Map sheet. If the pins in use do not match any defined time domain, then IG-XL defaults to the system time domain (the “All” time domain), which is a single time domain that represents all pins in the test system. Note that if the time domains used for timing and patterns do not match, then IG-XL returns an error.

IG-XL also checks patterns for time domain assignments at pattern load. However, specifying time domains during compile helps to highlight issues earlier.

If running multiple time domains synchronously, you must create patterns that maintain proper synchronization across the time domains. This includes:

- Executing the correct number of vectors in each time domain based on cycle periods

- Ensuring that time domains enter and exit SRM subroutines at the correct times

- Ensuring that loops and repeats are synchronized

How IG-XL Assigns Time Domains to Patterns

During pattern compile, IG-XL references the Pin Map sheet so that the compile algorithm knows the available time domains in a pattern. You can hard-code a time domain into the compiled pattern by specifying the time_domain parameter either in the .atp header or using the Pattern Compiler window. If IG-XL successfully compiles the time domain into the pattern, IG-XL only verifies that it is still valid during pattern load. (If the associated time domain pin group has changed, IG-XL reverifies that time domain assignment is valid. Otherwise, IG-XL just loads the pattern with the original time domain.)

If you do not specify any time domain for a pattern during compile, IG-XL automatically chooses a time domain. This occurs both during pattern compile and pattern load. If no time domain is specified during compile, IG-XL checks the pin map and finds the “best-fit” time domain pin group that matches pins and instruments in the pattern. If no matching time domain exists, or no time domains are defined, IG-XL chooses the default “All” domain.

During pattern load, the same behavior applies where IG-XL verifies that the existing time domain is still valid. If the existing time domain is invalid, IG-XL retries the automatic assignment algorithm.

In summary, IG-XL provides two ways of assigning time domains to patterns:

- User-specified: You specify a time domain when compiling the pattern, and this time domain remains valid during pattern load as long as nothing changes. In this case, IG-XL never assigns a time domain automatically. IG-XL returns an error if the time domain pin group has changed and no longer matches the pattern pins.

- Automatic assignment: IG-XL finds the best-fit time domain pin group and assigns it. This may occur during pattern compile, pattern load, or both, depending on pin map changes between compile and load.

Note that this behavior can result in this issue: Once a time domain is specified in a Pin Map sheet and no other action is taken, IG-XL may automatically assign a time domain to a pattern. As a result, you may have a run-time error where the pattern’s time domain does not match the time domain specified in the Time Sets or Time Sets (Basic) sheet.

When you use multiple time domains for operations such as in bursting multiple patterns at the same time, you must specify a time domain in each timing sheet that matches each associated pattern. Note, however, that this requirement can be inconvenient for tests whose patterns are burst serially (with a single time domain) and whose time domain is automatically assigned to those patterns.

The following table summarizes this behavior.

IG-XL Behavior in Assigning Time Domains to Patterns

Pin Map Condition	Pattern time_domain Parameter	Compiled Pattern	Pattern Load
No matching TDs defined	Not specified	No TD compiled to pattern.	Load tries to assign a TD, finds none specified, and so assigns the default “All” domain.
TD defined	Not specified	Algorithm finds best-fit TD pin group and compiles that into the pattern.	Load only verifies that the existing TD is still valid; if the pin map TD definition changed, it tries to reassign.
No matching TDs defined	TD specified	Compiler returns error trying to assign a TD that does not exist in the pin map.	N/A
TD defined	TD specified	Compiler verifies TD is in pin map and pins match, then compiles it into the pattern.	Load only verifies that the existing TD is still valid; if the pin map TD definition changed and is now invalid, IG-XL returns an error.

Note the following special case. You successfully compile a pattern with a specific time domain, but the test program you use to load that pattern has no time domains defined. In this case, the pattern load skips the check of time domains and always assigns the default time domain. This can happen if your test program uses a modified pin map or if you use another test program entirely. Since a compiled pattern can only contain a time domain if it exists in the referenced Pin Map

sheet, the only way this case can occur is if a pin map difference exists during pattern load into the test program.

< >

2015 Teradyne, Inc. All rights reserved.

pc_mtd.9.11

< >

Multiple Time Domains : Multiple Time Domain Programming : Using MTD Pattern Sets

Using MTD Pattern Sets

You can create patterns sets that support multiple time domains. A multiple time domain (MTD) pattern set combines one pattern or pattern set from each time domain to be executed together. You can create both burst and non-burst pattern sets, subject to the normal rules for bursting patterns together. An MTD pattern set does not need to contain the same number of patterns for each time domain, and the time domains do not need to end at the same time.

Note the following when defining MTD pattern sets:

- You must include the time domain name in the Time Domain field for each item in an MTD pattern set. If you leave the field blank, IG-XL returns a validation error.

You need not include an entry in the Time Domain field for items in a single time domain pattern set. If you leave the entire Time Domain column blank for items in the pattern set, then IG-XL programs that set as a single time domain.

- You must include exactly one pattern or pattern set from each time domain in an MTD pattern set. IG-XL returns a validation error if you include multiple items from the same time domain.

- You must specify the correct time domain for items in the MTD pattern set. If you specify an incorrect time domain for the pattern or pattern set (for instance, if you specify time domain TD2 for a patfile.pat pattern that is compiled using time domain TD1), then IG-XL returns a validation error.

- MTD pattern sets can only contain patterns or pattern sets that have single time domains. If you specify an MTD pattern or pattern set as an item of another MTD pattern set, then IG-XL

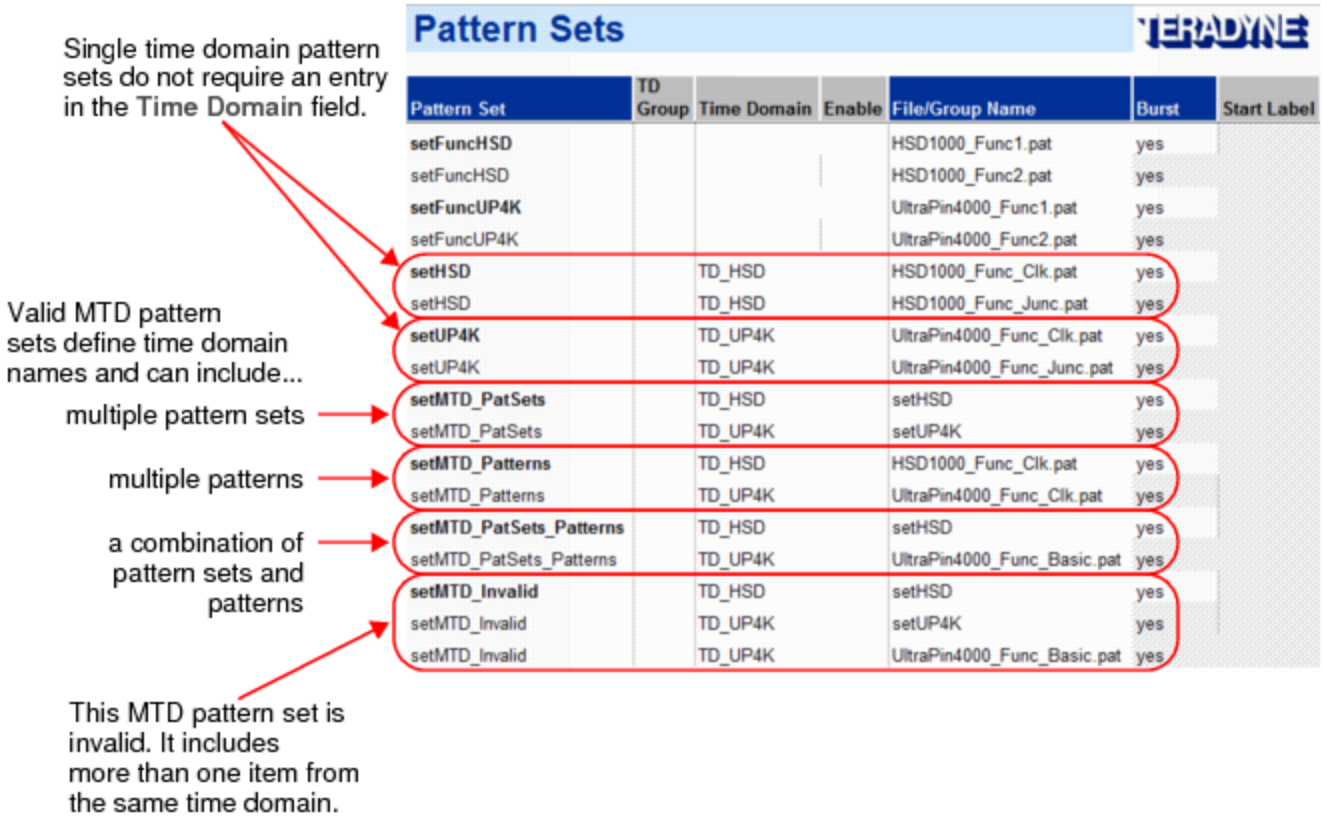
returns a validation error.

- MTD pattern sets cannot contain keepalive patterns. IG-XL returns a validation error if you include keepalive patterns.

☑ Note:

The TD Group column on this sheet is not supported in this release.

The following sample Pattern Sets sheet highlights the restrictions described above.



Pattern Set	TD Group	Time Domain	Enable	File/Group Name	Burst	Start Label
setFuncHSD				HSD1000_Func1.pat	yes	
setFuncHSD				HSD1000_Func2.pat	yes	
setFuncUP4K				UltraPin4000_Func1.pat	yes	
setFuncUP4K				UltraPin4000_Func2.pat	yes	
setHSD		TD_HSD		HSD1000_Func_Clk.pat	yes	
setHSD		TD_HSD		HSD1000_Func_Junc.pat	yes	
setUP4K		TD_UP4K		UltraPin4000_Func_Clk.pat	yes	
setUP4K		TD_UP4K		UltraPin4000_Func_Junc.pat	yes	
setMTD_PatSets		TD_HSD		setHSD	yes	
setMTD_PatSets		TD_UP4K		setUP4K	yes	
setMTD_Patterns		TD_HSD		HSD1000_Func_Clk.pat	yes	
setMTD_Patterns		TD_UP4K		UltraPin4000_Func_Clk.pat	yes	
setMTD_PatSets_Patterns		TD_HSD		setHSD	yes	
setMTD_PatSets_Patterns		TD_UP4K		UltraPin4000_Func_Basic.pat	yes	
setMTD_Invalid		TD_HSD		setHSD	yes	
setMTD_Invalid		TD_UP4K		setUP4K	yes	
setMTD_Invalid		TD_UP4K		UltraPin4000_Func_Basic.pat	yes	

Pattern Sets Sheet Setup for Multiple Time Domain Testing

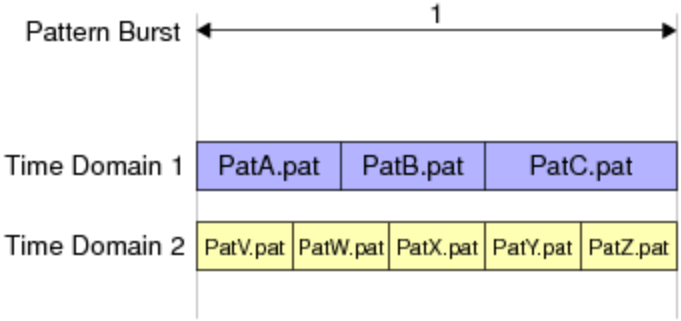
Executing and Datalogging Pattern Sets in Burst Mode

To execute MTD pattern sets in a single burst, set the Burst column to yes. When this burst mode is enabled, all pattern modules in the set are burst together. IG-XL executes pattern sets as follows:

1. Execute pattern sets concurrently.
2. Wait for all time domain PatGens to complete.
3. Execute functional test and datalog the results.

Note that IG-XL performs these steps only if the pattern set is executed using TheHdw.Patterns.Test. If you execute the pattern set using TheHdw.Patterns.Start, then IG-XL only performs step 1.

In the figure below, PatSet1 is on time domain 1 and includes pattern files PatA.pat, PatB.pat, and PatC.pat. PatSet2 is on time domain 2 and includes pattern files PatV.pat, PatW.pat, PatX.pat, PatY.pat, and PatZ.pat. This figure shows that you can reduce test time by bursting MTD pattern sets concurrently.



Pattern Set Execution in Burst Mode

For MTD pattern sets using burst mode, IG-XL returns test results based on the result mode you select. These options are available using the ResultMode parameter of the TheHdw.Patterns.Test VBT node. The options are:

tlResultModeOne	Return one pass/fail result for all time domains and pattern modules. If any module on any time domain fails, IG-XL returns a failing result.
tlResultModeDomain	Return a unique pass/fail result for each time domain. (Replaces tlResultModeGanged used in earlier releases.) IG-XL uses a separate test number for each time domain. Default.
tlResultModeModule	Return a unique pass/fail result for each module and time domain. (Replaces tlResultModeSequential used in earlier releases.) IG-XL uses a separate test number for each module in each time domain.

Note that you can also set these result modes using the ResultMode parameter in the Functional_T VBT test instance.

Note that the HRAM reburst feature is not supported for MTD pattern sets.

Below is a sample datalog for the example shown above using the tlResultModeModule option, where each pattern file has one module.

Number	Site	Test Name	Pattern	1st Failed Cycle	Total Failed Cycles
100	0	FuncTest	A	N/A	N/A
101	0	FuncTest	B	N/A	N/A
102	0	FuncTest	C	N/A	N/A
103	0	FuncTest	V	N/A	N/A
104	0	FuncTest	W	N/A	N/A
105	0	FuncTest	X	N/A	N/A

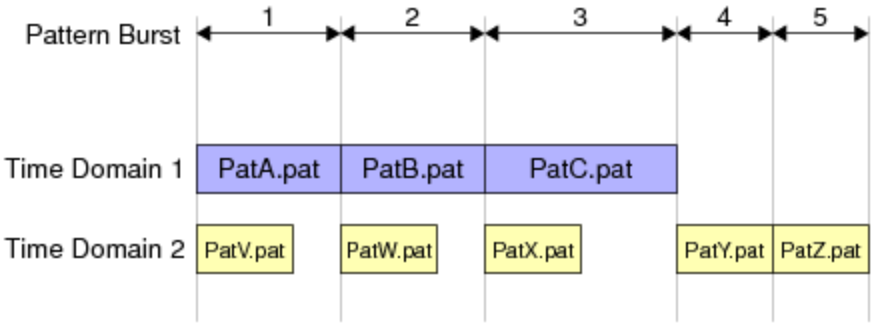
106	0	FuncTest	Y	N/A	N/A
107	0	FuncTest	Z	N/A	N/A

Executing and Datalogging Pattern Sets in Non Burst Mode

To execute MTD pattern sets in multiple bursts, set the Burst column to no. When this burst mode is enabled, each pattern module in the set is executed individually. IG-XL executes the pattern sets as follows:

1.	Execute first modules in nonburst mode pattern sets concurrently.
2.	Wait for all time domain PatGens to complete.
3.	Execute functional test and datalog for each module that was executed.
4.	Repeat steps 1-3 with next modules until all modules are executed.
5.	If the number of modules in these nonburst pattern sets is not equal, then execute the extra modules individually in separate bursts from step 6.
6.	Execute next module in nonburst mode pattern sets (extra modules).
7.	Wait for PatGen to complete.
8.	Execute functional test and datalog for the module that was executed.
9.	Repeat steps 6-8 until all extra modules are executed.

In the figure below, PatSet1 is on time domain 1 and includes pattern files PatA.pat, PatB.pat, and PatC.pat. PatSet2 is on time domain 2 and includes pattern files PatV.pat, PatW.pat, PatX.pat, PatY.pat, and PatZ.pat. This figure shows that running in nonburst mode affects test time due to how IG-XL executes MTD pattern files.



Pattern Set Execution in Non Burst Mode

For nonburst pattern sets, IG-XL returns a separate test result for each module regardless of the datalog setup's result mode setting. The following is a sample datalog for the example shown above, where each pattern file has one module.

Number	Site	Test Name	Pattern	1st Failed Cycle	Total Failed Cycles
100	0	FuncTest	A	N/A	N/A
101	0	FuncTest	V	N/A	N/A
102	0	FuncTest	B	N/A	N/A
103	0	FuncTest	W	N/A	N/A
104	0	FuncTest	C	N/A	N/A
105	0	FuncTest	X	N/A	N/A
106	0	FuncTest	Y	N/A	N/A
107	0	FuncTest	Z	N/A	N/A

< >

2015 Teradyne, Inc. All rights reserved.

pc_mtd.9.12

< >

Multiple Time Domains : Multiple Time Domain Programming : Programming MTD Test Instances

Programming MTD Test Instances

Create multiple time domain tests on a Test Instances sheet by assembling the elements needed for a functional test. When using the Functional_T VBT test template, you must program two fields differently for multiple time domain operation:

Time Sets	In this field, enter the name of a Time Set sheet for each time domain separated by a comma. For example, if you are testing one HSD1000 time domain together with one UltraPin4000 time domain, you could enter TS_HSD1000, TS_UP4000.
Patterns	In this column, enter the name of an MTD pattern set. Alternatively, you can also enter a pattern or pattern set for each time domain separated by a plus (+) sign. For example, if you are using pattern sets to test one HSD1000 time domain and one UltraPin4000 time domain, you could enter Patset_HSD1000 + Patset_UP4000.

< >

pc_mtd.9.13

< >

Multiple Time Domains : Multiple Time Domain Programming : Executing VBT Code on Time Domains

Executing VBT Code on Time Domains

In VBT, you can execute code on specific time domains. The TimeDomains node is an optional node for digital instruments. The following properties and interface nodes are available under TheHdw.Digital.TimeDomains:

- Alarms
- CMEM
- Enable
- FailedPins
- HRAM
- Patgen
- ShareTimesetNamespace
- Timing
- Tracker

Note that these nodes are also available under the standard TheHdw.Digital node (without going through TimeDomains). Thus, the TimeDomains node is a way to perform normal VBT operations on specific time domains. For example, to see if the last pattern burst on a time domain named "TD_HSD" passed, you can execute the following code:

```
Dim TestResult As Boolean
TestResult = TheHdw.Digital.TimeDomains("TD_HSD").Patgen.PatternBurstPassed
```

If you do not use the TimeDomains node with the properties and methods of the interface nodes listed above, then IG-XL uses the System time domain, which is a single time domain that is comprised of all pins defined in the test program. This usage can lead to unexpected results with test programs that use multiple time domains.

If your test program uses multiple time domains, you should always use the TimeDomains node and explicitly specify the time domains to program when using the properties and methods of the interface nodes listed above. This programming practice ensures that IG-XL is reading and writing correct information for the time domains you intend to program.

For more information, see TheHdw.Digital.TimeDomains in the VBT Syntax Reference.

---

pc_mtd.9.14

< >

Multiple Time Domains : Multiple Time Domain Programming : Executing VBT Code on MTD Patterns

Executing VBT Code on MTD Patterns

You can also use VBT code to simultaneously load and execute multiple patterns on different time domains. Use a plus (+) sign to construct multiple time domain patterns. For example, the following VBT functions load and execute two different patterns:

```
TheHdw.Patterns("pat1.PAT+pat2.PAT").Load
TheHdw.Patterns("pat1.PAT+pat2.PAT").Test
```

< >

pc_mtd.9.15

< >

Multiple Time Domains : Multiple Time Domain Programming : MTD Flag Handshaking

MTD Flag Handshaking

You can perform CPU flag handshaking in multiple time domains. To set up CPU flags, you must add CPU flag loops in pattern files for all time domains and write a VBT function that executes the flag code.

When using CPU flags with multiple time domains, note the following:

- Each time domain can enter CPU flag loops independently. You can execute a CPU flag loop on one time domain without executing it on another.
- You must use VBT to enable the flag match state independently for each time domain using TheHdw.Digital.TimeDomains.Patgen.SetFlagMatch.
- You can reset CPU flags independently for each time domain or for all time domains together using TheHdw.Digital.Patgen.Continue. Note that you must use TheHdw.Digital.TimeDomains(domainName).Patgen.Continue node to change CPU flags for a specific time domain.
- The Functional_T VBT test template supports CPU flags with multiple time domains allowing you to set the flag wait states by time domain.
- If you want time domains to remain synchronized when entering and exiting CPU flag loops, you must ensure that the patterns in each time domain remain synchronized in the loops and that you clear all time domain flags at the same time.

IG-XL automatically programs flag delays while executing TheHdw.Digital.ApplyLevelsTiming. IG-XL only programs flags delays for contexts where multiple time domains are in use.

Note the following restrictions when using MTD flag loops:

- The MTD context can only contain two time domains.
- The major vector period MOSC count must be divisible by 4 for each time domain.
- Only one unique period value is allowed for each time domain.
- MOSC must be programmed the same for each time domain.
- Timing for each time domain must be programmed on a separate timing sheet.
- Master Timeset field must be programmed on each HSD1000 or UltraPin800 timing sheet involved. It can be a local timeset or a master (name of the other timeset sheet).

IG-XL returns an error at run time if the ratio of periods cannot be supported and you either specify a flag match interpose function in a functional test instance or use the flag handshaking VBT methods TheHdw.Digital.Patgen.Continue or TheHdw.Digital.TimeDomains.Patgen.Continue. These functions return errors because they change CPU flag states for multiple time domains. If the flag delays are not set properly in hardware, the time domains may not have their flags changed at the same time and could cause execution of the patterns to get out of sync. If you do not need to execute patterns synchronously, then you can ignore the error condition, and IG-XL will set the flag delays to 0.

The example below shows how IG-XL handles flag delay programming when using multiple time domains with different periods in CPU loops.

Flag Delay Period Ratio Example Calculation

An example test program has two time domains:

- TD_A is programmed at 100 Mbps in dual mode on HSD1000.
- TD_B is programmed at 200 Mbps in By16 mode on UltraPin4000.

This example requires a MOSC of 625 ps to attain the period for both time domains. These periods and modes result in a major period ratio of 32:128. To check the periods that define this ratio, use

this formula:

Time Domain Period = ((1/Data Rate) * PatGen Mode)/MOSC

In this case, the periods are calculated as follows:

TD_A Period = ((1/100Mbps)*2)/625ps = 32

TD_B Period = ((1/200Mbps)*16)/625ps = 128

IG-XL finds the best fit MOSC ratio and programs the flag delay correctly for both time domains. If you have more than two time domains, then you must use TheHdw.Digital.Patgen.FlagDelay to set the flag delay register for the additional time domains. This VBT property programs the PatGen's flag delay register so that the change in flag state occurs simultaneously across all time domains.

---

pc_mtd.9.16

< >

Multiple Time Domains : Multiple Time Domain Programming : Datalogging MTD Tests

Datalogging MTD Tests

IG-XL provides several options for datalogging the results of a multiple time domain test. These options are available through the ResultMode parameter of the TheHdw.Patterns.Test VBT node. The options are:

tlResultModeDomain	Report a unique pass/fail result for each time domain. Default. (Replaces tlResultModeGanged used in earlier releases.) IG-XL uses a separate test number for each time domain.
tlResultModeModule	Report a unique pass/fail result for each module and time domain. (Replaces tlResultModeSequential used in earlier releases.) IG-XL uses a separate test number for each module in each time domain.
tlResultModeOne	Report one pass/fail result for all time domains and pattern modules. If any module on any time domain fails, IG-XL returns a failing result.

Note that you can also set these result modes using the ResultMode parameter in the Functional_T VBT test instance.

Optionally, you can create your own datalog output by reading the pass/fail results of the previously executed pattern burst. You can read the results per time domain using the TheHdw.Digital.TimeDomains.Patgen.PatternBurstPassed and then write the result to the datalog using the TheExec.Datalog.WriteFunctionalResult VBT node.

The datalog report below shows the three result modes for the multiple time domain loopback test example (see HSD1000 and UltraPin4000 MTD Loopback Example).

Datalog report

10/22/2009 17:36:09

Prog Name: UltraPin4000_Demo_Prog Rev 0.6.1.xls
Job Name: UltraPin4000_Demo_Prog Rev 0.6.1.xls
Lot: Adventure
Operator: Ulysses
Test Mode: BowBend
Node Name: TELEMACHUS
Part Type: Hero
Channel map: Chans
Environment: WineDarkSea

Site Number:
0

Device#: 10

Number	Site	Test Name	Pattern	1st Failed Cycle	Total Failed Cycles
<MTD_PatSet_One>					
0	0	MTD_PatSet_One	hsd_mtd+up4k_mtd	N/A	N/A
<MTD_PatSet_Domain>					
1	0	MTD_PatSet_Domain	hsd_mtd	N/A	N/A
2	0	MTD_PatSet_Domain	up4k_mtd	N/A	N/A
<MTD_PatSet_Module>					
3	0	MTD_PatSet_Module	HSD_Func1	N/A	N/A
4	0	MTD_PatSet_Module	HSD_Func2	N/A	N/A
5	0	MTD_PatSet_Module	HSD_Func3_mod1	N/A	N/A
6	0	MTD_PatSet_Module	HSD_Func3_mod2	N/A	N/A
7	0	MTD_PatSet_Module	UP4K_Func1	N/A	N/A
8	0	MTD_PatSet_Module	UP4K_Func2	N/A	N/A
9	0	MTD_PatSet_Module	UP4K_Func3_mod1	N/A	N/A
10	0	MTD_PatSet_Module	UP4K_Func3_mod2	N/A	N/A

Site Failed tests/Executed tests

Site	Failed tests	Executed tests
0	0	11

Site	Sort	Bin
0	N/A	N/A

<	>

pc_mtd.9.17

< >

Multiple Time Domains : Multiple Time Domain HRAM Capture

Multiple Time Domain HRAM Capture

IG-XL captures vector data for all time domains individually in HRAM. You can use PatternTool and HRAM PatternTool to look at HRAM contents independently for each time domain. Use these methods for setting up HRAM capture for multiple time domains:

- HRAM Setup Panel
- Digital Time Domains HRAM VBT Syntax
- DataCollect Setup Panel

HRAM Setup Panel

You can apply different HRAM setups to each time domain using the HRAM Setup panel in either HRAM Display or Waveform Display. To set up HRAM independently for each time domain, follow these steps:

1. Select the instrument from the Instrument pull-down menu (HSD4G for UltraPin4000, HSDM for HSD1000 and UltraPin800, and HSDMQ for UltraPin1600).
2. Select the time domain name from the Time Domain pull-down menu.

The time domain names listed in this menu are the ones defined on the Pin Map sheet.

3. Create the desired HRAM setup and apply it to the selected time domain.

4.	Repeat these steps to apply different HRAM setups for other time domains in your test program.

☑ **Note:**

If the Instrument field selection conflicts with the Time Domain field selection, then the entry in the Time Domain field takes precedence.

For more information, see HRAM Setup.

Digital Time Domains HRAM VBT Syntax

You can set up HRAM independently for each time domain using this VBT node:

TheHdw.Digital.TimeDomains(domainName).HRAM

Use this node to set the trigger for capture and also to read back the HRAM setup and capture information. If you do not use this node to set up HRAM, IG-XL writes the setup to all time domains.

For more information, see TimeDomains.HRAM in the VBT Syntax Reference.

DataCollect Setup Panel

You can apply a basic datalog setup for HRAM capture by creating a customized datalog setup file in the DataCollect Setup File dialog box. This HRAM Setup dialog box lets you either apply the HRAM setup to all time domains or create a separate setup for each time domain. If you use the HRAM Setup dialog box to create different HRAM setups for different time domains, the HRAM controls on the Setup File dialog box are disabled.

To create and apply independent HRAM setups for several different time domains:

1.	Click the Full Edit button in the HRAM setup section of the Setup File dialog box.

This opens the HRAM Setup dialog box, which allows you to program separate HRAM setups for each of the time domains in your test program, which are available from the drop-down list.

2.	Select the time domain name from the Time Domain menu and create the desired HRAM setup using the controls in this dialog box.

For more information on how to use this panel to create and apply an HRAM setup, see HRAM Setup in DataCollect Setup.

---

pc_mtd.9.18

< >

Multiple Time Domains : Multiple Time Domain Programming Examples

Multiple Time Domain Programming Examples

The following examples are included:

- Fractional Bus Sheet MTD Example for Synchronous Pattern Start
- HSD1000 and UltraPin4000 MTD Loopback Example
- UltraWave Duplex MTD Example

< >

pc_mtd.9.19

< >

Multiple Time Domains : Multiple Time Domain Programming Examples : Fractional Bus Sheet MTD Example for Synchronous Pattern Start

Fractional Bus Sheet MTD Example for Synchronous Pattern Start

Follow this procedure when using the Fractional Bus sheet to establish frequency relationships between time domains for synchronous pattern start:

1. Set the full MOSC range from 1.0GHz to 2.0GHz.

To allow the greatest flexibility around MOSC selection, you must include the following VBT statement in the OnProgramLoaded interpose function.
TheHdw.Digital.Timing.MOSCFullRange = True

2. Define time domains in the Pin Map sheet.

The figure below shows a simple pin map with two defined time domains.

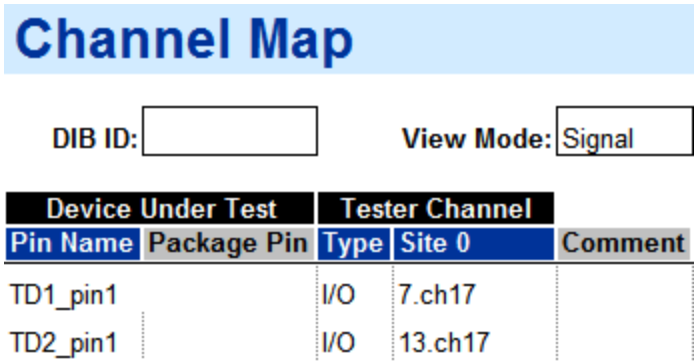
Pin Map

Group Name	Pin Name	Type
	TD1_pin1	I/O
	TD2_pin1	I/O
All_pins	TD1_pin1	I/O
All_pins	TD2_pin1	
TD1	TD1_pin1	TimeDomain
TD2	TD2_pin1	TimeDomain

Pin Map Sheet for Fractional Bus Example

3. Assign channels to unique hardware time domains in the Channel Map sheet.

The figure below shows the channel map for the fractional bus example.



Channel Map

DIB ID: View Mode: Signal

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
TD1_pin1		I/O	7.ch17	
TD2_pin1		I/O	13.ch17	

Channel Map Sheet for Fractional Bus Example

4.	Define the frequency relationship between time domains in the Fractional Bus sheet.

This sheet uses the following columns:

Fractional Bus Set	Enter the fractional bus set name to describe the group of frequencies.
Symbol	Enter the symbol name that represents the fractional timing for the fractional bus specification. You can use this name in a Time Sets or Time Sets (Basic) sheet just like an AC, DC or Global spec.
Time Domain	Select a valid time domain name from the Pin Map sheet.
Value	This column contains the actual frequency calculated by the fractional bus algorithm for the time domain. IG-XL calculates this value.
Reference	Enter the reference frequency for the fractional bus set. You can use any value as a reference.
Numerator	Enter the numerator of the ratio to be used with reference frequency to calculate the actual fractional bus frequency value.
Denominator	Enter the denominator of the ratio to be used with reference frequency to calculate the actual fractional bus frequency value.

This example programs one time domain to run at 200MHz and the other time domain to run at 533.333MHz. The two domains must maintain an exact fractional relationship to communicate properly with the device. In this case, that exact fractional relationship is 8:3.
To achieve the 533.333MHz data rate, this example uses Dual 2X mode, which means the T0 frequency for that domain is half that rate, or 266.666MHz. This is critical as this is the important frequency that the MOSC value must be selected to achieve.
The figure below shows this frequency relationship defined in a Fractional Bus sheet.

Fractional Bus TERADYNE

Fractional Bus Set	Symbol	Time Domain	Value	Reference	Numerator	Denominator
FracBusSet1	TD1_freq	TD1	200.E+06	100.E+06	2	1
FracBusSet1	TD2_freq	TD2	266.667E+06		8	3

Fractional Bus Sheet Defining Frequency Relationships for Two Time Domains

You can enter the Reference, Numerator and Denominator values directly or define them using AC specs. IG-XL uses these values to calculate a frequency value for the time domain that the digital hardware can achieve exactly and that exactly matches the frequency relationship defined by this sheet. IG-XL assigns the resulting values to the Fractional Bus "spec" name defined in the Symbol column.

5.	Program separate Time Sets (Basic) sheets for each time domain.

Use the Fractional Bus sheet's symbol names in the same way as an AC spec to define the T0 period and edge timing.

The figure below shows the Time Sets (Basic) sheet for the first time domain, which uses the Single timing mode and the TD1_freq symbol name as a spec value to program both the Cycle Period and Drive Return columns.

Time Sets (Basic) TERADYNE

Timing Mode: Single Master Timeset Name:
Time Domain: TD1 Strobe Ref Setup Name:

	Cycle	Pin/Group			Data		Drive				Compare				Edge Resolution
Time Set	Period	Name	Clock Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Ref Offset	Mode
TD1_timeset	=1/_TD1_freq	TD1_pin1			PAT	RL		0	=0.5/_TD1_freq		Off				Machine

Time Sets (Basic) Sheet Time Domain 1 for Fractional Bus Example

The figure below shows the Time Sets (Basic) sheet for the second time domain, which uses the Dual timing mode, the 2X pin setup, and the TD2_freq symbol name as a spec value to program both the Cycle Period, Drive Return First, Drive Second, and Drive Return Second columns.

Time Sets (Basic) TERADYNE

Timing Mode: Dual imeset Name:
Time Domain: TD2 Setup Name:

	Cycle	Pin/Group			Data		Drive				Compare				Edge Resolution
Time Set	Period	Name	Clock Period	Setup	Src	Fmt	First	Return First	Second	Return Second	Mode	First	Second	Ref Offset	Mode
TD2_timeset	=1/_TD2_freq	TD2_pin1		2X	PAT	RL-2X	0	=0.25/_TD2_freq	=0.5/_TD2_freq	=0.75/_TD2_freq	Off				Machine

Time Sets (Basic) Sheet Time Domain 2 for Fractional Bus Example

☑ **Note:**

The equation used to define the T0 period must take the exact form shown above with the only change being the actual Fractional Bus symbol name (that is, =1/_<FracBusSymbolName>). Any variation to the form of this equation may result in improper operation.

6.	Reference both timeset sheets in the Time Sets column of the test instance definition.

Test Instances TERADYNE

	Test Procedure			DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
FractionalBus	VBT	Burst				Case1	Sel0	TSB_TD1,TSB_TD2		Levels		

Fractional Bus Test Instances Sheet

7.	Use a functional test template or VBT code to download timing and start patterns.

The VBT module below shows how to do synchronous pattern start for two different patterns using these two time domains.

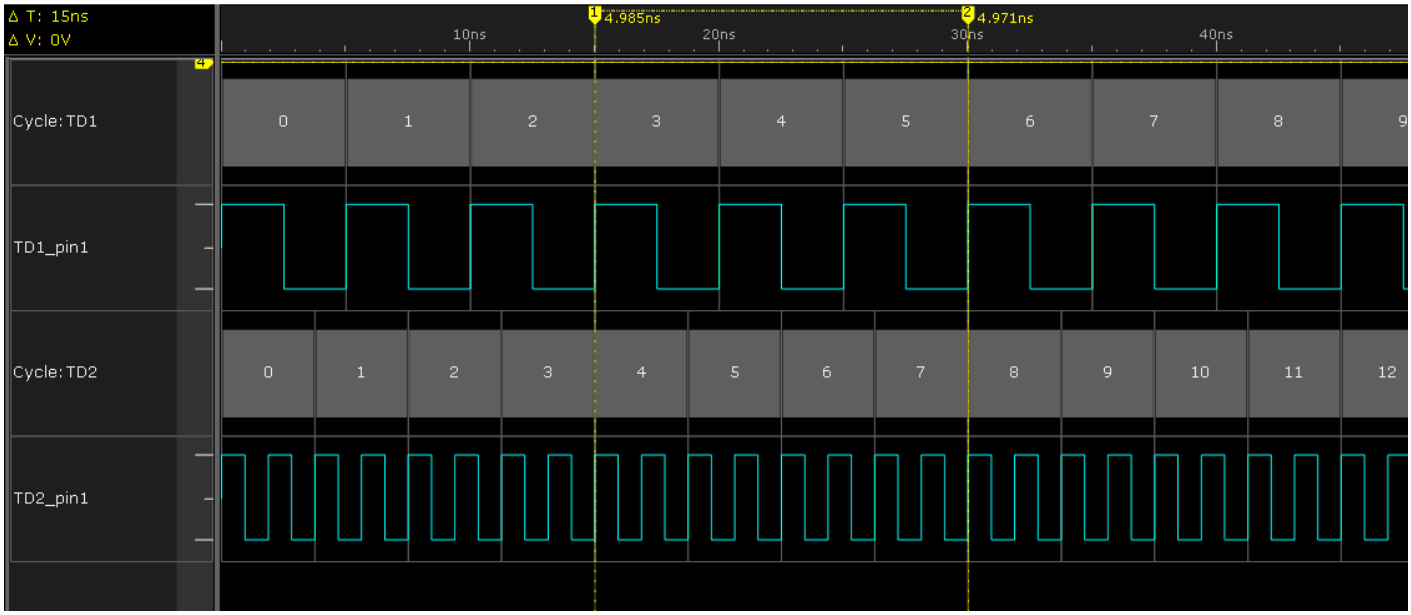
```
Function Burst(TD1_PatternName As Pattern, TD2_PatternName As Pattern) As Long
    On Error GoTo errHandler

    TheHdw.Digital.ApplyLevelsTiming True, True, True

    TheHdw.Patterns(TD1_PatternName & "+" & TD2_PatternName).Start
    TheHdw.Digital.Patgen.HaltWait

    Exit Function
errHandler:
    If AbortTest Then Exit Function Else Resume Next
End Function
```

When the patterns are started, you can confirm that the ratios between the time domains are exact using Waveform Display. In the figure below, time domain 1 is equal to three DUT cycles, and time domain 2 is equal to eight DUT cycles.



Waveform Display Showing Exact Ratios Between Time Domains

< >

2015 Teradyne, Inc. All rights reserved.

pc_mtd.9.20

< >

Multiple Time Domains : Multiple Time Domain Programming Examples : HSD1000 and UltraPin4000 MTD Loopback Example

HSD1000 and UltraPin4000 MTD Loopback Example

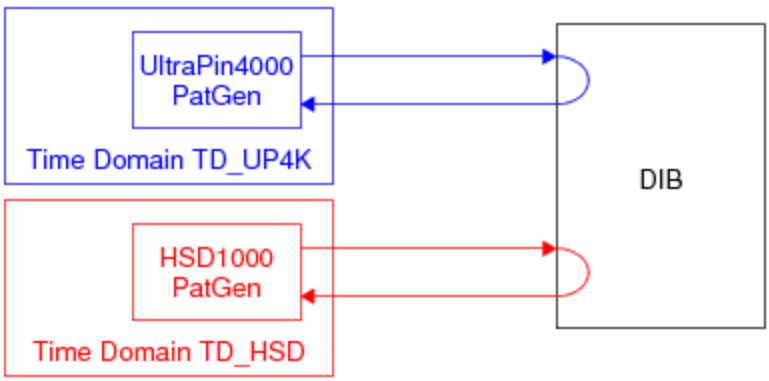
The following example shows how to program two time domains to conduct loopback testing using HSD1000 and UltraPin4000.

☑ **Note:**

This example uses a master timeset name on a Time Sets (Basic) sheet so that all timeset periods are derived from the UltraPin4000 MOSC. Note that this approach is valid; but in IG-XL V8.0 and later, you should use the Fractional Bus sheet to set up the frequency relationships between time domains for synchronous pattern start. See Fractional Bus Sheet MTD Example for Synchronous Pattern Start.

The figure shows the basic setup for this loopback test, where:

- Time Domain TD_UP4K is the UltraPin4000.
- Time Domain TD_HSD is the HSD1000.



Multiple Time Domain Setup for Loopback Testing

Follow these steps to set up your test program.

1.	Set the full MOSC range from 1.0GHz to 2.0GHz.

To allow the greatest flexibility around MOSC selection, you must include the following VBT statement in the OnProgramLoaded interpose function.

TheHdw.Digital.Timing.MOSCFullRange = True

DataTool Sheet Programming

2.	Program a pin map with both HSD1000 and UltraPin4000 I/O pins and group the HSD1000 pins into time domain TD_HSD and the UltraPin4000 pins into time domain TD_UP4K. Use the TimeDomain keyword in the Type column to define time domain pin groups.

Pin Map

Group Name	Pin Name	Type
	dq10	I/O
	dq11	I/O
	dq12	I/O
	dq13	I/O
	dq14	I/O
	dq15	I/O
	hsd_ref1	I/O
	hsd_ref2	I/O
	hsd_ref3	I/O
	hsd_ref4	I/O
MCK0	mck0_p	Differential
MCK1	mck1_p	Differential
DQS0	dqs0_p	Differential
DQS1	dqs1_p	Differential
ALL_DIFF_IO	MCK0	Differential
DQ0_7	dq0	I/O
DQ8_15	dq8	I/O
DQ_ALL	DQ0_7	I/O
HSD_ALL	hsd_ref1	I/O
TD_UP4K	MCK0	TimeDomain
TD_HSD	HSD_ALL	TimeDomain

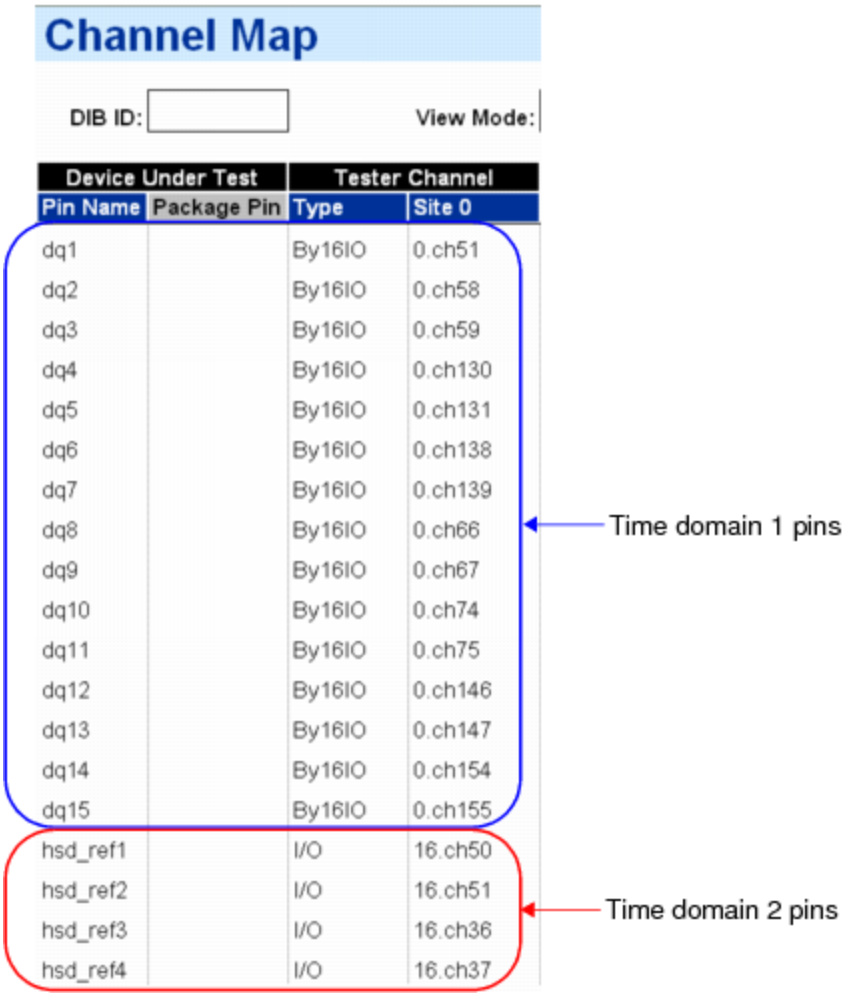
UltraPin4000 time domain pin group

HSD1000 time domain pin group

Multiple Time Domain Pin Map Setup

3.	Program a channel map to include pins for both time domains.

You can program only one time domain per digital instrument. In this example, time domain TD_UP4K uses slot 0, and time domain TD_HSD uses slot 16. You can also create a multisite channel map. IG-XL reports test results for each site and each time domain individually. Note that you can control how time domain results are datalogged (see Datalogging MTD Tests).



Multiple Time Domain Channel Map Setup

4.	Program the timing by setting up two Time Sets (Basic) sheets, one for the HSD1000 and another for the UltraPin4000 time domains.

This is where you set up two Time Sets sheets with two different cycle periods for the two time domains.

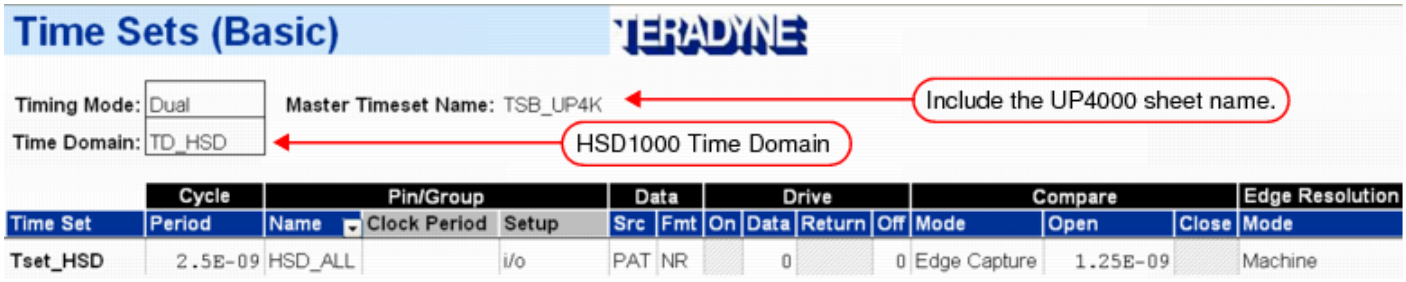
Note that you must include the Time Sets (Basic) sheet name for the UltraPin4000 time set in the HSD1000 Time Sets (Basic) sheet in the Master Timeset Name field.

☑ **Note:**

With the introduction of the Fractional Bus sheet in V8.0, you can establish frequency relationships between time domains for synchronous pattern start without having to program a master timeset name. See Fractional Bus Sheet MTD Example for Synchronous Pattern Start.

When you program multiple time domains using only HSD1000, UltraPin800, or UltraPin1600 instruments, you have the option to use one Time Sets sheet to program these time domains. In this case, all time domain periods are derived from one MOSC mathematically.

Use this method only if all time sets on the sheet are integer multiples of the same MOSC period. This integer multiple relationship must also hold during any period characterization that is performed on these time domains.



	Cycle	Pin/Group			Data		Drive				Compare			Edge Resolution
Time Set	Period	Name	Clock Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Mode
Tset_HSD	2.5E-09	HSD_ALL		i/o	PAT	NR		0		0	Edge Capture	1.25E-09		Machine

	Cycle	Pin/Group			Data		Drive				Compare			Edge Resolution
Time Set	Period	Name	Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Mode
Tset_up4k	250.E-12	DQS0		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	DQS1		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	MCK0		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	MCK1		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq0		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq1		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq2		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq3		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq4		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq5		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq6		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq7		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq8		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq9		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq10		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq11		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq12		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq13		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq14		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine
Tset_up4k	250.E-12	dq15		i/o	PAT	NR		0		0	Edge	187.5E-12		Machine

Multiple Time Domain Time Sets Sheet Setup

Pattern and Test Instance Programming

5.	Create a pattern file for each time domain used by the test instance. See the UltraPin4000_Func_Clk.atp and HSD1000_Func.atp pattern files below.

```
// UltraPin4000_Func_Clk.atp file. This file has been truncated.
opcode_mode = By16;
import tset Tset_up4k;
srm_vector UltraPin4000_Func_module ($tset,          DQS0     DQS8 )
{
                    > Tset_up4k      .0      .L;
                    > Tset_up4k      .1      .H;
                    > Tset_up4k      .0      .L;
                    > Tset_up4k      .1      .H;
                    > Tset_up4k      .1      .H;
                    > Tset_up4k      .0      .L;
```

```
                        > Tset_up4k      .1      .H;
                        > Tset_up4k      .1      .H;
                        > Tset_up4k      .1      .H;
halt                     > Tset_up4k      .0      .L;
}
// HSD1000_Func.atp file. This pattern has been truncated.
opcode_mode = dual;
import tset Tset_HSD;

srm_vector HSD_Loopback ( $tset,          TD2_drv   TD2_rcv )
{
                        > Tset_HSD       .0      .L;
                        > Tset_HSD       .1      .H;
                        > Tset_HSD       .0      .L;
                        > Tset_HSD       .1      .H;
                        > Tset_HSD       .1      .H;
                        > Tset_HSD       .0      .L;
                        > Tset_HSD       .1      .H;
                        > Tset_HSD       .1      .H;
                        > Tset_HSD       .1      .H;
halt                     > Tset_HSD       .0      .L;
}
PatternTool shows all time domain patterns at the same time.
```

HSD1000_Func.PAT (HSD1000_Func_module - VM)

TSet	Command	Label	Vector	sd_ref1	sd_ref2	sd_ref3	sd_ref4	Comment
Tset_HSD			0	0	0	0	0	
Tset_HSD			1	0	0	0	0	
Tset_HSD			2	0	0	0	0	
Tset_HSD			3	0	0	0	0	
Tset_HSD			4	1	1	1	1	
Tset_HSD			5	1	1	1	1	
Tset_HSD			6	0	0	0	0	
Tset_HSD			7	0	0	0	0	
Tset_HSD			8	0	0	0	0	
Tset_HSD			9	0	0	0	0	
Tset_HSD			10	1	1	1	1	
Tset_HSD			11	1	1	1	1	
Tset_HSD			12	L	L	L	L	
Tset_HSD			13	L	L	L	L	
Tset_HSD			14	L	L	L	L	
Tset_HSD			15	L	L	L	L	
Tset_HSD			16	H	H	H	H	

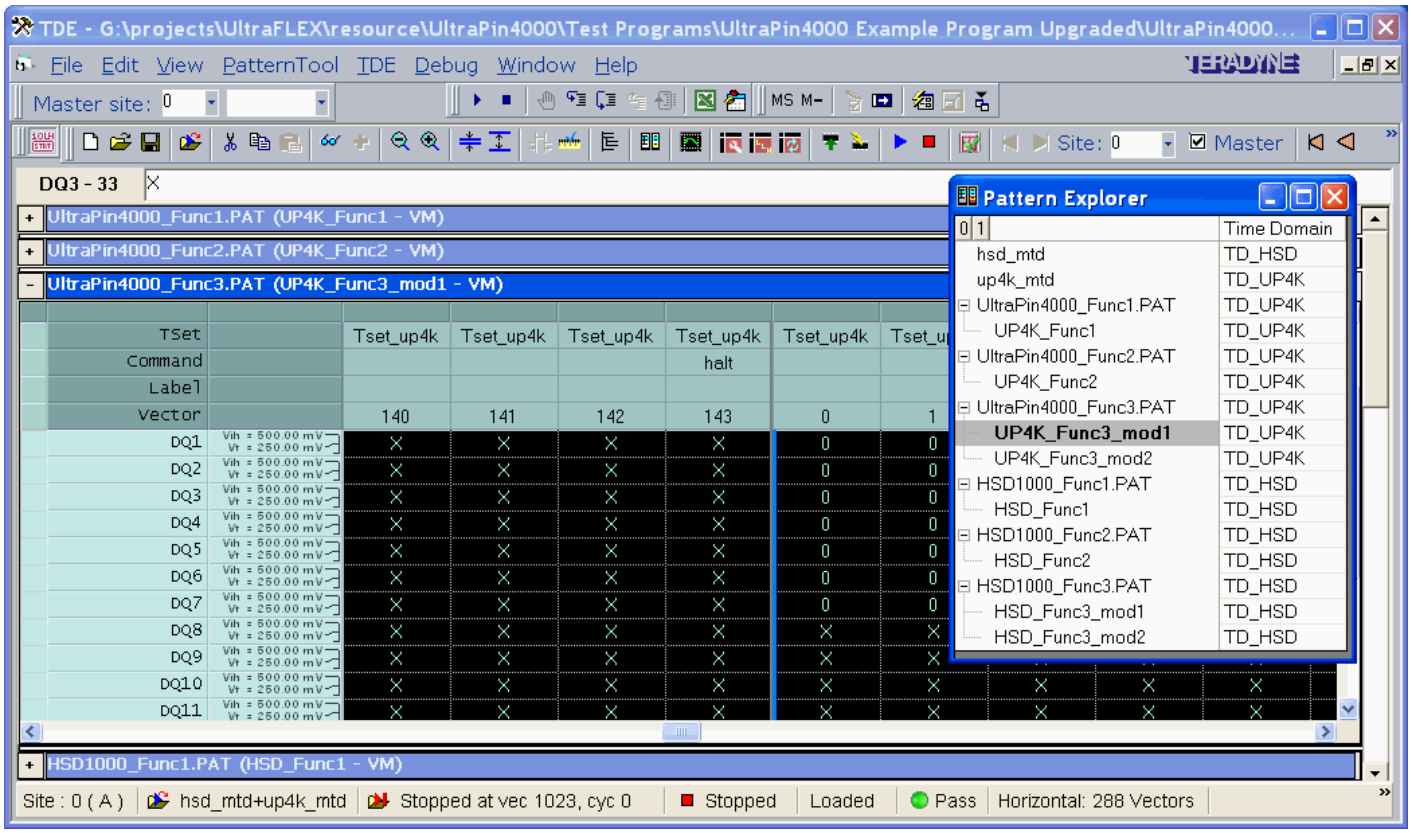
Multiple Time Domain HSD1000 Loopback Pattern in PatternTool

UltraPin4000_Func_Clk.PAT (UltraPin4000_Func_module - VM)

TSet	Command	Label	Vector	DQS0	DQS1	DQ0	DQ1	DQ2	DQ3	DQ4	DQ5	DQ6	DQ7	DQ8	DQ9	DQ10	DQ11	DQ12	DQ13	DQ14	DQ15
Tset_up4k			0	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			1	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			2	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			3	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			4	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			5	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			6	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			7	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			8	1	X	1	1	1	1	1	1	1	1	X	X	X	X	X	X	X	X
Tset_up4k			9	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			10	1	X	1	1	1	1	1	1	1	1	X	X	X	X	X	X	X	X
Tset_up4k			11	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			12	1	X	1	1	1	1	1	1	1	1	X	X	X	X	X	X	X	X
Tset_up4k			13	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			14	1	X	1	1	1	1	1	1	1	1	X	X	X	X	X	X	X	X
Tset_up4k			15	0	X	0	0	0	0	0	0	0	0	X	X	X	X	X	X	X	X
Tset_up4k			16	1	H	1	1	1	1	1	1	1	1	H	H	H	H	H	H	H	H

Multiple Time Domain UltraPin4000 Loopback Pattern in PatternTool

Note that the Pattern Explorer window shows the time domain used for each pattern file and module and allows you to select which pattern you want to view in PatternTool and select the pattern to re-execute using the right-click pull down menu option, Execute Selected Patterns.



Pattern Explorer for Multiple Time Domain Patterns

6. Use a Pattern Sets sheet to group patterns together by time domain.

When using an optional Pattern Sets sheet, follow the guidelines described in Pattern Sets for Multiple Time Domains Testing.

Pattern Sets TERADYNE

Pattern Set	TD Group	Time Domain	File/Group Name	Burst	Start Label	Stop Label	Comment
HSD_MTD		TD_HSD	.\pats_hsd1000\HSD1000_Func1.PAT	yes			
HSD_MTD			.\pats_hsd1000\HSD1000_Func2.PAT	yes			
HSD_MTD			.\pats_hsd1000\HSD1000_Func3.PAT	yes			This pattern has 2 modules
UP4K_MTD		TD_UP4K	.\pats_hsd4000\UltraPin4000_Func1.PAT	yes			
UP4K_MTD			.\pats_hsd4000\UltraPin4000_Func2.PAT	yes			
UP4K_MTD			.\pats_hsd4000\UltraPin4000_Func3.PAT	yes			This pattern has 2 modules
ALL_MTD		TD_HSD	HSD_MTD	yes			
ALL_MTD		TD_UP4K	UP4K_MTD	yes			

Multiple Time Domain Pattern Sets Sheet Setup

7. Program the Test Instances sheet to include test instances using the Functional_T VBT test instance.

The MTD_Func_T test instance will include time sets for both time domains.

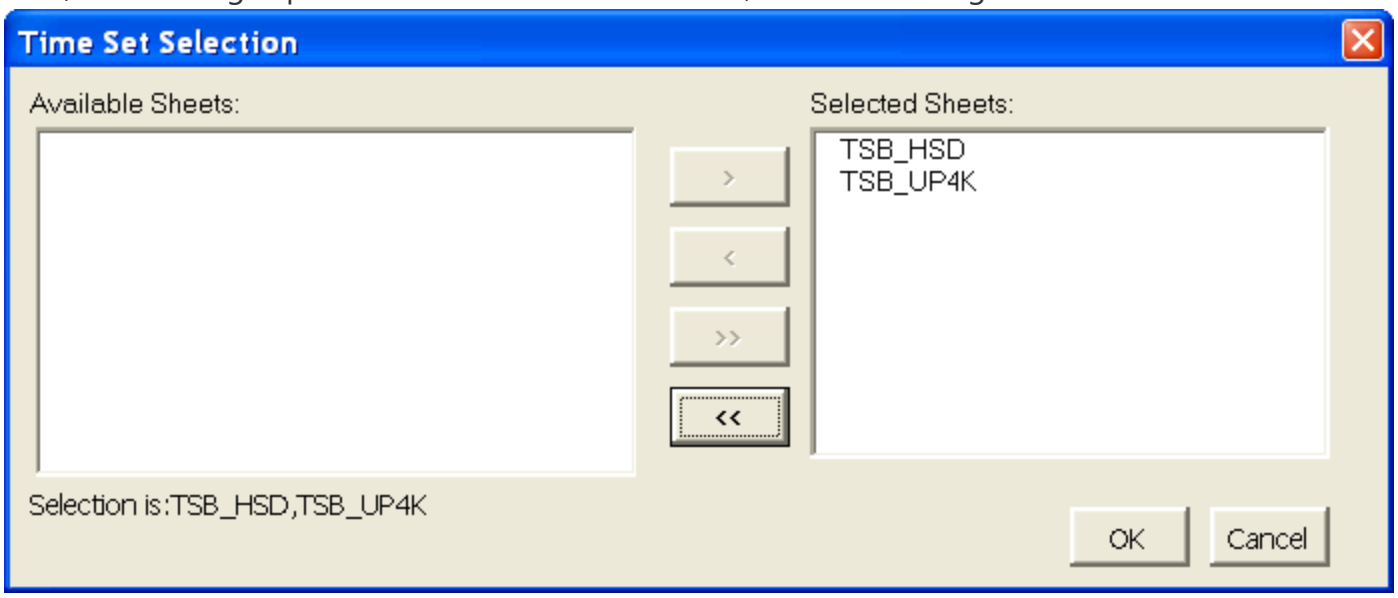
Test Instances TERADYNE

	Test Procedure			DC Specs		AC Specs		Sheet Parameters		
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels
Continuity_Template	VBT	PinPMU_T								
HSD_Func_T	VBT	Functional_T				Loopback	Sel0	TSB_HSD		Levels_HSD
UP4000_Func_T	VBT	Functional_T				Loopback	Sel0	TSB_HSD,TSB_UP4K		Levels_UP4K
MTD_Func_T	VBT	Functional_T				Loopback	Sel0	TSB_HSD,TSB_UP4K		Levels_MTD
MTD_PatSet_One	VBT	Functional_T				Loopback	Sel0	TSB_HSD,TSB_UP4K		Levels_MTD
MTD_PatSet_Domain	VBT	Functional_T				Loopback	Sel0	TSB_HSD,TSB_UP4K		Levels_MTD
MTD_PatSet_Module	VBT	Functional_T				Loopback	Sel0	TSB_HSD,TSB_UP4K		Levels_MTD

Multiple Time Domain Test Instances Sheet Setup

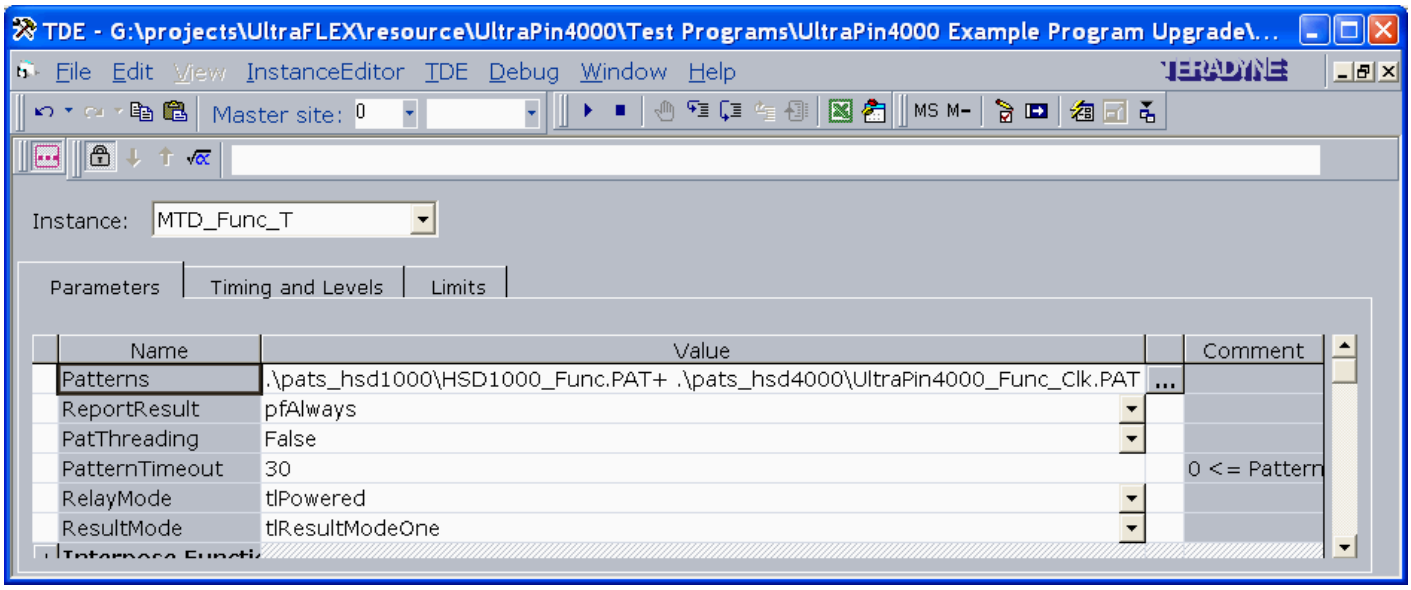
The Functional_T test instance supports options in the ResultMode parameter that allow you to choose how to report pass/fail results on specific time domains. For more information, see ResultMode.

Note that you can select or change the time sets in this sheet by right-clicking in the Time Sets field, which brings up the Time Set Selection window, shown in the figure below.



Multiple Time Domain Time Set Selection Window

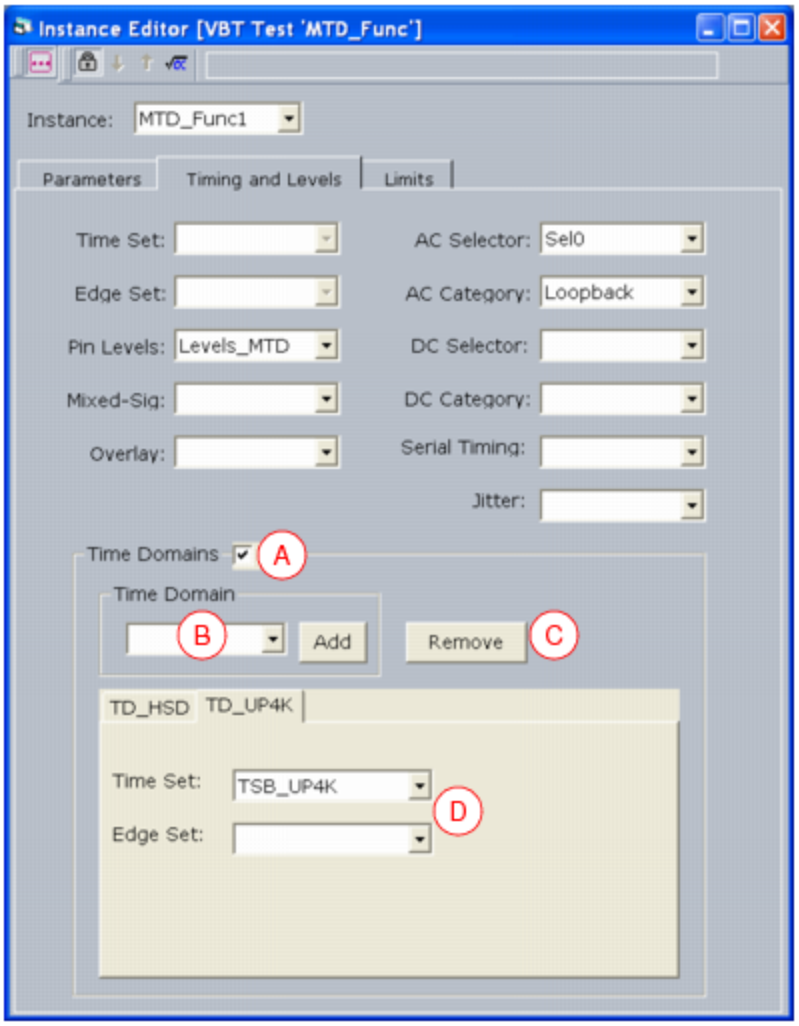
8.	Open the Instance Editor for the MTD_Func_T test instance and include the compiled patterns (.PAT files) for both time domains in the Value field of the Parameters tab. You can list a pattern or pattern set for each time domain separated by the + syntax, as shown in the figure below. Alternatively, you can list a multiple time domain pattern set.



Instance Editor Setup for the MTD_Func_T Test, Parameters Tab

Note that you can also select or change the time domains using the Time Domains area in the Timing and Levels tab. To set up multiple time domains using this interface:

- Enable the Time Domains by clicking the Time Domains check box (see A).
- Select the time domain name from the Time Domain pull down and click the Add button to include a new tab for each time domain name (see B).
- Click the Remove button to remove any unwanted time domain that is currently selected (see C).
- Select the time set and edge set name in the Time Set and Edge Set menus (see D).



Instance Editor Setup for the MTD_Func1 Test, Timing and Levels Tab

9.	Program the Flow Table sheet to include the test instances from the Test Instances sheet.

Flow Table

		Gate			Command	
Label	Enable	Job	Part	Env	Opcode	Parameter
	cont				**Test**	Continuity_Template
					Test	HSD_Func_T
					Test	UP4000_Func_T
					Test	MTD_Func_T
					Test	MTD_PatSet_One
					Test	MTD_PatSet_Domain
					Test	MTD_PatSet_Module

Multiple Time Domain Flow Table Setup

pc_mtd.9.21

< >

UltraWave Duplex MTD Example

The following example shows the elements that are included in an UltraWave test program for multiple time domains. This example describes a program that uses an MWSource in one time domain for the RX side of a device and an MWReceiver in another time domain for the TX side of the device.

The following figures show an example pin map and channel map.

Pin Map

Group Name	Pin Name	Type
	Source	Analog
	Measure	Analog
	Dig1	I/O
	Dig2	I/O
MWSourceOnly	Dig1	TimeDomain
MWSourceOnly	Source	
MWReceiveOnly	Dig2	TimeDomain
MWReceiveOnly	Measure	

UltraWave MTD Pin Map

Channel Map

DIB ID: View Mode: Signal

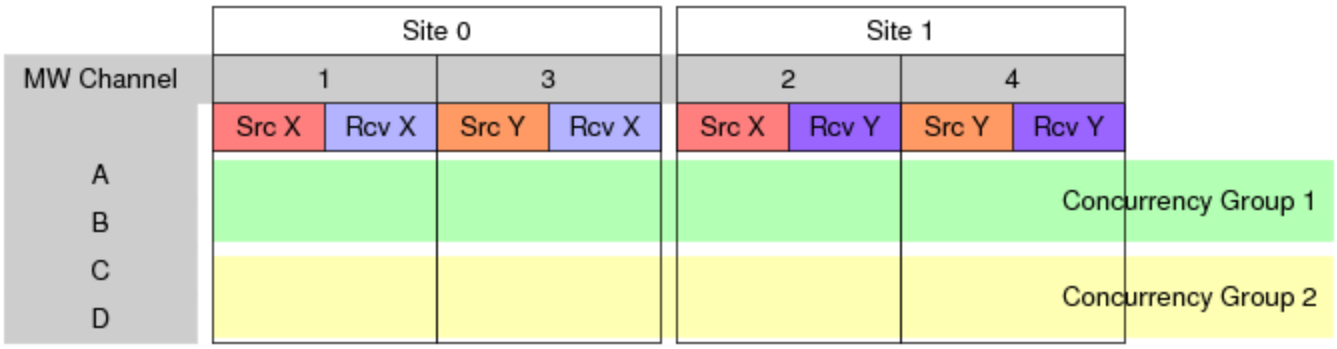
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
Source		MW	2.MW1A	2.MW2A
Measure		MW	2.MW3A	2.MW4A
Dig1		I/O	1.ch16	1.ch17
Dig2		I/O	1.ch128	1.ch129

UltraWave MTD Channel Map

For most UltraWave 12G instruments, mapping of the front end module used for each channel is not complex since the module is the channel map number. So, in this example:

- Pin Source uses front end module 1 (MW1A) for Site 0 and module 2 (MW2A) for Site 1.
- Pin Measure uses front end module 3 (MW3A) for Site 0 and module 4 (MW4A) for Site 1.

However, mapping of MWSource and MWReceiver is not as obvious. This figure below shows the best way to partition resources, mapping odd sites to the even front end modules and even sites to the odd modules. This organization expands in the same way for more sites and measure boards.



Mapping of MWSource and MWReceiver for Concurrent Test

You must also include VBT code to OnProgramLoaded:

TheHdw.MW.EnableIntraSlotMTD = True

So, the function would look something like this:

```
Function OnProgramLoaded()
    On Error GoTo errHandler
    ' Put code here
    TheHdw.MW.EnableIntraSlotMTD = True
    Exit Function

errHandler:
```

```
    HandleExecIPError "OnProgramLoaded"
End Function
```

For more information, see EnableIntraSlotMTD in the VBT Syntax Reference.

If you want to control pattern interlocking while a pattern is running, add the following to OnProgramStarted:

```
TheHdw.MW.POPInterlock = tlMWPOPInterlockEnable
```

So, the function would look something like this:

```
Function OnProgramStarted()
    On Error GoTo errHandler
    ' Put code here
    TheHdw.MW.POPInterlock = tlMWPOPInterlockEnable
    Exit Function

errHandler:
    HandleExecIPError "OnProgramStarted"
End Function
```

For more information, see POPInterlock in the VBT Syntax Reference.

The program also includes patterns defined as follows:

- One time domain would contain tester source instruments (such as ModSrc) for the RX side of the DUT

- One time domain would contain tester receive instruments (such as MWReceiver) for the TX side of the DUT

---

pc_psets.5.1

<	>

Parameter Sets (PSets)

Parameter Sets (PSets)

A parameter set (PSet) is a set of values that describes a specific instrument setup. You can call a PSet setup to program an instrument to a predefined state multiple times in a program. Using PSets can reduce test time and improve repeatability.

When you execute a PSet, it sets instrument properties to a programmed state. When you call a PSet from within a pattern, you can change the instrument setup without stopping or pausing pattern execution. PSet execution occurs when you call the PSet by name from either a pattern or VBT. You use VBT or a PSets sheet to define a PSet.

This section describes general PSet usage and programming as it applies to all instruments. It includes the following topics:

- Creating a PSet
- Loading PSets to Memory
- Applying a PSet
- Benefits of PSets
- Incremental PSet Programming
- Considerations When Using PSets

For information on instruments that use PSets, click Related Topics.

Programmable Parameters

A PSet defines a programmed state for an instrument and contains parameters such as voltage, voltage range, and amplitude. Each instrument has PSets specific to programming parameters for that instrument. When an instrument receives a PSet microcode from a pattern, it goes to the programmed state represented by that PSet.

A PSet cannot program a connection. You must create all connections required for a PSet before applying the PSet.

PSet Memory

A PSet is stored in PSet memory which exists on an instrument board and is used to program an instrument channel to a specified state when invoked. Each instrument has a certain set of PSet programmable parameters and a certain amount of PSet memory. Refer to each instrument's help for specific details on PSet memory.

< >

2015 Teradyne, Inc. All rights reserved.

pc_psets.5.2

< >

Parameter Sets (PSets) : Creating a PSet

Creating a PSet

You create a PSet using VBT language statements in the VBT editor. The formats for building PSets are:

.Add Method	Creates and initializes a software image of the PSet that requires parameters to be programmed separately.
.Item Method	Allows you to program the PSet of the given name.
.Set Method	Creates a software image of the PSet and requires all parameters to be specified.

.Add Method

The .Add method is a VBT statement that creates a PSet in PSet memory. An example for the DCVI is:

```
TheHdw.DCVI.Pins("Pin_Name").PSets.Add("PSet_Name")
```

Once this function is executed, the .Add method creates a software image of the PSet and initializes its values to a known state.

Programming the .Add method is not enough to create a PSet. You have only created a PSet name without any programmed instrument properties.

.Item Method

You use the .Item method to program an instrument's properties when using the .Add method. Each property you want programmed to a specified value must be entered into the code as shown in the following example for DCVI:

```
With TheHdw.DCVI.Pins("Pin_Name").PSets.Item("PSet_Name")
    .Current = -0.075
```

```
    .CurrentRange = 0.2
    .Voltage = -0.1
    .VoltageRange = 5
    .Mode = tlDCVIModeCurrent
    .Meter.Mode = tlDCVIMeterVoltage
    .Meter.VoltageRange = 5
End With
```

Note that any property not specified in a PSet is set to the default, which is often the same value as the reset state of the instrument. For example, if you create a PSet for a DCVI and do not specify a current, the current is set to the instrument's reset value. This may cause a test to fail.

☑ **Note:**

If you do not want an instrument parameter set to a reset value, you must program it in the PSet. Even if a property was specified earlier, when a PSet is loaded unspecified instrument states are programmed to default states. It is good programming practice, therefore, to specify all properties in a PSet.

.Set Method

The .Set method is a VBT statement that creates a PSet in PSet memory. A DCVI example is:

```
TheHdw.DCVI.Pins("Pin_Name").PSets.Item("PSet_Name").Set tlDCVIModeVoltage, 3.6, 5, 0.2, 0.2, 6, 6, False, False, False, tlDCVIMeterCurrent, 5, 0.2, 10000, 1, 0, 1000, False
```

Note when using this form of programming that you should define all parameters at once. This ensures there is no opportunity for a default value to be used accidentally. Otherwise, if something is not specified and a default value is programmed, it may not be immediately obvious and take some time to debug.

The .Set method is harder to interpret as to which parameter is being set to what value, making the .Item method a more readable programming method. However, you can use a slightly more verbose method of programming the .Set method as follows:

```
TheHdw.DCVI.Pins("Pin_Name").PSets.Item("PSet_Name").Set _
Mode:=tlDCVIModeVoltage, _
Voltage:=3.6, _
VoltageRange:=5, _
Current:=0.2, _
CurrentRange:=0.2, _
compliancerangehigh:=6, _
compliancerangelow:=6, _
conditionbitmodechange:=False, _
conditionbitcapturecomplete:=False, _
conditionbitwaveformcomplete:=False, _
metermode:=tlDCVIMeterCurrent, _
```

```
metervoltagerange:=5, _
metercurrentrange:=0.2, _
capturesamplerate:=10000, _
capturesamplesize:=1, _
captureextrasamples:=0, _
sourcesamplerate:=1000, _
sourcestoponmodealarm:=False
```

< >

2015 Teradyne, Inc. All rights reserved.

pc_psets.5.3

<	>

Parameter Sets (PSets) : Loading PSets to Memory

Loading PSets to Memory

Each instrument has a maximum number of PSets. The number of PSets loaded to memory depends on the PSet memory, and this number varies by instrument. Refer to each instrument's description.

Use an OnProgramValidated interpose function to minimize any time spent during program execution.

A PSet is loaded to hardware when the PSet is applied. A PSet can be applied in the following ways.

- Use the .Apply VBT statement
- PSet microcode is executed in a pattern

Loading of PSets into memory depends on the PSet memory size and the number of PSets. The following determines how PSet memory is handled:

- If the total number of PSets for a particular channel is less than the hardware maximum, then all PSets are loaded to that channel on first execution and are not touched.
- If the maximum number of PSets is exceeded in an entire program flow but not in a single burst, IG-XL handles loading and unloading of PSet memory as needed.
- If the maximum number of PSets is exceeded in a single pattern burst, IG-XL returns an error.

Once a PSet is loaded into memory it remains there until it is overwritten by another PSet or the PSet memory is reset. A PSet already in memory executes faster than a PSet that must be loaded to

memory.

---

pc_psets.5.4

< >

Parameter Sets (PSets) : Applying a PSet

Applying a PSet

You can apply a PSet to hardware using the following methods:

- Applying a PSet Using VBT
- Applying a PSet From a Pattern Created Using PatternTool
- Applying a PSet From a Pattern Created Using a Text Editor

Applying a PSet Using VBT

To apply a PSet using VBT language, use the .Apply method. For example:

TheHdw.DCVI.Pins("Pin_Name").PSets.Item("PSet_Name").Apply

When the .Apply method is executed, the instrument is programmed as defined in the PSet.

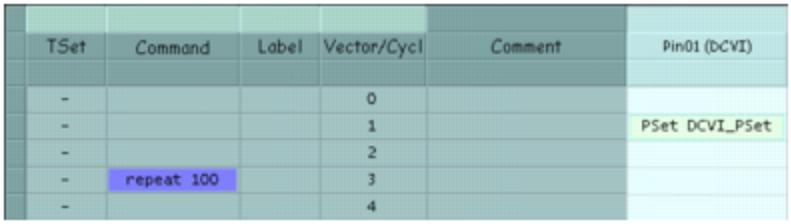
Applying a PSet From a Pattern Created Using PatternTool

To apply a PSet in a pattern created using PatternTool, use the PSet microcode and enter the PSet name in the drop-down list as shown in the figure.

Using DCVI Microcode Dialog to Select a PSet

Select the PSet name from the list and click OK. The PSet appears in the PatternTool under the instrument column as shown in the figure.

Note that you can also type in a PSet manually.



Using PatternTool to Apply a PSet

Once the line of the pattern containing the PSet microcode is executed, the PSet is invoked and the instrument is programmed as defined in the PSet.

Applying a PSet From a Pattern Created Using a Text Editor

To apply a PSet in a pattern created using a text editor, use the PSet microcode and enter the PSet name as follows:

```
(pin1:DCVI = PSet PSet_Name)      > -    ;
```

When the pattern reaches the line containing the PSet microcode, the instrument applies the PSet and takes the values defined in the PSet.

< >

2015 Teradyne, Inc. All rights reserved.

pc_psets.5.5

< >

Parameter Sets (PSets) : Benefits of PSets

Benefits of PSets

PSets provide the following benefits:

- Ease of Use
- Test Time Reduction
- Improved Test Repeatability

Ease of Use

PSets make it easier to program an instrument to the same state quickly and repeatably. This can save programming time, particularly for larger test programs.

For example, a test using the DCVI requires that you change mode and program voltage.

1. Create a PSet as follows:

```
With TheHdw.DCVI.Pins("Pin_Name").PSets.Item("DCVI_VoltageMode")
    .Current = -0.075
    .CurrentRange = 0.2
    .Voltage = -0.1
    .VoltageRange = 5
    .Mode = tlDCVIModeVoltage
    .Meter.Mode = tlDCVIMeterCurrent
    .Meter.CurrentRange = 5
End With
```

You can also set these parameters on a PSet sheet. See PSets Sheet.

2.	Invoke this PSet from VBT using the following statement:

Call TheHdw.DCVI.Pins("Pin_Name").PSets("DCVI_VoltageMode").Apply

3.	Program the instrument setup to another state using VBT or another PSet.

4.	To return to the initial state, call the first PSet by name as follows:

Call TheHdw.DCVI.Pins("Pin_Name").PSets("DCVI_VoltageMode").Apply

You do not need to program each of the parameters individually again.

Test Time Reduction

PSets improve test time by eliminating handshaking, pattern starts, and multiple statements to serially program instruments. Depending on the number of instruments required for a test and the interactions required, this time savings can be modest to significant. An example of potential test time savings is a test regulator with different regulating points where the active regulating point is determined by digital programming.

Using .Apply

Using the .Apply method can reduce test time. It is much faster to apply a PSet than to program each setting in a serialized statement when you need to change several parameters, possibly including mode. Applying a PSet takes roughly the same time as changing one parameter.

Using a Pattern

PSets used with patterns allow a high level of control across the range of instruments available. Patterns allow very time-controlled programming. PSets allow you to program different instrument setups, while microcode besides the PSet microcode allows you to program a single instrument action.

Setting an instrument property without a PSet, such as programming a voltage, requires you to stop the pattern, program the voltage, and restart the pattern. This requires test time. Using a PSet, you program the voltage to the level specified in the PSet without stopping the pattern. Instead, you build a wait time into the pattern by repeating empty vectors, allowing the instrument the time required to reach the programmed state.

Load Times

PSets are loaded on demand. If you use a VBT .Add or .Set method, the required PSet is loaded when required and then executed. If a pattern is executed with PSets, the PSets are loaded if required. To conserve test time, this load only occurs when needed, not at every run.

Improved Test Repeatability

The ability to program instrument parameters using PSets from a pattern allows for the increased repeatability of tests.

Inclusion of a pattern generator (PatGen) on each instrument board gives you precise control of all instrument boards. A primary function of a PatGen is to manage time by programming arbitrary periods of time with high precision using TSets (time sets). All PatGens in a test system follow a single reference clock, ensuring precise synchronization of all instruments in the tester, whether digital, analog, or DC.

The ability to guarantee that an instrument or group of instruments always provides the same stimulus at the same time, every time, is a key component of repeatability.

You can improve repeatability by combining the precision time control of a PatGen with the ability to set an instrument state using PSets. This combination ensures that instrumentation does the same thing at the same time from one run to the next, independent of test computer speed or operating system idiosyncrasies.

---

pc_psets.5.6

< >

Parameter Sets (PSets) : Incremental PSet Programming

Incremental PSet Programming

Since PSets are defined per channel, you can use a single PSet to program different channels with different settings. Consider a DCVI being set up with the following PSet with “vreg_out” being a pin group and “vr_vout1” and “vr_vout3 being pin names that belong to that pin group:

```
Call TheHdw.DCVI.Pins("vreg_out").PSets.Add("vreg_iload")
With TheHdw.DCVI.Pins("vreg_out").PSets.Item("vreg_iload")
    .Current = -0.075
    .CurrentRange = 0.2
    .Voltage = -0.1
    .VoltageRange = 5
    .Mode = tlDCVIModeCurrent
    .Meter.Mode = tlDCVIMeterVoltage
    .Meter.VoltageRange = 5
End With
```

When the PSet is applied, the DCVI for the specified channel is set to the programmed values.

One way to program a second and third channel is to create a new PSet for each channel. A faster and more efficient way to do this, however, uses the following statements after the first .item line:

```
TheHdw.DCVI.Pins("vr_vout1").PSets.Item("vreg_iload").Current = -0.020
TheHdw.DCVI.Pins("vr_vout3").PSets.Item("vreg_iload").Current = -0.200
```

In these statements the PSet name is the same. The pin name and the current value is different for each use of the PSet, but all other properties are programmed identically. The code would appear as follows:

```
Call TheHdw.DCVI.Pins("vreg_out").PSets.Add("vreg_iload")
With TheHdw.DCVI.Pins("vreg_out").PSets.Item("vreg_iload")
    .Current = -0.075
```

```
    .CurrentRange = 0.2
    .Voltage = -0.1
    .VoltageRange = 5
    .Mode = tlDCVIModeCurrent
    .Meter.Mode = tlDCVIMeterVoltage
    .Meter.VoltageRange = 5
End With
TheHdw.DCVI.Pins("vr_vout1").PSets.Item("vreg_iload").Current = -0.020
TheHdw.DCVI.Pins("vr_vout3").PSets.Item("vreg_iload").Current = -0.200
```

This example shows the incremental programming of a PSet as any parameter can be changed at any time in the program flow.

You can also take advantage of when a device output must be measured and then later used as an input.

For instance, a load regulation test requires an input voltage to the regulator. Nominally, that input voltage might be 4V for a 3V output voltage. If the test spec was written such that the input voltage must be 0.8V above the measured output voltage, then a PSet for the input voltage could be modified device to device such that it was always a measured output voltage plus 0.8V. The test need not be rewritten since the PSet name remains constant. Only the underlying value changes as needed.

Programming in this way retains the readability, throughput, repeatability of using PSets.

< >

pc_psets.5.7

< >

Parameter Sets (PSets) : Considerations When Using PSets

Considerations When Using PSets

PSets are a powerful way to set up test programs, and you should understand them before using them. Consider the following topics when using PSets in a program to be certain the test program operates as expected.

Waiting for Pattern Completion

When using patterns with PSets, you must wait for patterns to complete. This is actually good programming practice. With PSets and some other microcode, however, it is vital that IG-XL is aware that a pattern has completed to synchronize with the hardware.

This is accomplished in the same way a regular pattern is completed. In VBT, you can do a pattern .Run which has an implicit wait or use TheHdw.Digital.Patgen.start with TheHdw.Digital.Patgen.HaltWait. Note that this wait must be executed at some point, but the program does not necessarily have to sit idle while the pattern executes. It is acceptable to start the pattern, proceed to run other VBT language, and then do a halt wait after a sufficient period of time has elapsed.

Alarm Behavior During Patterns

Some devices require a hold state or clocking pattern to be run when conducting a series of functional tests. Normally, the pattern starts at the beginning of the test series, and it runs asynchronously as tester instruments are programmed and measurements are conducted.

An Alarm window opens when the pattern starts so that alarms for analog instruments and digital channels are reported at the end of pattern execution. This allows an analog instrument's captured data to be properly flagged with an alarm after that data is processed. IG-XL then uses this flagged data to appropriately bin devices.

The problem is that transient alarms can be reported for all tester instruments that are modified while a pattern executes. One way to modify a tester during a pattern execution is using PSets.

To deal with these transient alarms do the following:

1.	Disable all alarms for instruments modified in the pattern in VBT.

2.	Re-enable all alarms before the beginning of the measurement in VBT.

This does not prevent transient alarms, but it prevents them from being captured by IG-XL. This ensures that only alarms actively occurring at the time of a measurement are reported.
Another method where applicable is to perform all setups and measurements from within a pattern using PSets. Use the enable alarms and disable alarms microcodes around PSet declarations to avoid latching transient alarms.

< >

	2015 Teradyne, Inc. All rights reserved.	

pc_shutdowns.6.1

<	>

Shutdowns

Shutdowns

When some UltraFLEX instruments encounter conditions that are unsafe to the instrument, to the DIB, or to your device, they enter a safe state. The process of entering this state is called a shutdown. Other instruments in the tester may respond to such conditions, even if they do not cause them.

During a shutdown, an instrument ignores all messages from the tester computer and moves to the safe state, using a sequence of events designed to protect itself from further damage and from potentially dangerous conditions. Whenever an instrument shutdown occurs, all instruments that respond to the shutdown signal also perform their own specific shutdown sequences. Therefore, after a shutdown, the instruments are no longer in the state that the test programs expects. Testing stops and you must restart DataTool.

A shutdown indicates that a condition exists that could potentially damage an instrument. Repeatedly invoking a shutdown on any instrument can affect the instrument's reliability.

Shutdowns are always enabled and cannot be customized.

This section includes the following topics:

- Key Terms
- Shutdown Wire
- Shutdown Reporting
- Shutdown Sequence

Each set of instruments has its own conditions that can cause a shutdown.

For information on instruments that support shutdowns, click Related Topics.

<	>

pc_shutdowns.6.2

< >

Shutdowns : Key Terms

Key Terms

Shutdown	A sequence of events that shuts down an instrument to prevent possible damage. For example, a channel overheats due to a bad device.
SMC Fault	A sequence of events to remove power from an instrument to prevent possible damage. Indicates the tester hardware is broken.
Shutdown wire	A connection between all instrument boards that support shutdowns. This connection allows all boards to be safely shut down when a shutdown condition occurs on any board in the test system.

< >

pc_shutdowns.6.3

< >

Shutdowns : Shutdown Wire

Shutdown Wire

The shutdown wire consists of a wire bussed to all boards in the test system. Any board that can be damaged by excess terminal voltage or current or internal temperature monitors and invokes the shutdown wire.

Invoking a Shutdown

When an instrument board experiences a hazardous condition, it shuts down and grounds the shutdown wire. Any instrument that invokes a shutdown also responds to shutdowns.

Responding to a Shutdown

When an instrument senses a ground condition on the shutdown wire (from some other board), it also shuts itself down.

Some instruments have no events that invoke a shutdown but do respond to shutdowns from other instruments.

< >

pc_shutdowns.6.4

< >

Shutdowns : Shutdown Reporting

Shutdown Reporting

Shutdowns are checked at the end of measurements and patterns and when capture results are retrieved. Any shutdown that occurred is read from the hardware and reported as having occurred. You can also use the VBT property TheHdw.Alarms.ShutdownStatus to check for shutdowns.

< >

pc_shutdowns.6.5

<	>

Shutdowns : Shutdown Sequence

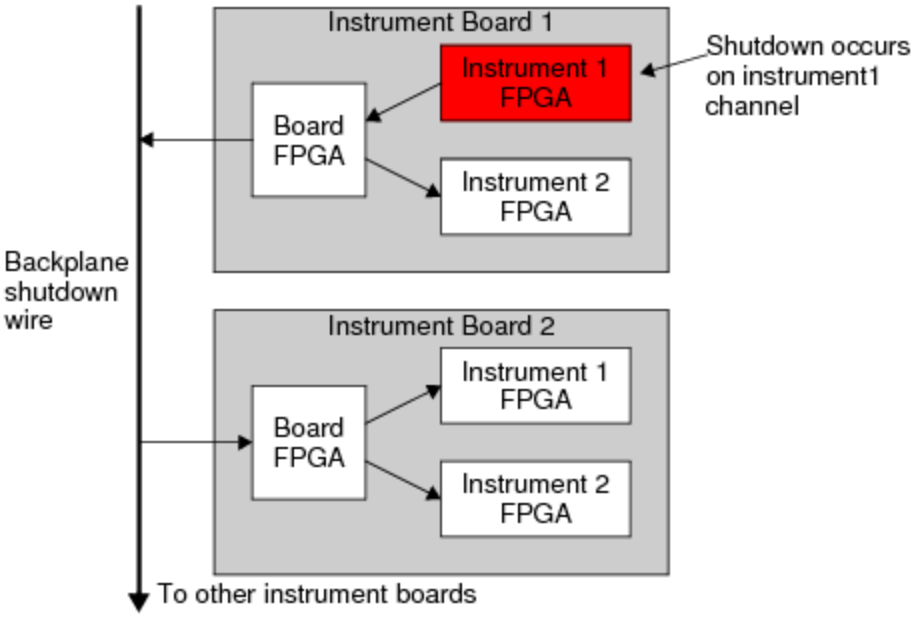
Shutdown Sequence

Every instrument that invokes shutdown must latch it, allowing IG-XL to report which instrument caused the shutdown. The support board monitors the backplane shutdown wire so IG-XL has one place to read the shutdown status for the entire tester.

When a shutdown condition occurs on an instrument board, the board starts a shutdown sequence to protect itself. The sequence is as follows:

1.	The board notifies all its instrument channels that a shutdown condition has occurred.
2.	The board grounds the shutdown wire to signal all other instruments in the test system to initiate their own shutdown sequences.
3.	TCIO and pattern communications are locked to prevent interruptions to the shutdown sequence.
4.	All channels shut down in parallel.

The figure shows a shutdown condition occurring on an instrument board



Shutdown Sequence

☑ **Note:**

An instrument undergoes the same shutdown sequence whether it invokes a shutdown itself or another instrument invokes the shutdown. For shutdown procedures for a specific instrument, see help for that instrument.

< >

2015 Teradyne, Inc. All rights reserved.

pc_shutdowns.6.6

< >

Shutdowns : Shutdown Sequence : Recovering from a Shutdown

Recovering from a Shutdown

Recovering from a shutdown requires a test system reset. If you try to rerun the test program before resetting the test system, IG-XL returns the following run-time error:

{-1} : error FLO:0263 : ** Test program could not be executed because a previous run has resulted in a fatal condition requiring a full system reset. Please close any Production Lot data collection and either cycle test system power or exit and reload the test program. **

To clear a system shutdown, two things must happen:

- You must resolve the hazardous condition causing the shutdown.
- IG-XL must call an initialization routine. This requires the following:
  - halting the test program in which the shutdown occurred
  - Stopping and restarting DataTool
  - Revalidating the test program

IG-XL can recover automatically from some shutdowns. For more information, see Automatic Shutdown Recovery.

< >

pc_shutdowns.6.7

< >

Shutdowns : Shutdown Sequence : Automatic Shutdown Recovery

Automatic Shutdown Recovery

While you should avoid all shutdowns through safe programming techniques, some shutdowns are unavoidable. When you foresee conditions that may cause a shutdown, you may want to fail the device but continue testing automatically.

IG-XL allows you to recover from shutdown conditions up to three times in a single IG-XL session. When you enable automatic shutdown recovery and a shutdown occurs, IG-XL fails all devices under test, resets, and continues testing from the beginning of the test program (the top of the flow). If a fourth shutdown occurs in the same session, testing stops and operator intervention is required. See Recovering from a Shutdown.

⚠ **Caution:**

Investigate all shutdowns and, when possible, resolve the conditions that caused them. Frequent shutdowns can damage your device, the DIB, or the tester.

To enable automatic shutdown recovery, use the following statement:

TheHdw.Alarms.ShutdownRecoveryEnable = True

To check whether a shutdown condition is present, read the following property:

TheHdw.Alarms.ShutdownStatus

To check the number of times a recovery has occurred in the current session, read the following property:

TheHdw.Alarms.ShutdownRecoveryCount

< >

pc_testerconfig.7.1

<	>

Tester Configuration

Tester Configuration

Each time you load a test program or cycle test system power, IG-XL queries ID PROMs in the tester to determine the tester configuration. It then writes the configuration information to two ASCII files. IG-XL uses this configuration information to ensure that the necessary tester hardware is present to execute the test program. If the hardware is not present, IG-XL returns validation errors, and you cannot execute the test program.

You can override the hardware configuration of the tester by creating a configuration file. For example, you may need to validate a test program on a tester that does not contain all the required hardware. A configuration file can override the hardware configuration, to allow the test program to validate.

Tester configuration is also meaningful offline. In the offline case, the tester configuration is always read from a simulated configuration file.

This section includes the following topics:

- Configuration Files
- Simulated Configuration Files (Offline Execution)
- Configuration File Format
- Sample Tester Configuration File
- Recording the Tester Configuration in STDF

pc_testerconfig.7.2

<	>

Tester Configuration : Configuration Files

Configuration Files

As output from the process described in this section, IG-XL creates two configuration files:

CurrentConfig.txt	Contains the mapped test head configuration, including any overrides. The file also contains any other information that was read from a user-created configuration file (TesterConfig.txt). A comment in the file indicates the location of any user-created configuration file that was found and used to create this file. This information is useful because the user-created file can be in several locations.
CurrentChannelMap.txt	Lists all tester channels available, based on the current configuration and shows the mapping of the tester channels to DIB channels and to signal names. This file is useful when filling in a Channel Map sheet. See Specifying Tester Channels.

IG-XL always creates these files, whether the program is running in tester mode or in offline mode. IG-XL writes the files in the folder \<installdir>\tester.

How Configuration Files Are Created

For a program running on a tester (as opposed to offline execution), IG-XL uses the following logic to read the tester configuration and any user-created tester configuration files:

1.	Read the ID PROMs from the hardware to determine what is in the test head.

2.	Check for a user-created configuration file, in this order:

\<curdir>\TesterConfig.txt
\<installdir>\tester\TesterConfig.txt
\<installdir>\bin\TesterConfig.txt
Place any user-created configuration file in the test program folder (<curdir>). In this way, other users of the tester are not affected.

3.	As soon as a file is found, IG-XL merges it with the configuration that it has read from hardware. A TesterConfig.txt file is required, because it contains the mapping of the instrument boards to tester slots. The file also lets you modify the configuration that is read from hardware.

For instance, certain instruments add internal cable connection information during installation that modifies the standard hardware configuration. In these cases, IG-XL reads the TesterConfig.txt file and adds any internal cable connection information required by one or more instruments.
On merging the hardware configuration and the configuration file, see the next section.

4.	Write CurrentConfig.txt and CurrentChannelMap.txt in \<installdir>\tester.

Any error that occurs while reading the configuration file causes initialization to fail. You must correct the error before attempting to validate or run the test program.
How TesterConfig.txt Information Is Merged
If a user-created TesterConfig.txt file is found, it does not simply replace the hardware configuration. Instead, the two are merged, on a slot-by-slot basis, and the result is a combination of the two. If a slot number in the configuration file matches a slot number in the hardware configuration, the slot information from the configuration file replaces the slot information from the hardware configuration.
In this way, you can add a slot number that does not appear in the hardware configuration. This lets you bring up a board when the EEPROM has not been programmed with the board's ID. You can force a driver to map at a specific slot, even if a different board is present at that slot.
Note that the TesterConfig.txt file does not need to duplicate any slot numbers from the hardware configuration that you want to use as is. You need to enter only modifications from the hardware configuration.
Deleting Slots
You can also force IG-XL to ignore a slot that was read from the hardware. To do this, you use the code "Delete" or "0" in the field following the instrument board name. For example:

```
1    HSD-M    Delete
```

or

```
9    BBAC-15    0
```

Specifying “Delete” or “0” effectively removes the instrument from the configuration, even if it still physically exists in the tester. If an instrument contains multiple daughter boards in the same slot, a single "Delete" or "0" on the main instrument entry removes these as well. For example, given this entry:

```
2.0   HSD-4G     974-245-00 900ea81 0835-A 5445
      HSD-4G     956-128-99 3045ca5 0820-B 5445
      PowerBoard 956-462-00 30459f2 0833-A 5445
      ChanBoard0 956-160-00 3045aa7 0835-A 5445
      ChanBoard1 956-160-00 3049336 0835-A 544
```

you need only a single "Delete":

```
2.0   HSD-4G     Delete
```

You can also “zero out” fields in the TesterConfig.txt file if you want to cancel a slot number from the hardware configuration when running online. For example, if you have HSD-4G instruments in slots 0 and 2 that you want IG-XL not to use, create a TesterConfig.txt with the following slot 0 and 2 entries:

```
0.0   HSD-4G     000-000-00 0000000 0000-A 0000
      HSD-4G     000-000-00 0000000 0000-A 0000
      PowerBoard 000-000-00 0000000 0000-A 0000
      ChanBoard0 000-000-00 0000000 0000-A 0000
      ChanBoard1 000-000-00 0000000 0000-A 0000
2.0   HSD-4G     000-000-00 0000000 0000-A 0000
      HSD-4G     000-000-00 0000000 0000-A 0000
      PowerBoard 000-000-00 0000000 0000-A 0000
      ChanBoard0 000-000-00 0000000 0000-A 0000
      ChanBoard1 000-000-00 0000000 0000-A 0000
```

< >

pc_testerconfig.7.3

<	>

Tester Configuration : Simulated Configuration Files (Offline Execution)

Simulated Configuration Files (Offline Execution)

If you are running a program offline, IG-XL always reads the tester configuration from a configuration file called SimulatedConfig.txt. The sequence is as follows:

1.	Check for a user-created configuration file, in this order:

\<curdir>\SimulatedConfig.txt

\<installdir>\tester\SimulatedConfig.txt

\<installdir>\bin\SimulatedConfig.txt

As soon as a file is found, go to the next step. A SimulatedConfig.txt file is required for offline execution.

2.	Write CurrentConfig.txt and CurrentChannelMap.txt in \<installdir>\tester.

The CurrentConfig.txt file indicates which SimulatedConfig.txt file was used in its creation.

Any error that occurs while reading the configuration file causes initialization to fail. You must correct the error before attempting to validate or run the test program.

IG-XL includes a model configuration file:

\<installdir>\bin\SimulatedConfig.txt

You can copy this file to the \tester folder and edit it to reflect the desired simulated configuration.

In addition, the \bin folder contains other configuration files for specific instruments. These are named:

SimulatedConfig_<instrument>.txt

To use one of these files, you can copy it to the \tester folder and rename it SimulatedConfig.txt.

Finally, you can copy a working CurrentConfig.txt file from the \tester folder, rename it SimulatedConfig.txt and edit to the desired simulated configuration.

Keep the SimulatedConfig.txt file in the same folder as the test program, which is the first place IG-XL looks for the file. In this way, each workbook can have its own file, and the config files do not interfere with each other.

In a certain case, you may get an “unknown resource” error. If an instrument is not included in the simulated configuration file but is in the channel map, you might think that you could set the instrument’s pins to “N/C” in the channel map and run the job. But instead this will return an “unknown resource” error on any VBT calls to the instrument. The instrument must appear somewhere in the configuration file, even in a slot that is that is not used by the channel map.

For more information, see Sample Tester Configuration File.

For instrument-specific simulated configuration file examples, click Related Topics.

< >

2015 Teradyne, Inc. All rights reserved.

pc_testerconfig.7.4

<	>

Tester Configuration : Configuration File Format

Configuration File Format

IG-XL generates the CurrentConfig.txt file using a specific format. The same format must be used by any user-created TesterConfig.txt file (for overriding the hardware configuration) or SimulatedConfig.txt file (for running offline).

To understand the format, examine the model configuration file included with IG-XL in: \<installdir>\bin\SimulatedConfig.txt

This model file has several sections, delimited by start and end tokens (for example, the tokens <TEST_HEAD> and </TEST_HEAD>). The sections are:

- The version of the configuration file (the <VERSION> token must be the first token in the file)
- The tester information section (<TESTER_INFO>), which lists tester serial number and production number, IG-XL version and build, and configuration date and time
- The cabling section (<CABLE>), which describes any required internal cable connection information between instrument boards
- The license enablers section (<LICENSE_ENABLERS>), which describes the available licensed features for each instrument, the enabled license level, the number of licenses, and a list of valid license enabling points
- The test head section, which lists slots and the boards that are in the slots, thereby mapping the board configuration in the test head

The test head section contains any number of lines with the following format:
<slot-number>[.<sub-slot>] <board-type> <id-prom-value>
The fields are separated by any number of blanks or tabs.

<slot-number>	The integer slot number, starting with 0. The slot number –1 is reserved to designate a slotless board.
<sub-slot>	The integer subslot number, to designate a rider board on the parent board (the main slot number).
<board-type>	A string that identifies the board type: for example, "HSD-M," "DC-30," or "SupportBoard." There is a defined list of valid board type identifiers, and the string entered here must match one of them. See the model SimulatedConfig.txt file in the \bin folder, which contains the valid names.
<id-prom-value>	A field that consists of four subfields separated by blanks: <boardtype> <serial> <rev> <company>

Any line or portion of a line beginning with a pound sign (#) is ignored up to the carriage return and can be used as a comment, for example, to cause a board to be ignored.
Blank lines (lines consisting of carriage returns with or without preceding white space) are also ignored. Later lines in the file can override earlier lines without error or warning.
For more information, see Sample Tester Configuration File.

< >

2015 Teradyne, Inc. All rights reserved.

pc_testerconfig.7.5

<	>

Tester Configuration : Sample Tester Configuration File

Sample Tester Configuration File

The following simulated configuration file shows the basic file format for tester configuration files and includes examples of all instrument types currently available on the UltraFLEX. This file does not represent an actual tester configuration. For the simulated configuration files used with each instrument, click the Related Topics button in Simulated Configuration Files (Offline Execution).

```
<Version>
1
</Version>

<TESTER_INFO TI 1>
TESTER SERIAL NUMBER:
TESTER PRODUCTION NUMBER:
IG-XL VERSION: 6.10.00_uflx
IG-XL BUILD: 04.24.06.05.28
CONFIG DATE: 05/05/06
CONFIG TIME: 15:26:47
</TESTER_INFO TI 1>

<TEST_HEAD TH 1>
#slot[.subslot]  Type            idprom (type serial rev company)
1.0             HSD-M           979-219-00 0000000 0000-A 0000
                HSD-M           956-361-00 0000000 0000-A 0000
2.0             MWMeasure       805-041-00 3032CDD 0741-A 5445
                MWMeasure       939-399-00 3033E3B 0740-A 5445
                FrontEnd1       599-021-00 A00001F 0744-B 5445
```

	FrontEnd2	599-021-00 A000020 0744-B 5445
	FrontEnd3	599-021-00 A000024 0744-B 5445
	FrontEnd4	599-021-00 A000023 0744-B 5445
	SplitterAtten1	599-022-00 E0000A6 0742-C 5445
	SplitterAtten2	599-022-00 E0000A4 0742-C 5445
	Receiver1	939-401-00 3033E04 0744-A 5445
	Receiver2	939-401-00 303B326 0744-A 5445
	Digitizer	939-400-00 3034076 0742-A 5445
	PowerDistribution	939-411-00 302F06E 0741-A 5445
	PowerBoard	939-402-00 303500F 0738-A 5445
3.0	HSS-6400	974-338-02 103cc14 0537-A 5445
	HSS-6400	956-422-00 3007ec5 0517-A 5445
	PowerBoard	956-426-00 300a735 0446-A 5445
	PSP	956-308-00 3008f94 0419-B 5445
	Clock	956-423-01 3009d7e 0537-B 5445
	TxRx0	956-424-01 300bb66 0537-B 0000
	TxRx1	956-424-01 300bb51 0537-B 0000
	TxRx2	956-424-01 300fd97 0537-B 5445
	TxRx3	956-424-01 300bb69 0537-B 0000
4.0	Turbo-Base	974-215-00 0000000 0000-A 0000
	Turbo-Base	956-078-00 0000000 0000-A 0000
	PowerBoard	949-984-01 0000000 0000-A 0000
	Turbo-BaseRider	949-987-02 0000000 0000-A 0000
5.0	Turbo-Frame	974-213-00 0000000 0000-A 0000
	Turbo-Frame	939-367-00 0000000 0000-A 0000
	PowerBoard	939-364-00 0000000 0000-A 0000
	Turbo-CC-1	939-321-00 0000000 0000-A 0000
	Turbo-CC-2	939-321-00 0000000 0000-A 0000
5.1	Turbo-SIM	974-228-00 0000000 0000-A 0000
	Turbo-SIM	939-366-00 0000000 0000-A 0000
6.0	GigaDigHD2	805-007-00 0000000 0000-A 0000
	GigaDigHD2	956-436-00 0000000 0000-A 0000
7.0	GigaDigHD1	805-007-10 0000000 0000-A 0000
	GigaDigHD1	956-436-00 0000000 0000-A 0000
8.0	AWG6G	805-008-01 0302D6BC 0646-D 5445
	aka: 805-008-00	
	AWG6G	956-445-01 0302D1E8 0649-D 5445
	PowerBoard	956-437-60 0302ECB9 0620-A 5445

AWG6GRider 956-443-00 0302D4F1 0629-B 5445
10.0 MWSynthRef 805-044-00 302E1A3 0733-A 5445
MWSynthRef 939-399-00 30308F7 0743-A 5445
PLLFeedback1 939-405-00 31116D2 0737-A 5445
PLLFeedback2 939-405-00 31116CC 0737-A 5445
AWG 939-390-00 311109E 0740-B 5445
Modulator 599-025-00 E0000B6 0735-B 5445
BaseBand 939-403-00 302E4C6 0743-A 5445
RefSourceFanout 599-026-00 E00002A 0738-A 5445
OutputDivider 599-024-00 E000022 0738-B 5445
PowerBoard 939-402-00 302AF21 0738-A 5445
13.0 HDVS1 974-232-00 002AD33F 0543-A 5445
HDVS1 956-071-00 0301069F 0537-A 5445
HDVS1Power 956-073-00 03010198 0534-A 5445
13.1 HDVS1Rider-0 806-740-92 03015648 0531-A 5445
aka: 974-233-00
HDVS1Rider-0 966-072-00 03015648 0531-A 5445
aka: 956-072-00
13.2 HDVS1Rider-1 806-740-92 03015655 0531-A 5445
aka: 974-233-00
HDVS1Rider-1 966-072-00 03015655 0531-A 5445
aka: 956-072-00
13.3 HDVS1Rider-2 806-740-92 03015649 0531-A 5445
aka: 974-233-00
HDVS1Rider-2 966-072-00 03015649 0531-A 5445
aka: 956-072-00
13.4 HDVS1Rider-3 806-740-92 03015671 0531-A 5445
aka: 974-233-00
HDVS1Rider-3 966-072-00 03015671 0531-A 5445
aka: 956-072-00
13.5 HDVS1Rider-4 806-740-92 03015664 0531-A 5445
aka: 974-233-00
HDVS1Rider-4 966-072-00 03015664 0531-A 5445
aka: 956-072-00
13.6 HDVS1Rider-5 806-740-92 03015657 0531-A 5445
aka: 974-233-00
HDVS1Rider-5 966-072-00 03015657 0531-A 5445
aka: 956-072-00

13.7 HDVS1Rider-6 806-740-92 03015651 0531-A 5445
aka: 974-233-00
HDVS1Rider-6 966-072-00 03015651 0531-A 5445
aka: 956-072-00
13.8 HDVS1Rider-7 806-740-92 0301566C 0531-A 5445
aka: 974-233-00
HDVS1Rider-7 966-072-00 0301566C 0531-A 5445
aka: 956-072-00
13.9 HDVS1Rider-8 806-740-92 03015669 0531-A 5445
aka: 974-233-00
HDVS1Rider-8 966-072-00 03015669 0531-A 5445
aka: 956-072-00
13.10 HDVS1Rider-9 806-740-92 03015658 0531-A 5445
aka: 974-233-00
HDVS1Rider-9 966-072-00 03015658 0531-A 5445
aka: 956-072-00
13.11 HDVS1Rider-10 806-740-92 03015667 0531-A 5445
aka: 974-233-00
HDVS1Rider-10 966-072-00 03015667 0531-A 5445
aka: 956-072-00
13.12 HDVS1Rider-11 806-740-92 0301564E 0531-A 5445
aka: 974-233-00
HDVS1Rider-11 966-072-00 0301564E 0531-A 5445
aka: 956-072-00
14.0 HexVS 974-205-00 0000000 0000-A 0000
HexVS 939-265-00 0000000 0000-A 0000
16.0 DC-30 805-002-00 0000000 0000-A 0000
DC-30 949-901-00 0000000 0000-A 0000
17.0 DC-75 805-005-04 0029F5FF 0346-B 5445
aka: 805-005-00
DC-75 949-969-00 0600B50D 0346-B 5445
DC-75Rider 949-970-00 06005D28 0225-B 5445
18.0 DCIOBoard 974-390-01 0000000 0000-A 0000
DCIOBoard 956-448-00 0000000 0000-A 0000
19.0 HexVS 974-294-00 030087D6 0514-K 5445
HexVS 956-388-00 0300B3ED 0448-A 5445
HexVSPowerBoard 939-266-02 0300A46C 0443-B 5445
HexVSAnalogBoard 956-387-00 0300ABA9 0514-A 5445

```
20.0          HSD-I             974-242-00 0000000 0000-A 0000
              HSD-I             956-128-00 0000000 0000-A 0000
21.0          VSM               979-296-00 0000000 0000-A 0000
              VSM               956-361-00 0000000 0000-A 0000
22.0          MWSynthLO         805-043-00 02C1876 0733-A 5445
              MWSynthLO         939-399-00 302B2E3 0743-A 5445
              PLLFeedback1      939-405-00 31116CA 0737-A 5445
              PLLFeedback2      939-405-00 31116D6 0737-A 5445
              OutputDivider1    599-024-00 E000014 0738-B 5445
              OutputDivider2    599-024-00 E000017 0738-B 5445
              AWG               939-390-00 311109E 0740-B 5445
              Modulator         599-025-00 E0000B1 0740-B 5445
              BaseBand          939-403-00 302DAF0 0743-A 5445
              PowerBoard        939-402-00 3031BED 0738-A 5445
23.0          BBAC-15           979-214-00 0000000 0000-A 0000
              BBAC-15           956-375-00 0000000 0000-A 0000
24.0          SupportBoard      974-220-12 0000000 0000-A 0000
              SupportBoard      956-354-00 0000000 0000-A 0000
35.0          VSM               979-296-00 0000000 0000-A 0000
              VSM               956-361-00 0000000 0000-A 0000
62.0          DistributionBoard 956-355-00 0000000 0000-A 0000
              DistributionBoard 956-355-00 0000000 0000-A 0000
66.0          VirtualDSPBrd     979-223-40 1111111 0000-A 0000
              VirtualDSPBrd     979-223-40 1111111 0000-A 0000
</TEST_HEAD TH 1>

<CABLE MWSYNTH 1>
22.LO         2.LO
22.CW/ModSrc  2.CW/ModSrc1
10.CW/ModSrc  2.CW/ModSrc2
</CABLE MWSYNTH 1>

<LICENSE_ENABLERS>
BBACCap                 48          N/A          48,46,44,42,40,38,
                                                 36,34,32,30,28,26,
                                                 24,22,20,18,16,14,
                                                 12,10,8,6,4,2,1,0
BBACCapBandwidth        15          N/A          15,3
```

BBACSrc	48	N/A	48,46,44,42,40,38, 36,34,32,30,28,26, 24,22,20,18,16,14, 12,10,8,6,4,2,1,0
BBACSrcBandwidth	15	N/A	15,3
EdgeSets	32	N/A	32,16,8,4,2,1
HDVS1Channels	768	N/A	768,288,280,272,264, 256,248,240,232,224, 216,208,200,192,184, 176,168,160,152,144, 136,128,120,112,104, 96,88,80,72,64,56, 48,40,32,24,16,8,0
HSD3200_MemoryDepth	512	N/A	512,256,128,64
HSD3200_MTO	1	N/A	1,0
HSD3200_Scan	1	N/A	1,0
HSD3200_SignatureRecv	1	N/A	1,0
HSD3200_SourceSync	1	N/A	1,0
HSD3200_Speed	3200	N/A	3200,2600,2200,2000, 1600,1400,1200,1000
HSD800_Differential	1	N/A	1,0
HSD800_DSIO	1	N/A	1,0
HSD800_FocusedCal	1	N/A	1,0
HSD800_HVPin	1	N/A	1,0
HSD800_MemoryDepth	256	N/A	256,128,96, 64,32,16,8
HSD800_MTO	1	N/A	1,0
HSD800_Scan	1	N/A	1,0
HSD800_SignatureRecv	1	N/A	1,0
HSD800_SourceSync	1	N/A	1,0
HSD800_Speed	1000	N/A	1000,800,700,600, 500,400,300,200
IG-XL_DataTool	N/A	N/A	N/A
IG-XL_TesterServiceDaemon	N/A	N/A	N/A
MainSlots	36	N/A	36,24,16,12,8,6,4
MWAWG	1	N/A	1,0
MWChannels	11	N/A	11,8,4,0
MWNoiseSource	1	N/A	1,0

MWPhaseNoise	1	N/A	1,0
MWSAM	3	N/A	3,2,1,0
MWS-Parameters	1	N/A	1,0
TimeSets	256	N/A	256,128,64,32,16,8
VHFACCapture	48	N/A	48,46,44,42,40,38, 36,34,32,30,28,26, 24,22,20,18,16,14, 12,10,8,6,4,2,1,0
VHFACCaptureSamplingRate	125	N/A	125,80
VHFACSource	48	N/A	48,46,44,42,40,38, 36,34,32,30,28,26, 24,22,20,18,16,14, 12,10,8,6,4,2,1,0
VHFACSourceSamplingRate	400	N/A	400,100

</LICENSE_ENABLERS>

< >

pc_testerconfig.7.6

<	>

Tester Configuration : Recording the Tester Configuration in STDF

Recording the Tester Configuration in STDF

IG-XL uses the file CurrentConfig.Txt to describe the tester and the installed instruments. To record information about the configuration of a test system at test time, use the function tl_WriteCurrentConfigToSTDF. This function writes CurrentConfig.txt information to Generic Data Records (GDR) to the open STDF file.

Calling this function creates two GDR entries in your STDF file:

- Test System Information
- Test Head Information

Test System Information

The first GDR stores the following information from the <TESTER_INFO> section of the current configuration file:

- Tester serial number
- Tester production number
- IG-XL version
- IG-XL build
- Configuration file date

- Configuration file time

Each item appears as a variable-length ASCII field in the GDR. For example:

```
GDR:TTESTER SERIAL NUMBER:|TTESTER PRODUCTION NUMBER:|
 TIG-XL VERSION: 8.30.00_uflx|TIG-XL BUILD: 02.01.15.03.46|
 TCONFIG DATE: 02/03/15|TCONFIG TIME: 16:44:07
```

This example shows the ATDF representation of the record, so the fields are preceded by the data type code ("T" in this example) and delimited by the field separator character (by default, this is the vertical bar).

Test Head Information

The second GDR stores information for each instrument in the <TEST_HEAD> section of the current configuration file:

- <slot-number>[.<subslot>]
- <board-type>
- ID PROM contents (in the format <boardtype> <serial> <rev> <company>)

If an instrument contains rider boards, the GDR also stores this information for each rider board. It does not include alias lines (that is, lines that begin with a.k.a.).

The format of the instrument data is the same as in CurrentConfig.txt. Each line in the configuration file appears as a variable-length ASCII field in the GDR. For example (as represented in ATDF):

```
GDR:T#slot[.subslot]    Type              idprom (type serial rev company)|
 T1.0              HSD-M             979-219-00 0000000 0000-A 0000|
 T                 HSD-M             956-361-00 0000000 0000-A 0000|
 T2.0              HSS-6400          974-338-02 0000000 0000-A 0000|
 T                 HSS-6400          956-422-01 0000000 0000-A 0000|
 T                 PowerBoard        956-426-00 0000000 0000-A 0000|
 T                 PSP               956-308-00 0000000 0000-B 0000|
 T                 Clock             956-423-01 0000000 0000-B 0000|
 T                 TxRx0             956-424-01 0000000 0000-B 0000|
 T                 TxRx1             956-424-01 0000000 0000-B 0000|
 T                 TxRx2             956-424-01 0000000 0000-B 0000|
 T                 TxRx3             956-424-01 0000000 0000-B 0000|
 T3.0              HSD-I             974-242-00 0000000 0000-A 0000|
 T                 HSD-I             956-128-00 0000000 0000-A 0000|
```

```
T       PowerBoard      956-126-00 0000000 0000-A 0000|
T       ChanBoard0      956-124-00 0000000 0000-A 0000|
T       ChanBoard1      956-124-00 0000000 0000-A 0000|
T6.0     GigaDigHD2      805-007-00 0000000 0000-A 0000|
T       GigaDigHD2      956-436-00 0000000 0000-A 0000|
T7.0     GigaDigHD1      805-007-10 0000000 0000-A 0000|
T       GigaDigHD1      956-436-00 0000000 0000-A 0000|
T8.0     AWG6G           805-008-00 0000000 0000-A 0000|
T       AWG6G           956-445-00 0000000 0000-A 0000
```

This function does not record information about cabling or licenses.

Usage

Datalogging must be enabled and configured when you call this function. The output for the datalog stream must be an STDF file. If you have not turned on datalogging or you have not specified an STDF output file, the function returns an error.

It is recommended that you call the function from OnProgramEnded. If you call the function before testing and datalogging have started, the function returns a warninng and does not write the GDRs.

To write hardware configuration to STDF, do the following:

1. Set up datalogging in your test program.

   a. Select an STDF file as the destination and assign a file name.

   In the DataCollect Setup window, check STDF File as output and supply a file name. You can set these options in your test program if you prefer.

   b. Turning datalogging on.

2. Include tl_WriteCurrentConfigToSTDF() in OnProgramEnded.

The function assembles test system configuration information into Generic Data Records and writes this information to the STDF output file.

This function returns a Boolean with the following meaning:

True	The function successfully wrote the GDR to the STDF output.
False	An error occurred when creating the record or writing it to the file.

The following example calls the function to record test system configuration information to the STDF output file.

```
Function OnProgramEnded()
    On Error GoTo errHandler
    Dim Status As Boolean
    Status = tl_WriteCurrentConfigToSTDF
    Exit Function
errHandler:
    HandleExecIPError "OnProgramEnded"
End Function
```

The example assumes that datalogging has been turned on and that an STDF file has been specified for output. This setup is usually performed in OnProgramLoaded.

Related Topics

- Configuration File Format
- Generic Data Record (GDR)
- Creating, Modifying, and Deleting a Data Stream
- TheExec.Datalog.Setup.DatalogSetup
- Program Flow Functions
